



Python 量化人的 Modern C++

从研究原型到可靠、可测量的生产系统

作者：Modern C++ 量化教程项目

组织：面向 Quant Developer 与 Research Engineer

时间：2026 年 7 月

版本：0.1

读者基础：已掌握 Python



先建立正确性，再讨论速度；先取得证据，再做优化。

前言

许多量化研究者第一次需要 C++，并不是因为 Python “不够好”，而是因为系统边界发生了变化：研究原型开始处理更大的数据，回测需要可重复的延迟，策略要接入已有的交易基础设施，或者团队希望把资源使用和失败方式变得可预测。本书要解决的，正是从“能用 Python 表达策略”到“能用 C++ 交付可靠量化组件”之间的工程距离。

本书假设你已经熟悉变量、函数、类、容器、异常、测试和虚拟环境。我们不会把一章花在解释循环是什么，而会追问：对象究竟在哪里，谁负责释放资源，复制发生在何时，接口承诺了什么，以及性能结论如何被测量证明。

贯穿全书的项目是一台小型事件驱动回测引擎。它不追求框架功能数量，而追求边界清楚：行情成为事件，策略产生订单意图，撮合器产生成交，组合模块更新状态，统计模块报告结果。每增加一个语言特性，都必须让这个系统更正确、更容易理解或更容易测量。

建议边读边运行代码。遇到性能章节时，先记录假设，再运行基准；遇到并发章节时，先画出所有权和停止协议，再写线程。真正有用的 Modern C++，不是特性清单，而是一套让成本、生命周期和契约变得可见的工具。

目录

前言	i
第一部分 从零写出 C++ 程序	1
第一章 首次构建与诊断	2
1.1 本章输出：从空目录得到一行确定结果	2
1.2 第一份源文件：每个符号都是什么意思	2
1.2.1 场景：让操作系统找到程序入口	2
1.2.2 逐行读取程序结构	2
1.2.3 立即练习	3
1.3 单文件命令与构建流水线	3
1.3.1 从空目录运行 GCC 主线	3
1.4 三个故障实验：先分类，再修复	4
1.4.1 编译失败：缺少分号	4
1.4.2 链接失败：声明存在，定义缺失	4
1.4.3 运行期失败：程序主动报告启动错误	4
1.4.4 诊断记录模板：不要从最后一行猜原因	5
1.5 最小 CMake 目标与干净构建	5
1.5.1 Windows 差异只放在命令边缘	6
1.6 自测、编码任务与面试表达	6
1.7 本章小结	6
第二章 类型、表达式与控制流：写出第一个行情筛选器	7
2.1 本章任务：从五行行情得到一个确定答案	7
2.2 对象、类型与初始化	7
2.2.1 场景：一行数据需要四个有边界的对象	7
2.2.2 语法规则与最小实验	8
2.2.3 逐步观察：Python 名称不等于 C++ 对象	8
2.2.4 失败诊断与立即练习	9
2.3 表达式、auto 与显式转换	9
2.3.1 场景：价格乘数量还不够	9
2.3.2 语法规则与可运行路径	9
2.3.3 失败诊断与立即练习	10
2.4 条件表达式、if 与 switch	10
2.4.1 场景：有效买单必须同时通过多道门	10
2.4.2 布尔规则与短路求值	10
2.4.3 失败诊断与立即练习	11
2.5 循环与作用域	11
2.5.1 场景：已知行数与未知次数是两种循环	11
2.5.2 计数循环与对象可见范围	11
2.5.3 条件循环的完整练习答案	12

2.5.4 失败诊断与立即练习	13
2.6 整合：读取、筛选并汇总固定行情	13
2.6.1 按执行顺序走一遍	14
2.7 自测、练习与面试表达	14
2.8 本章小结	15
第三章 函数、引用、const 与多文件组织	16
3.1 本章任务：把循环中的计算拆成多文件工具	16
3.2 从表达式到函数	16
3.2.1 场景：同一公式需要一个名字和边界	16
3.2.2 重构路线：一次只移动一个边界	17
3.2.3 定义、参数、返回与局部作用域	17
3.3 按值、引用与可空指针	18
3.3.1 场景：函数是否可以改动调用者	18
3.3.2 指针：允许“没有观察对象”	20
3.4 重载：同名操作的不同参数表	20
3.5 头文件、实现文件与翻译单元	20
3.5.1 场景：让不同源文件共享同一接口	20
3.6 三个诊断实验：定义数量与对象寿命	22
3.6.1 缺失定义	22
3.6.2 重复定义	22
3.6.3 悬空引用	22
3.7 从干净目录复现与章节验收	22
3.8 自测、编码任务与面试表达	23
3.9 本章小结	23
第四章 标准数据工具、算法与 CSV	24
4.1 从一系列行情认识 string、vector 与 array	24
4.1.1 场景：长度会变的数据列	24
4.2 迭代器是半开区间中的位置	25
4.2.1 场景：算法不应依赖具体容器写法	25
4.3 从普通函数到标准算法与 lambda	26
4.3.1 场景：循环的意图比循环机械步骤更重要	26
4.4 文件边界与命令行参数	28
4.4.1 场景：统计程序需要真实输入文件	28
4.5 CSV 校验：先拒绝坏行，再做统计	29
4.5.1 场景：逗号文本不等于可信行情	29
4.6 从干净目录复现输出任务	30
4.7 自测、编码任务与面试表达	31
4.8 本章小结	31
第五章 类与行情分析器	32
5.1 从公开记录认识 struct 与 enum class	32
5.1.1 场景：先把同一行字段放进一个对象	32
5.2 class 把有效状态写进构造边界	33
5.2.1 场景：行情一旦存在就必须可统计	33

5.3 成员函数维护分析器不变量	35
5.3.1 场景：只统计一个代码且至少有一行	35
5.4 多文件 CLI 与异常边界	37
5.4.1 场景：领域错误必须带回 CSV 行号	37
5.5 从干净目录交付基础篇项目	38
5.6 自测、编码任务与面试表达	38
5.7 本章小结	39
第二部分 Modern C++ 对象与组件	40
第六章 对象生命周期、RAII 与所有权	41
6.1 本章输出：让所有权状态可以被观察	41
6.2 对象生命周期、复制与移动	41
6.2.1 场景：一次行情缓冲区究竟销毁几次	41
6.2.2 完整实验与逐行轨迹	42
6.2.3 失败诊断与立即练习	43
6.3 RAII：把释放变成结构性结果	43
6.3.1 场景：提前返回也必须关闭报告文件	43
6.3.2 完整实验：文件流离开作用域后立即可读	43
6.3.3 失败诊断与立即练习	44
6.4 值、唯一所有权与共享所有权	44
6.4.1 选择顺序：先问能否直接拥有值	44
6.4.2 完整输出任务	44
6.4.3 失败诊断与立即练习	45
6.4.4 整合补充：把所有权意图写进函数接口	45
6.5 引用、裸指针与底层内存识读	46
6.5.1 场景：观察价格不等于拥有价格	46
6.5.2 完整实验：借用、C 数组与手动释放	46
6.5.3 失败诊断与立即练习	47
6.6 失败实验：让 AddressSanitizer 证明悬空	47
6.7 自测、练习与面试表达	48
6.8 本章小结	48
第七章 STL 与 ranges	49
7.1 从访问模式选择容器	49
7.1.1 场景：时间序列与按代码查询不是同一种访问	49
7.2 view 是惰性借用，不是新容器	51
7.2.1 场景：先描述筛选，再决定何时遍历	51
7.3 把算法组合成行情管道	52
7.3.1 场景：过滤、排序、查找、转换与归约各守一个职责	52
7.4 浮点归约有顺序	53
7.4.1 场景：可归约不等于满足结合律	53
7.5 独立快照与输出任务	54
7.6 自测、编码任务与面试检查点	54
7.7 本章小结	54

第八章 接口与值语义	56
8.1 先写用例，再决定类	56
8.1.1 场景：一条行情怎样变成持仓	56
8.2 值对象：构造后始终有效	57
8.2.1 场景：不让非法订单流入后续组件	57
8.3 正常缺席不是非法对象	59
8.3.1 场景：策略可以合理地不下单	59
8.4 组合是默认装配方式	61
8.4.1 场景：策略、交易所和组合协作但互不拥有对方内部状态	61
8.5 什么时候才需要运行时多态	64
8.5.1 场景：启动后按配置选择一种策略	64
8.6 独立快照与验收命令	65
8.7 自测、编码任务与面试检查点	66
8.8 本章小结	66
第九章 模板、<code>concepts</code> 与多态	67
9.1 从普通函数开始	67
9.1.1 场景：先让一种策略工作	67
9.2 函数模板：把类型变成参数	68
9.2.1 场景：阈值策略和持有策略共享一次调用逻辑	68
9.3 <code>concept</code> ：给所需能力命名	69
9.3.1 场景：把“像策略”写成可检查契约	69
9.4 约束改善入口诊断，不证明运行结果	71
9.4.1 场景：同样把整数误当策略	71
9.5 静态与动态多态的真实取舍	72
9.5.1 场景：类型何时确定	72
9.6 独立快照与验收命令	74
9.7 自测、编码任务与面试检查点	74
9.8 本章小结	75
第十章 错误处理与可观测性	76
10.1 先给失败分类，再选择机制	76
10.1.1 场景：四种“不成功”并不是同一件事	76
10.2 显式错误值保留输入上下文	78
10.2.1 场景：解析器知道字段，边界知道处置策略	78
10.3 强异常保证：失败前后公开状态相同	79
10.3.1 场景：批次第二笔失败，第一笔不能残留	79
10.4 日志、指标与追踪各回答什么	82
10.4.1 场景：一条坏行只产生一条可行动记录	82
10.5 独立快照与验收命令	84
10.6 自测、编码任务与面试检查点	85
10.7 本章小结	86

第三部分 可靠工程	87
第十一章 CMake 与依赖	88
11.1 本章输出：从源码树复现一个目标图	88
11.2 配置不是编译：先读四个阶段	88
11.2.1 场景：为什么旧可执行文件会制造假绿	88
11.3 用目标表达库、应用、测试与示例	89
11.4 PRIVATE、PUBLIC 与 INTERFACE 是传播方向	91
11.5 Debug、Sanitized 与 Release 各自回答什么	93
11.6 外部依赖也应表现为目标	93
11.7 独立快照、输出任务与验收	94
11.8 本章小结	94
第十二章 测试与调试工具	95
12.1 本章输出：一份可复核的故障诊断报告	95
12.2 从行为开始：一次红—绿—整理循环	96
12.2.1 先写能失败的账本事实	96
12.3 Release 陷阱：assert 不是测试框架	97
12.4 逻辑错误：用 GDB 检查数据流	98
12.5 编译器警告与静态分析：不运行也能发现什么	98
12.6 Sanitizer：让未定义行为留下运行证据	99
12.6.1 ASan：越界与悬空	99
12.6.2 UBSan：合法内存上的非法运算	100
12.7 症状决定工具，工具不能代替根因	101
12.8 输出任务：诊断而不是照抄命令	101
12.9 本章小结	101
第四部分 量化性能与互操作	102
第十三章 数据布局、缓存与分配	103
13.1 本章输出：正确性受控的布局报告	103
13.2 对象表示：成员之外还有填充	104
13.2.1 规则与可观察实验	104
13.3 连续访问与无关字段：AoS 还是 SoA	105
13.4 预分配：减少搬迁，不等于增加元素	106
13.5 Arena 与 std::pmr：分配策略不是所有权策略	106
13.6 False sharing：先声明缓存行假设	107
13.7 有证据后才进入章节快照	107
13.8 本章小结	108
第十四章 Benchmark 与 profiling	109
14.1 本章输出：一份可被推翻的性能报告	109
14.2 微基准协议：先保护语义，再开始计时	109
14.3 分布与指标：一个数字回答不了四个问题	110
14.4 Profiler 先定位，再用 Amdahl 约束收益	111
14.5 失败实验：结果不同，性能比较立即作废	112

14.6 从原始数字到有限结论	112
14.7 输出任务与自测	114
14.8 本章小结	114
第十五章 并发、任务与背压	115
15.1 本章输出：同一结果，三种执行模型	115
15.2 所有权与 happens-before	115
15.3 任务、future 与异常传播	116
15.4 mutex、条件变量与复合不变量	116
15.5 缓冲满就是业务协议	117
15.6 启动、停止与关闭协议	117
15.7 失败实验：让 ThreadSanitizer 建立证据	118
15.8 测量开销，不追求无锁炫技	118
15.9 输出任务与自测	119
15.10 本章小结	119
第十六章 数值稳定性与 Python 互操作	120
16.1 本章输出：先验证算法，再跨边界	120
16.2 浮点表示、比较与容差来源	120
16.3 求和顺序、补偿项与在线方差	121
16.4 金额、收益率与方差不是同一种数	122
16.5 逐元素调用、批量复制与零拷贝借用	122
16.6 dtype、连续性与缓冲所有权	123
16.7 GIL 与可选 pybind11 薄绑定	123
16.8 输出任务与自测	123
16.9 本章小结	124
A 工具链速查	125
A.1 本书验证环境	125
A.2 构建 C++	125
A.3 构建 PDF	125
A.4 日志检查	125
附录 B 术语表	126
附录 C 练习答案与思路	127
C.1 第 1 章	127
C.2 第 2 章	127
C.3 第 3 章	128
C.4 第 4 章	129
C.5 第 5 章	130
C.6 第 6 章	131
C.7 第 7 章	132
C.8 第 8 章	133
C.9 第 9 章	134
C.10 第 10 章	136
C.11 第 11 章	137

C.12 第 12 章	137
C.13 第 13 章	138
C.14 第 14 章	139
C.15 第 15 章	140
C.16 第 16 章	141
附录 D 推荐阅读与 30 天路线	143
D.1 推荐阅读	143
D.2 第 1 周：语言与所有权	143
D.3 第 2 周：接口与可靠性	143
D.4 第 3 周：性能与并发	143
D.5 第 4 周：项目与求职	143
D.6 完成判据	144
参考文献	145

第一部分

从零写出 C++ 程序

第一章 首次构建与诊断

内容提要

- ❑ 从空目录写出、编译并运行第一个 C++ 程序
- ❑ 逐个读懂 `#include`、`main`、花括号、分号和标准输出
- ❑ 区分源文件、目标文件、可执行文件以及编译与链接
- ❑ 用最小 CMake 目标完成一次全新的目录外构建
- ❑ 通过三个隔离实验辨认编译、链接和运行期故障

注 先修知识：会在终端进入目录、用文本编辑器保存文件，并能运行 Python 脚本；不要求任何 C++ 语法或编译经验。

Python 迁移边界：Python 可以直接运行 `python script.py`；C++ 源文件必须先变成当前平台的可执行文件。

常见错误与诊断：先记录完整命令和退出状态，再判断失败发生在编译、链接还是程序已经启动之后。

1.1 本章输出：从空目录得到一行确定结果

量化开发的第一个交付物不应是“编辑器里看起来像代码”，而是一条别人可以复现的构建链。本章任务从一个空目录开始，创建 `first_program.cpp` 与 `CMakeLists.txt`，最终得到：

```
quant-cpp-ready
```

完成判据有四项：单文件 GCC 命令能构建并运行；CMake 能在全新的 `build/` 目录配置和构建；程序精确输出上述文本并以状态 0 结束；你能仅凭命令停在哪一步，把三个故障实验归为编译、链接或运行期。

1.2 第一份源文件：每个符号都是什么意思

1.2.1 场景：让操作系统找到程序入口

Python 文件可以从第一条顶层语句开始执行。C++ 可执行程序需要一个约定入口，操作系统启动进程后由运行时进入名为 `main` 的函数。先完整保存下面的真实源码，不要省略标点：

Listing 1.1: 第一个可运行的 C++ 程序

```
1 #include <iostream>
2
3 int main() {
4     // Keep the first observable result exact and easy to verify.
5     std::cout << "quant-cpp-ready\n";
6     return 0;
7 }
```

1.2.2 逐行读取程序结构

`#include <iostream>` 是预处理指令。行首的 `#` 表示它在正式编译前处理；尖括号中的 `iostream` 是标准库头文件名，它让当前源文件看见标准输入输出工具的声明。这一行末尾没有分号，因为它不是 C++ 语句。

`int main()` 声明程序入口。这里先把 `int` 理解为“该入口会交回一个整数状态”，`main` 是标准规定的名称，空圆括号表示本版本暂不接收命令行参数。左花括号 `{` 开始函数体，匹配的右花括号 `}` 结束函数体；缩进帮助人阅读，但花括号才决定结构。

`//` 从当前位置开始一行注释；`/*` 与 `*/` 则界定可以跨行的块注释。编译器不会把注释内容当作要执行的语句，但块注释不能嵌套：遇到第一个 `*/` 就已经结束，因此不要用它包住一段本身含块注释的代码。`std::cout` 是标准输出流；`std::` 指明名称属于标准库命名空间，双冒号是作用域解析符。`<<` 把右侧内容送入输出流，双引号包住要输出的文本，`\n` 表示换行。字符串和数字的正式类型留到第 2 章，本章只要求能正确识读这条输出语句。

行尾的分号结束一条 C++ 语句。`return 0;` 把成功状态交回运行环境；在 Linux/WSL 中，状态 0 约定为成功，非零表示失败。分号、圆括号、花括号和引号都是语法的一部分，不能像 Python 中某些可选标点那样随意删去。

注 Python 迁移提示：Python 的缩进同时承担语法结构，`print()` 由运行时按名称查找。C++ 用花括号划分函数体，用分号结束语句，并在编译阶段解析 `std::cout`。两者都能向终端输出文本，但程序如何到达“可运行”状态并不等价。

1.2.3 立即练习

先只把源码中的 `//` 注释改成单行的 `/* ... */` 注释，预测输出应逐字符不变，再构建运行；不要嵌套块注释。然后只把输出文本改为 `quant-cpp-learning`，预测哪一行会变化并再次运行。若只改文本却出现语法错误，优先检查成对双引号、末尾分号和 `\n` 是否仍在；完成后恢复章首约定输出，确保精确测试重新通过。

1.3 单文件命令与构建流水线

1.3.1 从空目录运行 GCC 主线

在 Linux 或 WSL 中创建目录，用编辑器把清单保存为 `first_program.cpp`，然后运行：

```
mkdir ch01-first-build
cd ch01-first-build
g++ -std=c++20 -Wall -Wextra -Wpedantic \
    first_program.cpp -o first_program
./first_program
echo $?
```

`g++` 是 GCC 的 C++ 驱动程序；`-std=c++20` 选择本书语言版本；三个警告选项要求编译器报告常见可疑代码；`-o first_program` 指定输出文件名。反斜杠位于 shell 行末时表示命令在下一行继续，它不是源文件内容。`./` 明确运行当前目录中的文件，`echo $?` 查看刚结束程序的状态，应为 0。

一条 `g++` 命令隐藏了多步流水线：

source → preprocess → compile → object file → link → executable → run.

预处理展开 `#include` 等指令；编译器检查当前翻译单元并产生目标文件；链接器把目标文件和所需库组合为可执行文件；只有最后一步才真正启动你的程序。想观察中间边界，可把命令拆开：

```
g++ -std=c++20 -Wall -Wextra -Wpedantic \
    -c first_program.cpp -o first_program.o
g++ first_program.o -o first_program
./first_program
```

`-c` 表示只编译、不链接，所以第一条命令成功只能证明目标文件已经生成，不能证明最终程序的所有名称都有定义。

注Python 迁移提示：导入 Python 模块也可能失败，但普通 Python 工作流不会先生成目标文件再调用链接器。C++ 的阶段边界使一部分错误更早暴露，也要求你看清究竟是哪条命令失败，而不是把所有诊断统称为“编译不过”。

1.4 三个故障实验：先分类，再修复

故意失败源码都隔离在 `code/ch01/failures/`，不进入默认绿色构建。每个实验只验收稳定类别和非零状态，不绑定某一版 GCC 的完整英文句子。

1.4.1 编译失败：缺少分号

`missing_semicolon.cpp` 的输出语句末尾故意少了分号：

```
1 std::cout << "quant-cpp-ready\n"
2 return 0;
```

运行 `g++ -std=c++20 -c code/ch01/failures/missing_semicolon.cpp`。编译器会在读到 `return` 附近报告“期望分号”类别；箭头位置可能落在下一行，因为解析器直到看到下一个记号才确认上一条语句没有结束。根因是语法不完整，修复是在输出语句后恢复分号。立即练习：只删除 `return 0;` 后的分号，比较首个诊断位置，然后恢复文件。

1.4.2 链接失败：声明存在，定义缺失

`missing_definition.cpp` 包含一个函数声明和一次调用，但项目没有提供函数体：

```
1 double expected_notional(); // declaration only
2 // main later calls expected_notional()
```

先用 `-c` 生成目标文件会成功，因为当前文件已经知道调用的名称和返回类型；再运行：

```
g++ missing_definition.o -o missing_definition
```

链接器应报告 `undefined reference` 类别。函数声明与定义会在第 3 章从零教学；此处只建立诊断规则：能产生 `.o` 但不能产生可执行文件，问题就在链接阶段。修复是提供且参与链接的唯一定义，而不是反复修改编译警告选项。立即练习：分别记录两条命令的退出状态，确认前者为 0、后者非零。

1.4.3 运行期失败：程序主动报告启动错误

`startup_failure.cpp` 可以成功编译和链接，但启动后把错误写到 `std::cerr`，再 `return 2;`

```
1 std::cerr << "startup-error: market-data path is required\n";
2 return 2;
```

这模拟量化程序在启动检查中发现缺少行情路径。`std::cerr` 是标准错误流，便于把诊断与正常结果分开；状态 2 是本实验定义的失败协议。若可执行文件已经启动并打印上述消息，故障阶段就是运行期，不是链接。立即练习：只把状态改为 3，运行后用 `echo $?` 验证；随后恢复为 2，使自动验收重新通过。

1.4.4 诊断记录模板：不要从最后一行猜原因

编译器和链接器可能在一个根因后产生许多级联消息。可靠记录按固定顺序写五项：当前目录与完整命令；第一条非零退出的命令；失败前最后生成的产物；第一条能够解释后续消息的诊断；修复后重跑的命令和结果。只截取终端末尾一句，常会丢掉真正的文件名、行号和阶段。

阶段也能由产物反推：没有目标文件，先查预处理或编译；已有 .o 而没有可执行文件，查链接；已有可执行文件并且程序已经输出诊断，查运行期输入或业务状态。若旧产物仍在，先删除它或换用全新构建目录，避免把昨日成功生成的文件误当成今日命令的证据。

立即练习：为三个实验各写一行“命令 → 最后产物 → 退出状态 → 稳定类别”。三行应分别停在源码、目标文件和已启动进程处；若分类只写“报错”，还没有完成诊断。

1.5 最小 CMake 目标与干净构建

手写 GCC 命令适合学习阶段边界；项目增长后，应用 CMake 描述“要构建什么”。下面的文件与第一个程序放在同一目录，并参与本项目的干净构建测试：

Listing 1.2: 第一个最小 CMake 项目

```
1 cmake_minimum_required(VERSION 3.20)
2 project(ch01_first_build LANGUAGES CXX)
3
4 set(CMAKE_CXX_STANDARD 20)
5 set(CMAKE_CXX_STANDARD_REQUIRED ON)
6 set(CMAKE_CXX_EXTENSIONS OFF)
7
8 add_executable(ch01_first_program first_program.cpp)
```

cmake_minimum_required 声明最低 CMake 版本；project 建立一个使用 C++ 的项目；三个 set 选择 C++20、要求编译器支持它并关闭厂商扩展；add_executable 创建名为 ch01_first_program 的目标，并把源文件交给它。

在包含这两个文件的项目目录中运行：

```
cmake -S . -B build
cmake --build build --target ch01_first_program
./build/ch01_first_program
```

-S . 指定源目录，-B build 指定目录外构建树；源码与生成文件因此不会混在一起。第一次运行前 build/ 不应存在，这叫干净构建。若只删除可执行文件而保留旧缓存，不能证明项目从零可配置。验收完成后可删除整个 build/，再重复三条命令。

注Python 迁移提示：Python 项目常用 pyproject.toml 描述包、用虚拟环境隔离依赖，并由解释器直接运行入口；这里的 CMakeLists.txt 描述原生构建目标。cmake -S . -B build 只读取项目并生成当前平台的构建系统，不会运行程序；cmake --build build 才调用编译器与链接器。CMake 因而更像跨平台的构建描述层，不是 C++ 解释器或包管理器。

做一次只有一个变量的失败练习：在项目副本里只把 add_executable 中的 first_program.cpp 改为不存在的 missing.cpp，并把配置输出写入全新的 build-broken/。预期 cmake -S . -B build-broken 在配置或生成阶段以非零状态结束，稳定类别是 Cannot find source file，而且不会产生可运行目标。把文件名恢复后删除 build-broken/，从空构建目录重跑配置、构建和程序；若仍失败，先核对源目录是否同时包含清单中的两个文件，再读第一条 CMake 诊断，而不是改编编译器选项。

1.5.1 Windows 差异只放在命令边缘

正文主线使用 Linux/WSL 与 GCC。在 PowerShell 中，CMake 的前两条命令保持不变；单配置生成器的程序通常写作 `.\build\ch01_first_program.exe`，Visual Studio 等多配置生成器可能位于 `build\Debug`。直接使用 MSVC 时警告选项是 `/W4 /permissive-`，而不是 GCC 的 `-Wall -Wextra -Wpedantic`。不要维护两套教学流程：以 `cmake --build` 屏蔽生成器细节，需要 Linux sanitizer 或书中精确 shell 命令时使用 WSL。

1.6 自测、编码任务与面试表达

1. `#include <iostream>` 在哪一阶段处理？为什么行末没有分号？
2. `int main()` 的名称、圆括号、花括号和返回状态分别承担什么职责？
3. `std::cout << "text\n";` 中，`std::`、`<<`、引号、转义和分号分别表示什么？
4. 单条 `g++` 命令隐藏了哪些阶段？`-c` 为什么能隔开编译与链接？
5. 缺分号、缺定义和状态 2 分别在哪个阶段失败？每种修复应改变什么？
6. CMake 的源目录和构建目录为什么应分开？怎样证明一次构建是干净的？
7. Windows 与 Linux 主线最值得记录的三个命令差异是什么，哪些心智模型不变？

编码任务：

1. 从空目录重建两个文件，先用 GCC 单文件命令得到 `quant-cpp-ready`，再用 CMake 在全新 `build/` 中得到同样结果。
2. 依次运行三个故障实验，为每个实验记录失败命令、阶段、非零状态、稳定诊断类别、根因和恢复步骤。
3. 在第一个程序中分别增加一行 `//` 注释和一段不嵌套的 `/* ... */` 注释，证明输出完全不变；再故意删除输出语句的分号并恢复。

问题 1.1 面试检查点 用 90 秒回答：“从 `first_program.cpp` 到正在运行的进程发生了什么？”回答必须按预处理、编译、链接、启动排序，并各给一个可观察产物或失败类别；最后说明为什么“编译成功”不一定意味着“程序可运行且业务成功”。

1.7 本章小结

第一个 C++ 程序已经建立最小闭环：头文件提供声明，`main` 提供入口，花括号和分号构成语法结构，标准流产生可观察输出，状态码向环境报告成功或失败。GCC 把源码经过预处理、编译和链接变成可执行文件，CMake 则把同一目标描述为可重复的目录外构建。诊断时不要只读最后一行错误：先确认停在哪条命令、是否已有目标文件、是否已有可执行文件、程序是否已经启动，再处理该阶段的首个根因。

第二章 类型、表达式与控制流：写出第一个行情筛选器

内容提要

- ❑ 把 Python 的“名称绑定对象”心智模型切换为 C++ 的“声明对象并初始化”
- ❑ 选择基本类型、定宽整数、字面量和 auto，并识别窄化
- ❑ 用算术、比较、布尔表达式和显式转换计算行情指标
- ❑ 用 if/else、switch、for 和 while 控制执行路径
- ❑ 从标准输入读取固定行情，筛选有效买单并汇总名义金额

注 先修知识：已完成第 1 章，能在 Linux/WSL 中编译、链接并运行单文件程序；本章不要求会自定义函数、容器、类、引用或指针。

Python 迁移边界：Python 变量是可重新绑定的名称；C++ 声明会创建具有固定类型和作用域的对象，复制一个基础类型对象会得到独立的值。

常见错误与诊断：先区分编译诊断与运行结果，再检查初始化、整数除法、隐式转换、分支边界和循环是否终止。

2.1 本章任务：从五行行情得到一个确定答案

研究原型里，一条筛选可能只是几行 `DataFrame` 表达式；面试白板题或低延迟数据入口却常要求你明确说明：每个字段是什么类型，坏数据走哪条分支，循环何时停止，累计值如何更新。本章的输出任务只使用基础语法，不借助尚未教学的自定义函数、结构体或容器。

程序从标准输入读取一个行数和五行数值行情。每行依次是“证券编号、方向代码、价格、数量”；方向 1 表示买，2 表示卖，其他代码无效。程序只选中编号、价格、数量均为正的买单，然后打印选中数、拒绝数、名义金额总和与每条选中记录的平均名义金额。

完成本章后，下面的输入必须得到逐字符一致的核心数值结果：

```
5
101 1 100.5 10
102 2 50.0 4
103 1 25.0 0
104 1 10.25 3
105 9 12.0 5
```

```
selected=2 rejected=3 buy_notional=1035.75 average=517.875
```

这就是本章的验收标准：不是“看懂语法”，而是能在不复制最终答案的前提下重写程序、解释每个对象的状态变化，并用给定数据复现输出。

2.2 对象、类型与初始化

2.2.1 场景：一行数据需要四个有边界的对象

读入一行行情时，程序需要保存证券编号、方向、价格和数量。如果这些值没有确定的初始状态，后面的筛选就可能读取垃圾值；如果价格被随意塞进整数，小数部分会悄悄丢失。C++ 因而要求先回答“创建什么类型的对象”，再讨论计算。

2.2.2 语法规则与最小实验

最常见的声明由“类型、对象名、初始化器、分号”组成。下面不是省略外壳的片段，而是参与 C++20 构建并由 CTest 检查输出的完整程序：

Listing 2.1: 对象、字面量与独立复制

```

1  #include <cstdint>
2  #include <iostream>
3
4  int main() {
5      int side_code{1};
6      double price{100.5};
7      std::int64_t quantity{10};
8      bool is_buy{true};
9      char venue{'X'};
10
11     int original{10};
12     int copied{original};
13     original = 11;
14
15     std::cout << "side=" << side_code << " price=" << price
16               << " quantity=" << quantity << " buy=" << is_buy
17               << " venue=" << venue << " changed=" << original
18               << " copied=" << copied << '\n';
19 }
```

`int` 保存整数，`double` 保存双精度浮点数，`bool` 只有 `true` 和 `false` 两种逻辑状态，`char` 保存一个字符。`std::int64_t` 来自 `<cstdint>`，在提供它的平台上恰有 64 位；当数量或时间戳需要明确位宽时，它比“希望 `long` 足够大”更可检查。

花括号 `{}` 同时标记对象从这里开始生命周期，并拒绝一部分会丢信息的窄化。空花括号执行值初始化：`int selected{}`；得到零，`bool is_buy{}`；得到 `false`。相反，局部基础类型 `int selected`；没有可靠的业务初值；读取它不是“随机但可用”，而是程序错误。

字面量本身也有类型。1 通常是 `int`，1.0 是 `double`，`true` 是 `bool`，单引号中的 `'X'` 是字符字面量，双引号中的 `"side="` 是字符串字面量。`'\n'` 表示换行字符，它与包含两个可见字符的 `"\n"` 不同。因此 `double price{100}`；可以精确表示该值，而 `int price{100.5}`；会在编译阶段被拒绝。

为了让本章不依赖读者猜测程序外壳，这里逐项说明：`#include <cstdint>` 与 `#include <iostream>` 让当前源文件看见所需标准库声明；`int main()` 定义程序入口，`int` 是返回状态的类型，空圆括号表示这个版本不接收命令行参数，随后的一对花括号包住入口函数体。`std::cout` 是标准输出流，`<<` 依次把右侧的字符串字面量和对象值送入流。`std::` 指明标准库名称所在的命名空间。最后一个右花括号结束 `main`；走到这里等价于成功返回状态 0。第 3 章才会系统教学如何定义和调用自己的函数。

2.2.3 逐步观察：Python 名称不等于 C++ 对象

在 Python 中，赋值把名称绑定到对象；两个名称可以指向同一个可变列表：

```

1  a = [10]
2  b = a
3  a[0] = 11
4  print(b[0])          # 11
```

在上面的完整 C++ 程序中，`original` 与 `copied` 是两个独立的 `int` 对象。`copied{original}` 用 `original` 当时的值初始化新对象，之后把 `original` 赋值为 11，不会改动 `copied`；可观察输出同时包含 `changed=11` 与 `copied=10`。

等号在对象创建之后出现时是赋值，不是初始化。对象的类型仍是 `int`，不能像 Python 名称那样下一行重新绑定成字符串。共享和借用当然可以在 C++ 中表达，但要用后续章节才教学的引用、指针或拥有型对象，不能从普通按值声明中猜出来。

注Python 迁移提示：两种语言都允许把已有值用于新变量；边界在于 Python 名称通常保存对象引用，而本章的 C++ 基础类型声明创建独立对象。Python 类型标注通常不改变运行期绑定规则，C++ 对象类型则直接限制初始化、运算和存储表示。

2.2.4 失败诊断与立即练习

故意失败文件 `code/ch02/failures/narrowing.cpp` 包含如下核心语句：

```
1 int rounded_price{100.5}; // intentional compile failure
```

用 `g++ -std=c++20 code/ch02/failures/narrowing.cpp` 单独触发它。预期的是“列表初始化拒绝窄化转换”这一诊断类别，而不是某一版 GCC 的完整英文措辞。根因是从 `double` 字面量到 `int` 会丢掉小数；修复不是换成圆括号躲开检查，而是先决定业务舍入规则，再显式转换。

立即练习：分别编译 `double price{10};` 与 `int quantity{10.0};`。前者的整数常量 10 能被 `double` 精确表示，可以通过；后者从浮点类型转为整数，即使写成 10.0 也属于列表初始化禁止的窄化，必须被拒绝。再把它改成 10.5，说明同一诊断规则，而不是误记成“没有小数部分就可以”。

2.3 表达式、auto 与显式转换

2.3.1 场景：价格乘数量还不够

名义金额是价格与数量的乘积，平均值还要除以选中记录数。这里会同时遇到运算符优先级、整数除法、结果类型和除零边界。Python 常把数值提升留给运行时；C++ 会先根据操作数类型决定表达式类型。

2.3.2 语法规则与可运行路径

算术运算符包括 `+`、`-`、`*`、`/` 和整数余数 `%`。乘除先于加减；括号既能改变顺序，也能让审阅者看清意图。比较运算 `<`、`<=`、`>`、`>=`、`==`、`!=` 产生 `bool`。

下面的完整程序由构建目标 `ch02_expressions` 编译，并输出三个可独立核算的结果：

Listing 2.2: 算术、类型推导与显式转换

```
1 #include <stdint>
2 #include <iostream>
3
4 int main() {
5     double price{100.5};
6     std::int64_t quantity{10};
7     int selected{2};
8
9     auto notional = price * static_cast<double>(quantity);
10    double average = notional / static_cast<double>(selected);
11    int integer_division = 5 / 2;
```

```

12
13     std::cout << "notional=" << notional << " average=" << average
14         << " integer_division=" << integer_division << '\n';
15 }

```

`auto` 让编译器从初始化表达式推导对象类型；它不是 Python 式动态类型。这里推导结果仍固定为 `double`。必须同时写初始化器，不能写孤立的 `auto notional`；。当类型本身承载业务信息时仍应显式写出，例如数量使用 `std::int64_t`，不要为了少打字全部改成 `auto`。

`static_cast<double>(quantity)` 明确表达“此处要以浮点数参与计算”。对本例的乘法，即使不写转换，`double` 价格也会促使数量转换为 `double`；保留显式转换是为了把边界写给读者，并为随后的除法建立习惯。程序依次得到 `notional=1005`、`average=502.5` 和 `integer_division=2`；最后一个结果来自两个 `int` 的 `5 / 2`。

复合赋值 `buy_notional += notional`；等价于按该运算规则更新左侧对象；`++selected`；把整数加一。二者都会修改左侧对象；如何声明只读对象留到第 3 章。

注Python 迁移提示：Python 的 `/` 总是产生浮点除法，`//` 才是向下取整除法；C++ 的 `/` 取决于两侧类型，两个整数相除先得到整数结果。`auto` 只省略静态类型拼写，不允许对象之后换类型。

2.3.3 失败诊断与立即练习

若平均值意外变成 602，先打印或说明除法两侧的类型，不要先怀疑格式化。另一个常见失败是选中数为零时仍做除法；浮点除零可能得到无穷而继续污染结果，整数除零则是更严重的运行期错误。正确做法是在控制流中显式保护分母。

立即练习：分别预测并运行 `7 / 3`、`7.0 / 3`、`static_cast<double>(7) / 3`。写出实际结果，并指出哪一步发生类型转换。

2.4 条件表达式、if 与 switch

2.4.1 场景：有效买单必须同时通过多道门

本章规则要求证券编号、价格、数量都为正，而且方向必须是买。任何一项失败都拒绝该行。把条件拆清楚比写一个“聪明”的长表达式更适合面试解释和生产审阅。

2.4.2 布尔规则与短路求值

逻辑与写作 `&&`，逻辑或写作 `||`，逻辑非写作 `!`。`a && b` 只有两侧都为真才为真，并且当 `a` 已为假时不会求值 `b`；`a || b` 在 `a` 已为真时不会求值 `b`。这种短路行为既能表达保护条件，也意味着右侧可能根本不执行，不能偷偷依赖它产生副作用。

完整分支实验同时包含布尔表达式、`switch` 和 `if/else`，给定内置行情时输出 `selected`：

Listing 2.3: 离散方向分派与有效性筛选

```

1  #include <cstdint>
2  #include <iostream>
3
4  int main() {
5      std::int64_t symbol_id{101};
6      int side_code{1};
7      double price{100.5};
8      std::int64_t quantity{10};
9

```



```

10  bool is_buy{};
11  switch (side_code) {
12      case 1:
13          is_buy = true;
14          break;
15      case 2:
16          is_buy = false;
17          break;
18      default:
19          is_buy = false;
20          break;
21  }
22
23  bool valid_row = symbol_id > 0 && price > 0.0 && quantity > 0;
24  if (is_buy && valid_row) {
25      std::cout << "selected\n";
26  } else {
27      std::cout << "rejected\n";
28  }
29  }

```

`if` 后的圆括号放条件：条件为真执行第一对花括号，为假执行 `else` 的花括号。即使分支只有一行，本书也保留花括号，避免后来新增语句时出现视觉缩进与真实控制流不一致。

方向代码是少量离散整数，适合用清单中的 `switch` 明确列出分支。`case` 标签必须是可在编译期确定的离散值。`break` 离开当前 `switch`；漏写时会继续执行下一标签，这叫贯穿，除非有明确意图和注释，否则通常是缺陷。`default` 接住未列出的代码，使新出现的脏数据不会悄悄被当成买单。

注Python 迁移提示：Python 的 `if/elif/else` 与 C++ 分支目标相似，但 C++ 用圆括号和花括号界定结构，并要求条件可转换为布尔值。C++ `switch` 不是 Python `match` 的完整对应物：这里只把它当作离散整数分派，不期待结构模式匹配。

2.4.3 失败诊断与立即练习

`if (side_code = 1)` 是赋值，不是比较：对象被改成 1，条件随后为真。高警告级别通常会提示这一可疑写法。修复为 `==`，并保持 `-Wall -Wextra -Wpedantic`。另一个边界是把 `price >= 0.0` 写成有效规则，这会错误接纳零价格：先从业务条件写出真值表，再翻译成表达式。

立即练习：把有效规则改成“买单且价格至少 10，数量在 1 到 100（含端点）”。用恰好位于 10、1、100 的三条记录验证端点，再用价格 9.99 验证拒绝路径。

2.5 for、while 与作用域

2.5.1 场景：已知行数与未知次数是两种循环

输入开头已经给出行数，适合用 `for` 精确处理五次；风险预算能接纳多少笔交易要到条件失败时才知道，适合用 `while`。选择循环形式是在表达终止依据，不只是个人风格。

2.5.2 计数循环与对象可见范围

计数循环也有独立、可构建的最小程序。它检查 0 到 4 的余数，选中三个偶数行：

Listing 2.4: 计数循环与块作用域

```

1  #include <iostream>
2
3  int main() {
4      int selected{};
5      for (int row{}; row < 5; ++row) {
6          int remainder = row % 2;
7          if (remainder == 0) {
8              ++selected;
9          }
10     }
11
12     std::cout << "selected=" << selected << '\n';
13 }

```

圆括号内由两个分号分隔的三个部分依次是：只执行一次的初始化 `int row{};`；每轮开始前检查的条件 `row < 5`；每轮末尾执行的更新 `++row`。`row` 因而依次为 0、1、2、3、4，共五轮；若误写成 `row <= 5` 就会多处理一轮。

每轮先用 `row % 2` 计算余数，并创建本轮的 `remainder`。余数为零时，`if` 分支把循环外的 `selected` 加一，因此第 0、2、4 轮共更新三次并输出 `selected=3`。花括号还建立作用域：`remainder` 每轮重新创建并在该轮末尾销毁，`selected` 则跨轮保存累计值；循环结束后也不能访问初始化部分声明的 `row`。把对象放在“足够用的最小作用域”能减少误用，也让读者更容易追踪状态。

2.5.3 条件循环的完整练习答案

当循环次数由状态决定时，把条件放在 `while` 后。下面的完整答案从 3200 元风险预算中反复扣除每笔 750 元，只有预算足够时才进入下一轮：

Listing 2.5: 用 while 计算可接纳交易数

```

1  #include <iostream>
2
3  int main() {
4      double remaining_budget{3200.0};
5      double risk_per_trade{750.0};
6      int accepted_trades{};
7
8      while (remaining_budget >= risk_per_trade) {
9          remaining_budget -= risk_per_trade;
10         ++accepted_trades;
11     }
12
13     std::cout << "accepted_trades=" << accepted_trades
14               << " remaining_budget=" << remaining_budget << '\n';
15 }

```

初始预算 3200；四轮后剩 200，第五轮开始前 `200 >= 750` 为假，因此接纳数为 4、剩余预算为 200。循环体必须让条件朝终止方向变化；若忘记扣减预算，就会形成无限循环。

注Python 迁移提示：两种语言的 `while` 都在每轮前检查条件。Python 常写 `for row in range(n)`；C++ 的经典 `for` 把初始化、条件和更新显式放在一处。本章还没有容器，所以暂不使用范围 `for`，它会在第 4 章教学。

2.5.4 失败诊断与立即练习

循环结果差一条时，手工列出“进入本轮前的 `row`、条件结果、本轮后的 `row`”三列，比盯着代码更快。程序长时间不结束时，检查每条分支是否都会更新终止状态，以及输入失败后是否还在重复使用旧值。

立即练习：把风险预算改为 2999、每笔风险改为 1000。先预测循环次数与余数，再运行验证，结果应为 `accepted_trades=2 remaining_budget=999`。

2.6 整合：读取、筛选并汇总固定行情

下面是本章输出任务的完整源码。它只组合已经教学的对象、初始化、表达式、转换、分支、循环和作用域。这里补齐输入与错误流规则，不要求读者从旧版第 1 章猜语法：`std::cin >> row_count` 把以空白分隔的文本转换后写入对象，多个 `>>` 可以从左到右连续提取；输入失败时流状态在条件中为假，前缀 `!` 将它取反。`std::cerr` 用于错误消息，正常结果仍送到已经教学的 `std::cout`。

Listing 2.6: 固定行情筛选与汇总

```

1  #include <cstdint>
2  #include <iostream>
3
4  int main() {
5      int row_count{};
6      if (!(std::cin >> row_count) || row_count < 0) {
7          std::cerr << "invalid row count\n";
8          return 1;
9      }
10
11     int selected{};
12     int rejected{};
13     double buy_notional{};
14
15     for (int row{}; row < row_count; ++row) {
16         std::int64_t symbol_id{};
17         int side_code{};
18         double price{};
19         std::int64_t quantity{};
20         if (!(std::cin >> symbol_id >> side_code >> price >> quantity)) {
21             std::cerr << "invalid market row " << row << '\n';
22             return 1;
23         }
24
25         bool is_buy{};
26         switch (side_code) {
27             case 1:
28                 is_buy = true;
29                 break;
30             case 2:
31                 is_buy = false;
32                 break;
33             default:

```

```

34     is_buy = false;
35     break;
36 }
37
38 bool valid_row = symbol_id > 0 && price > 0.0 && quantity > 0;
39 if (is_buy && valid_row) {
40     auto row_notional = price * static_cast<double>(quantity);
41     buy_notional += row_notional;
42     ++selected;
43 } else {
44     ++rejected;
45 }
46 }
47
48 double average{};
49 if (selected > 0) {
50     average = buy_notional / static_cast<double>(selected);
51 }
52
53 std::cout << "selected=" << selected << " rejected=" << rejected
54           << " buy_notional=" << buy_notional << " average=" << average
55           << '\n';
56 }

```

2.6.1 按执行顺序走一遍

1. row_count{} 先得到零；输入成功后变成 5。若读取失败或行数为负，程序向错误流报告并返回非零状态。
2. 三个累计对象从零开始。它们声明在循环外，因此每轮的更新会保留到下一轮。
3. 每轮创建四个字段对象并读取一行。第 1 行方向为买、各字段为正，行名义金额为 $100.5 \times 10 = 1005$ 。
4. 第 2 行是卖，第 3 行数量为零，第 5 行方向代码未知，三者都走拒绝分支。第 4 行贡献 $10.25 \times 3 = 30.75$ 。
5. 循环结束时总额为 $1005 + 30.75 = 1035.75$ ，选中数为 2。分母保护通过后，平均值为 $1035.75/2 = 517.875$ 。
6. 输出流依次写出标签与对象值，所以得到本章开头约定的确定字符串。

在 Linux/WSL 中，从项目根目录运行：

```

cmake -S . -B build
cmake --build build --target ch02_market_filter
./build/ch02_market_filter < code/ch02/data/market_rows.txt
ctest --test-dir build -R ch02_ --output-on-failure

```

Windows 的生成器可能把程序放到配置子目录，或在文件名后加 .exe；源码行为和测试期望不变。CTest 的预期字符串是独立写入的已知结果，不调用被测程序的算法重新计算自己。

2.7 自测、练习与面试表达

先不看答案，逐项给出定义、使用时机、代价或失败例子：

1. 初始化和赋值分别发生在对象生命周期的什么位置？`int x;` 与 `int x{}`；对局部对象有何差异？
2. 为什么 `auto` 不是动态类型？何时宁可显式写 `std::int64_t`？
3. 解释 `5 / 2` 与 `5.0 / 2` 的结果，并给出一个平均值被整数截断的失败例子。

4. `&&` 的短路规则是什么？它怎样保护右侧表达式，又可能怎样隐藏副作用？
5. `switch` 中漏写 `break` 会怎样？为什么仍应保留 `default`？
6. 分别给出适合 `for` 和 `while` 的终止依据，并说明一个无限循环的根因。

编码练习：

1. 从空文件开始重写行情筛选器，不复制清单。先只输出 `selected` 和 `rejected`，再加入总额与平均值；使用给定输入达到本章约定输出。完整可运行答案是 `code/ch02/market_filter.cpp`。
2. 写一个 `while` 程序：风险预算足够时接纳一笔固定风险交易，最后打印接纳数和余数。用 3200 与 750 得到 4 和 200。完整可运行答案是 `code/ch02/exercises/risk_budget_solution.cpp`。
3. 单独编译窄化失败实验，记录失败阶段和诊断类别；然后提出两种修复：改变目标类型，或在明确舍入政策后显式转换。不要把关闭警告当成修复。

问题 2.1 面试检查点 用 90 秒回答：“Python 名称绑定与 C++ 对象初始化有何不同？为什么花括号、显式类型和短路条件对行情入口有价值？”答案至少包含一个可观察输出、一个编译期失败和一个循环边界，而不是只罗列术语。

2.8 本章小结

C++ 程序从具有固定类型和作用域的对象开始；初始化建立初始状态，表达式根据操作数类型产生结果，显式转换标出有意的类型边界。`if` 和 `switch` 选择路径，`for` 和 `while` 重复路径，花括号同时限定控制结构与对象可见范围。你现在已经能只用基础语法完成一个有输入、有脏数据分支、有循环汇总且可测试的量化小程序。后续章节会在这一基础上加入函数、资源生命周期和标准容器。

第三章 函数、引用、const 与多文件组织

内容提要

- ❑ 从重复计算中提取函数，读懂声明、定义、调用、参数和返回值
- ❑ 比较按值、引用和可空指针调用者对象与寿命的可观察影响
- ❑ 用重载表达同一概念的不同参数形式，并辨认候选选择边界
- ❑ 用头文件、实现文件、include guard 和命名空间组织多个翻译单元
- ❑ 诊断缺失定义、重复定义和返回局部对象引用三类故障

注 先修知识：已完成第 1、2 章，能构建程序，并会声明基础类型对象、使用表达式、分支、循环、作用域和标准输入输出；不要求会容器或类。

Python 迁移边界：Python 函数参数仍是名称绑定；C++ 参数声明同时规定类型以及按值复制、引用借用或指针观察，多个源文件还要分别编译再链接。

常见错误与诊断：先确认调用点能否看见声明，再分别检查每个翻译单元是否编译、链接器是否找到唯一定义，以及引用或指针指向的对象是否仍然存活。

3.1 本章任务：把循环中的计算拆成多文件工具

第 2 章的行情筛选器把读取、计算和输出全部写在 `main` 中。程序继续增长时，重复的名义金额公式很容易在不同分支产生不同版本，也无法单独说明哪些函数只读取数据、哪些函数会修改累计状态。本章把计算拆成一个真实的多文件工具：入口从标准输入读取任意多行“价格数量”，实现文件负责累计总名义金额和总数量，最后输出成交量加权平均价（VWAP）。

给定：

```
100.00 10
101.00 20
99.00 10
```

必须逐字符得到：

```
notional=4010.00 quantity=40 vwap=100.25
```

这组结果可独立手算：总数量为 $10 + 20 + 10 = 40$ ，总名义金额为 $100 \times 10 + 101 \times 20 + 99 \times 10 = 4010$ ，所以 VWAP 为 $4010/40 = 100.25$ 。完成判据还包括：项目能从全新构建目录配置；参数实验能预测调用者是否改变；三个故障实验能按编译诊断或链接诊断分类。

3.2 从表达式到函数

3.2.1 场景：同一公式需要一个名字和边界

若每次累计都直接写 `price * quantity`，公式的含义、输入和结果只能靠上下文猜。函数把一段计算命名，并建立局部作用域。最小规则可以读作：

```
return_type function_name(parameter_declarations);
```


这一行是声明：分号表示这里只有函数接口，没有函数体。调用点只要先看见声明，就能检查实参数量与类型。定义重复相同的返回类型、名称和参数列表，随后用花括号给出函数体；定义末尾不再加分号。调用写作函数名、圆括号和实参，例如 `trade_notional(price, quantity)`。

3.2.2 重构路线：一次只移动一个边界

重构从第 2 章循环内的 `total_notional += price * quantity;` 开始，而不是直接重写最终项目。第一步把清单 3.2 的 `trade_notional` 定义放在单文件 `main` 之前，只把右侧表达式替换为函数调用，输出必须不变。第二步把声明放在 `main` 前、把同一定义移到 `main` 后，证明调用点只需先看见声明。第三步才把声明移入清单 3.1 的头文件、定义移入实现文件；最后让 CMake 同时编译入口和实现。每一步都用章首输入复跑，任何输出变化都说明移动边界时顺带改变了行为。

本章接口的真实头文件如下；其余多文件结构稍后逐项解释：

Listing 3.1: 行情计算函数声明与 include guard

```
1 #ifndef QUANT_CH03_MARKET_MATH_HPP
2 #define QUANT_CH03_MARKET_MATH_HPP
3
4 namespace quant::chapter3 {
5
6 double trade_notional(double price, int quantity);
7
8 double trade_notional(double price, int quantity, double multiplier);
9
10 void add_trade(double price, int quantity, double& total_notional,
11               int& total_quantity);
12
13 double volume_weighted_price(double total_notional, int total_quantity);
14
15 } // namespace quant::chapter3
16
17 #endif
```

3.2.3 定义、参数、返回与局部作用域

实现文件给出这些声明的定义：

Listing 3.2: 行情计算函数定义

```
1 #include "quant/ch03/market_math.hpp"
2
3 namespace quant::chapter3 {
4
5 double trade_notional(double price, int quantity) {
6     return price * quantity;
7 }
8
9 double trade_notional(double price, int quantity, double multiplier) {
10     return trade_notional(price, quantity) * multiplier;
11 }
```

```

12
13 void add_trade(double price, int quantity, double& total_notional,
14               int& total_quantity) {
15     total_notional += trade_notional(price, quantity);
16     total_quantity += quantity;
17 }
18
19 double volume_weighted_price(double total_notional, int total_quantity) {
20     if (total_quantity == 0) {
21         return 0.0;
22     }
23     return total_notional / total_quantity;
24 }
25
26 } // namespace quant::chapter3

```

以第一个函数为例，`double` 是返回类型；圆括号内依次声明两个参数对象。调用发生时，`price` 和 `quantity` 在函数作用域中创建，`return price * quantity;` 先计算表达式，再把结果交回调点，同时结束本次调用。两个参数以及函数体内创建的其他局部对象都在离开右花括号时结束生命周期。

`volume_weighted_price` 先检查分母。`return 0.0;` 是早返回：条件成立时函数立即结束，不再执行后面的除法。这里用零表示“没有成交量”的简化教学协议；生产接口往往需要显式表达“无结果”，第 10 章再比较可选值和错误机制。

声明中的参数名可以省略，但教学与审阅代码保留有意义的名称。定义必须让每个名称对应函数体中的对象。返回类型不是注释：声明承诺返回 `double`，调用表达式也因此具有 `double` 类型。

注Python 迁移提示：两种语言都用函数名和圆括号调用，也都有局部作用域与早返回。Python 在运行时把实参对象绑定给参数名称；这里的 C++ 声明在编译时固定参数和返回类型，并决定后续是复制还是引用。Python 函数可在不同路径返回不同种类对象，C++ 每条返回路径必须满足同一声明的返回类型。

失败诊断：若调用写在声明之前，编译器会报告名称未声明；若声明写作 `double trade_notional(double, int);`，定义却交换参数类型，二者就是不同函数，原调用可能在链接阶段找不到定义。修复时同时对照声明、定义和调用，不要只改其中一处。

立即练习：只把 `volume_weighted_price` 的零数量返回值从 0.0 改为 -1.0，用空输入运行工具，结果中的 VWAP 应从 0.00 变为 -1.00；随后恢复原协议。

3.3 按值、引用与可空指针

3.3.1 场景：函数是否可以改动调用者

接口不仅说明“接收一个 `double`”，还要说明函数获得独立副本还是借用调用者对象。下面的完整实验由 CTest 逐字符检查：

Listing 3.3: 四种参数模式的可观察差异

```

1 #include <iostream>
2
3 double add_tick(double price) {
4     price += 1.0;
5     return price;
6 }

```

```

7
8 double observe_price(const double& price) {
9     return price;
10 }
11
12 void charge_fee(double& cash, double fee) {
13     cash -= fee;
14 }
15
16 int main() {
17     const double original{100.0};
18     const double copied_result{add_tick(original)};
19     const double observed{observe_price(original)};
20
21     double cash{1000.0};
22     charge_fee(cash, 5.0);
23
24     const double* price_view{&original};
25     const double pointed{*price_view};
26     const double* missing{nullptr};
27
28     std::cout << "original=" << original << " copied_result=" << copied_result
29               << " observed=" << observed << " cash=" << cash
30               << " pointed=" << pointed << " null=" << (missing == nullptr)
31               << '\n';
32     return 0;
33 }

```

`double price` 是按值参数。`add_tick` 修改自己的参数对象并返回 101，但调用者的 `original` 仍为 100。对基础数值类型，按值通常最直接。

`const double& price` 是只读左值引用。`&` 表示参数引用调用者已有对象，不创建独立的业务值；前面的 `const` 禁止函数通过该名称赋值。`observe_price` 因而能读取 `original`，却不能写 `price += 1.0`；。对一个 `double`，只读引用通常没有性能优势；这里用小类型把语义显示清楚，后续较大的标准库和领域对象才更常用 `const T&`。

`double& cash` 是可变左值引用。`charge_fee` 对 `cash` 的赋值直接改变调用者对象，所以输出为 995。可变引用适合“函数职责就是修改这个对象”的接口；若函数主要产生一个新值，返回值通常更易组合和测试。调用语法不会额外显示 `&`，因此名称和文档应让副作用容易发现。

`const double original{100.0}`；还演示只读对象：初始化之后不能再赋值。`const` 约束的是通过该名称允许的操作，不表示数值被放到某种神秘的永久存储中。

注Python 迁移提示： Python 的实参传递不是“所有东西按引用”：参数名称绑定到同一对象，但对名称重新赋值不会改调用者，而修改共享的可变对象可能会。C++ 把三种意图写进函数签名：按值得到独立对象，`const&` 只读借用，`&` 可变借用。二者不能只靠表面调用语法互相类推。

失败诊断：把字面量 100.0 传给 `double&` 会在编译阶段失败，因为可变左值引用需要一个可修改、寿命明确的调用者对象；改成 `double cash{100.0}`；`charge_fee(cash, 5.0)`；。若只读函数意外要求可变引用，`const` 对象也无法调用它；正确修复是收窄函数权限为 `const double&` 或按值，而不是删除调用者的 `const`。

立即练习：只把 `charge_fee` 的第一个参数改成按值，保持函数体和调用点不变。预测局部参数仍会减去 5，但最终 `cash` 应从 995 变回 1000；运行验证后恢复可变引用。

3.3.2 指针：允许“没有观察对象”

引用必须绑定对象且不能改绑；指针对象保存地址，也可以保存空指针值。清单 3.3 中的三行摘录为：

```
1 const double* price_view{&original};
2 const double pointed{*price_view};
3 const double* missing{nullptr};
```

第一行用取地址运算符 & 得到 original 的地址；类型中的 * 声明“指向只读 double 的指针”。第二行的 *price_view 解引用地址并读取 100。相同符号依上下文承担声明符或运算符职责，不能只按字符背诵。

const double* missing{nullptr}; 明确表示当前没有对象；程序用 missing == nullptr 得到 1。指针可以为空，所以每次解引用前都必须由接口契约或条件证明非空。这里的裸指针只是非拥有观察，不负责销毁对象；price_view 也不能比 original 活得更久。所有权与数组指针在第 6 章继续展开。

注Python 迁移提示：Python 常用 None 表示没有对象；C++ 裸指针用 nullptr 表示没有地址。但 Python 名称会自动保持对象引用，非拥有 C++ 指针不会延长被观察对象的生命周期。引用适合必有对象，指针适合“可能没有”的观察接口。

失败诊断：直接计算 *missing 是未定义行为，不能把某次崩溃当成契约；修复是在解引用前检查非空，并让被指向对象覆盖整个使用期。立即练习：只把 price_view 初始化为 nullptr，用 if (price_view != nullptr) 保护读取；预期程序跳过读取而不崩溃，随后恢复指向 original。

3.4 重载：同名操作的不同参数表

同一领域动作有时存在不同的输入形式。本章头文件声明了两个 trade_notional：第一个接收价格和数量；第二个额外接收乘数。它们名称相同，参数数量不同，这叫函数重载。调用 trade_notional(price, quantity) 只有两个实参，编译器选择二参数候选；三实参调用选择另一个定义。三参数版本复用二参数版本，再乘以 multiplier，避免复制基础公式。

重载的签名必须在参数数量或参数类型上可区分；只改变返回类型不够，因为调用点不能总从接收结果的上下文唯一决定候选。默认实参也可能让两个候选同时可用，从而产生“调用有歧义”类别的编译错误。本书优先让重载表达同一概念；完全不同的业务动作应使用不同名称。

注Python 迁移提示：Python 后定义的同名函数通常替换前一个绑定，常用默认参数或 *args 手工分派。C++ 会在编译期建立同名候选集，根据实参数量、类型和所需转换选择唯一最佳匹配；没有可行候选或最佳候选不唯一都会编译失败。

失败诊断：新增 double trade_notional(double price, int quantity, double multiplier = 1.0); 后，二实参调用可能同时匹配原二参数版本和带默认实参的版本。根因是接口集合有歧义；修复为移除默认值、移除重复能力，或使用含义不同的名称。

立即练习：只把 add_trade 内部的二实参调用改为 trade_notional(price, quantity, 1.0)。结果应逐字符不变，因为乘数为 1；然后把乘数改为 0.5，预期总名义金额与 VWAP 都减半，而总数量仍为 40。

3.5 头文件、实现文件与翻译单元

3.5.1 场景：让不同源文件共享同一接口

每个 .cpp 会先经过预处理，再独立编译成一个翻译单元的目标文件。入口 main.cpp 需要看见函数声明，实现文件 market_math.cpp 负责给出定义。最后，链接器把两个目标文件组合成程序。头文件是共享声明的位置，不是“自动参与构建”的魔法文件。

#include "quant/ch03/market_math.hpp" 使用双引号查找项目头文件；标准库头文件继续使用尖括号。预处理器把头文件内容带入当前翻译单元。头文件可能经不同路径被重复包含，因此清单使用 include guard：

`#ifndef` 检查宏尚未定义, `#define` 建立标记, 末尾 `#endif` 关闭条件区域。同一翻译单元第二次读到它时, 声明区域会被跳过。

`namespace quant::chapter3 { ... }` 把名称放进项目自己的命名空间, 降低与其他库同名函数冲突的风险。调用点写全限定名 `quant::chapter3::add_trade`; 头文件不写 `using namespace`, 避免把选择强加给每个包含者。

`include guard` 立即练习: 只在入口中连续写两次同一个项目头文件, 然后运行预处理命令 `g++ -E -P -I code/ch03/include code/ch03/src/main.cpp`。输出中两个 `trade_notional` 重载声明应各出现一次, 而不是各出现两次; 这就是 `guard` 跳过第二次展开的可观察证据。删除重复的 `include` 后再继续构建。

入口文件只负责读取、调用和输出:

Listing 3.4: 多文件行情工具入口

```

1  #include "quant/ch03/market_math.hpp"
2
3  #include <iomanip>
4  #include <iostream>
5
6  int main() {
7      double total_notional{};
8      int total_quantity{};
9
10     double price{};
11     int quantity{};
12     while (std::cin >> price >> quantity) {
13         quant::chapter3::add_trade(price, quantity, total_notional,
14                                     total_quantity);
15     }
16
17     if (!std::cin.eof()) {
18         std::cerr << "input-error: expected price and quantity\n";
19         return 2;
20     }
21
22     const double vwap{
23         quant::chapter3::volume_weighted_price(total_notional, total_quantity)};
24     std::cout << std::fixed << std::setprecision(2)
25               << "notional=" << total_notional << " quantity=" << total_quantity
26               << " vwap=" << vwap << '\n';
27     return 0;
28 }
```

`std::cin >> price >> quantity` 每轮创建一组输入值; `add_trade` 通过两个可变引用更新循环外的累计对象。输入因文件结束而停止是正常路径; 若因为无法转换的文本停止, 程序向 `std::cerr` 写出稳定类别并返回 2。 `const double vwap{...}` 表示结果初始化后只读。输出用 `std::fixed` 和 `std::setprecision(2)` 固定两位小数; 它们改变后续流格式, 不改变内部 `double` 值。

独立快照的 `code/ch03/CMakeLists.txt` 同时列出两个 `.cpp`, 并用 `target_include_directories` 把 `include/` 加入该目标的头文件搜索路径。头文件本身不需要列为编译源; 真正分别产生目标文件的是两个实现源文件。

注Python 迁移提示: Python 的 `import` 通常加载模块对象并在运行时绑定名称; C++ 的 `#include` 是预处理期

文本包含，每个 `.cpp` 仍独立编译。头文件提供共同声明，定义则必须在最终链接中恰好满足需要。把 `include guard` 理解成模块加载缓存会得到错误心智模型。

失败诊断：若编译器报告找不到头文件，先检查 `include` 拼写和目标的 `include` 目录；若只有 `main.cpp` 进入目标，编译能通过但链接器会找不到 `market_math.cpp` 中的定义。修复是让正确实现文件参与同一目标，而不是把 `.cpp` 直接 `include` 进入入口文件。

立即练习：只从 `add_executable` 中删除 `src/market_math.cpp`，从全新构建目录编译。预期两个源文件不再同时参与目标，链接阶段报告缺失定义；恢复该文件后重新干净构建，输出应回到章首结果。

3.6 三个诊断实验：定义数量与对象寿命

故意失败文件位于 `code/ch03/failures/`，不进入默认绿色目标。测试只匹配稳定诊断类别，不绑定完整英文文本。

3.6.1 缺失定义

`missing_definition.cpp` 声明并调用 `fee_rate`，却没有函数体。源文件能编译成目标文件，链接时报告 `undefined reference` 或 `unresolved external symbol` 类别。根因不是“头文件没被执行”，而是最终链接集合没有定义。修复是提供签名一致的唯一定义并让对应目标文件参与链接。

3.6.2 重复定义

`duplicate_definition_a.cpp` 与 `duplicate_definition_b.cpp` 各自都能编译，却都定义了同一个 `fee_rate()`。把两个目标文件一起链接时应报告 `multiple definition` 或 `already defined` 类别。这是“一份声明可以被许多翻译单元看见，但非内联普通函数在程序中需要一个定义”的直接证据。修复是把声明留在头文件、定义只留在一个实现文件。

3.6.3 悬空引用

`dangling_reference.cpp` 返回局部 `double` 对象的引用。局部对象在函数返回时已经结束生命周期，调用者拿到的引用随即悬空。高警告级别编译应报告“返回局部对象引用”类别；本实验不运行生成的程序，因为读取悬空引用是未定义行为，不能把某次崩溃或数值当成稳定结果。修复是按值返回这个小对象，或引用一个已证明比调用者活得更久的对象。

立即练习：为三条实验各记录“两个编译是否成功、链接是否成功、是否允许运行”。答案依次是：单一源编译成功但链接失败；两个源分别编译成功但合并链接失败；编译产生寿命警告且禁止把运行结果当证据。

3.7 从干净目录复现与章节验收

在 Linux/WSL 中运行：

```
cmake -S code/ch03 -B build/ch03
cmake --build build/ch03 --target ch03_market_tool
./build/ch03/ch03_market_tool < code/ch03/data/market_rows.txt
cmake -S . -B build-linux
cmake --build build-linux
ctest --test-dir build-linux -R ch03_ --output-on-failure
```


前三条命令证明章节快照不依赖后续项目源码并得到精确输出；后三条从仓库根项目建立 Linux/WSL 测试构建树，再运行全部第 3 章证据。Windows 可改用 `cmake -S . -B build-mingw -G "MinGW Makefiles"`；多配置生成器还可能把可执行文件放在 `build\ch03\Debug` 并加 `.exe`，但源文件边界、声明/定义规则与测试预期不变。

3.8 自测、编码任务与面试表达

1. 函数声明、定义和调用分别提供什么信息？分号与花括号出现在何处？
2. 参数对象和返回值的生命周期边界是什么？早返回怎样改变控制流？
3. 按值、`const T&`、`T&` 与 `const T*` 分别允许观察或修改什么？哪个可以为 `nullptr`？
4. 重载候选靠什么区分？为什么只改变返回类型不能形成可用重载？
5. `.cpp`、头文件、翻译单元、目标文件和可执行文件怎样连接？
6. `include guard` 防止什么？为什么项目头文件通常使用双引号和命名空间？
7. 缺失定义、重复定义和悬空引用分别在每一阶段得到什么证据？

编码任务：

1. 从空目录重建 `code/ch03` 的头文件、实现文件和入口，使用固定输入得到章首精确输出，并从全新构建树复现。
2. 为行情工具增加只读函数 `double basis_points(const double& rate)`，返回 `rate * 10000.0`；用 0.0015 验收结果 15，并证明调用者值不变。完整答案见 `code/ch03/exercises/basis_points_solution.cpp`。
3. 运行三个故障实验，记录每个翻译单元的编译状态、链接状态、稳定类别、根因和最小恢复步骤；不要运行悬空引用程序来“看看会怎样”。

问题 3.1 面试检查点 用 90 秒回答：“如何把一个单文件行情程序拆成可链接的多文件 C++ 程序？”回答需包含声明与定义、按值和引用、`include guard`、命名空间、独立翻译单元以及一个缺失或重复定义的可观察诊断。

3.9 本章小结

函数声明让调用点在编译期检查接口，定义提供唯一实现，调用创建参数对象并取得返回结果。按值表达独立副本，`const&` 表达只读借用，`&` 表达可变借用，指针还可用 `nullptr` 表示没有观察对象；重载让同名动作拥有可区分的参数表。头文件共享声明，`include guard` 控制重复包含，命名空间管理名称，多个 `.cpp` 分别编译后由链接器组合。诊断时先问每个翻译单元是否编译，再问程序是否拥有所需且唯一的定义，最后确认引用或指针仍指向存活对象。

第四章 标准数据工具、算法与 CSV

内容提要

- ❑ 使用 `string`、`vector` 与 `array` 保存文本和行情列
- ❑ 从普通函数逐步改写出可读的 `lambda` 谓词
- ❑ 用范围 `for`、迭代器与标准算法遍历数据
- ❑ 从文件读取并校验 CSV, 产生带行号的稳定错误

注 先修知识: 已完成第 3 章, 会定义函数、使用引用和指针, 并能构建多文件程序。

Python 迁移边界: Python 的列表同时承担多种职责; C++ 容器把元素类型、存储方式和失效规则写入接口。

常见错误与诊断: 先检查文件是否打开, 再检查表头、列数、空字段和数值转换; 容器异常则检查索引范围、大小/容量与迭代器是否失效。

量化工作中的第一批 C++ 数据通常来自文本文件: 代码列是字符串, 价格和数量是数值列, 最终还要排序、筛选和汇总。本章不提前使用第 5 章才教学的 `struct`; 先用三个等长 `vector` 保存三列, 亲手看清容器、迭代和校验边界。章末交付物 `ch04_csv_stats` 从文件读取固定 CSV, 并精确输出:

```
1 rows=3 total_quantity=25 total_notional=2005.00 vwap=80.20
```

空代码、非法价格和错误列数必须以状态 2 退出, 并把行号与原因写到标准错误; 不能用部分解析结果继续统计。

4.1 从一行行情认识 `string`、`vector` 与 `array`

4.1.1 场景: 长度会变的数据列

Python 的 `list` 可以混放不同种类对象; 行情列通常不该这样。`std::vector<double>` 中尖括号里的 `double` 是元素类型, 整个容器只能保存可转换为该类型的对象。

`std::string` 是拥有字符序列的标准库类型。`std::array` 的两个模板实参分别给出元素类型和固定长度, 适合 CSV 表头这种固定槽位。

Listing 4.1: 字符串、动态序列、固定数组与范围循环

```
1 #include <array>
2 #include <iostream>
3 #include <string>
4 #include <vector>
5
6 int main() {
7     const std::string symbol{"AAPL"};
8     std::vector<double> prices{100.0, 101.5, 99.0};
9     prices.reserve(8);
10    prices.push_back(100.0);
11
12    const std::array<std::string, 3> header{"symbol", "price", "quantity"};
13    double total{0.0};
14    for (const double price : prices) {
15        total += price;
16    }
```

```

17
18     std::cout << "symbol=" << symbol << " symbol_size=" << symbol.size()
19         << " prices_size=" << prices.size()
20         << " capacity_at_least_8=" << (prices.capacity() >= 8)
21         << " header_last=" << header[2] << " total=" << total << '\n';
22 }

```

`prices{100.0, 101.5, 99.0}` 用列表初始化建立三个元素；`push_back` 在尾部增加一个元素。`size()` 是当前元素数，`capacity()` 是不重新分配存储时最多能容纳的元素数。`reserve(8)` 只保证容量至少为 8，不增加虚构行情；`resize(8)` 则会把大小改为 8，并为新增位置构造零值。

范围 `for` 的冒号可读作“从 `prices` 中依次取出”。这里的 `const double price` 按值复制一个小数值且禁止在循环体中修改副本。若要修改容器元素，可写 `double& price`；若只读大型元素，常写 `const T&`。这些参数模式已经在第 3 章建立。

`header[2]` 使用从零开始的索引取得第三项。方括号不做边界检查，`header[3]` 或 `prices[prices.size()]` 都越过有效范围，随后行为未定义。已知只需逐项访问时，范围 `for` 比手写索引更不容易越界。

容器的 `size()` 返回 `std::size_t`，这是标准库用来表示对象大小和索引的无符号整数类型。需要与 `size()` 比较的索引也使用 `std::size_t`；业务数量仍使用能表达其业务范围的类型，不要因为“都非负”就混为一列。

注 Python 迁移提示： Python 字符串和列表也有 `len` 与零起点索引，但 Python 越界会稳定抛出 `IndexError`；C++ 的 `operator[]` 以调用者已经证明范围为前提。Python 列表增长不暴露元素引用失效规则，`vector` 扩容却可能搬迁整段连续存储。

失败诊断：把输出中的 `header[2]` 改为 `header[3]`，编译器不必拒绝它，运行结果也没有可靠契约。根因是索引没有满足 $0 \leq index < size$ 。修复是使用范围循环，或在索引前显式证明范围；不要把一次“看似正常”的输出当成安全证据。

立即练习：把 `reserve(8)` 改为 `resize(8)`，预测 `push_back` 后 `prices_size` 变为 9，而总和仍为 400.5，因为新增元素值初始化为零。运行验证后恢复 `reserve`，说明“容量”和“元素数”为什么不能互换。

再只改固定数组：把 `header` 的第三项从 "quantity" 改为 "volume"，先预测精确输出中的 `header_last`，再编译运行，确认它变为 `header_last=volume`，而 `header.size()` 仍为 3。恢复原值，并说明修改元素与修改 `std::array` 的编译期长度是两件不同的事。

4.2 迭代器是半开区间中的位置

4.2.1 场景：算法不应依赖具体容器写法

标准算法通常不接收整个容器，而接收 `begin()` 与 `end()` 两个迭代器。区间写成 `[first, last)`：`first` 指向第一个元素，`last` 是尾后位置，不指向元素。`*first` 解引用当前位置，`++first` 前进到下一位置；只有在 `first != last` 时才能解引用。

Listing 4.2: 扩容后重新取得 `vector` 迭代器

```

1  #include <iostream>
2  #include <vector>
3
4  int main() {
5      std::vector<int> values;
6      values.reserve(1);
7      values.push_back(7);
8
9      auto first = values.begin();

```

```

10  const auto previous_capacity = values.capacity();
11  while (values.capacity() == previous_capacity) {
12      values.push_back(8);
13  }
14
15  const bool reallocated{values.capacity() != previous_capacity};
16  first = values.begin();
17  auto second = first;
18  ++second;
19  const bool has_multiple{second != values.end()};
20
21  std::cout << "reallocated=" << reallocated << " reacquired=" << *first
22              << " has_multiple=" << has_multiple << '\n';
23  }

```

实验先取得 `values.begin()`，再持续追加直到容量改变。容量改变意味着发生重新分配，旧迭代器已经失效，因此代码没有比较或解引用它，而是重新调用 `begin()`。输出 `reallocated=1 reacquired=7 has_multiple=1` 是可观察证据。

对 `vector`，引发重新分配的插入会使全部指针、引用和迭代器失效；未重新分配时，插入点之前的位置可保持有效，但插入点及其后位置仍可能失效。删除也会使删除点及其后位置失效。最稳妥的规则是：改变容器后，除非已逐条证明标准保证，否则重新取得位置。

注Python 迁移提示： Python 常直接迭代对象而不显式暴露 `iterator` 类型；在迭代列表时修改列表同样容易产生跳项等逻辑错误。C++ 还多一层内存有效性： `vector` 搬迁后，旧迭代器可能指向已经释放的存储。

故意失败的 `code/ch04/failures/stale_iterator.cpp` 保存旧迭代器，强制扩容后再解引用。它不进入绿色目标； `AddressSanitizer` 应非零退出并报告 `heap-use-after-free`。根因不是“`vector` 不可靠”，而是程序违反了失效规则。修复可在取得迭代器前预留足够容量，或在修改后重新取得迭代器。

立即练习：把安全实验改为先 `reserve(64)`，取得迭代器后只追加一个元素。记录容量是否改变；若没有改变，原迭代器仍指向第一个元素。随后恢复“持续追加直到扩容”的版本，比较两个条件而不是死记某个容量数值。

4.3 从普通函数到标准算法与 lambda

4.3.1 场景：循环的意图比循环机械步骤更重要

“逐项检查并计数”可由普通循环清楚表达；当任务正好是查找、计数、复制、排序、转换或归约时，标准算法直接说出操作类别。算法不会自动更快，也不应消灭所有循环：多状态推进、早停和需要精细错误上下文的解析逻辑，普通循环往往更易读。

Listing 4.3: 手写循环、命名谓词与 `lambda` 的同结果对照

```

1  #include <algorithm>
2  #include <iostream>
3  #include <iterator>
4  #include <numeric>
5  #include <vector>
6
7  bool is_valid_price(const double price) {
8      return price > 0.0;

```

```

9  }
10
11 int main() {
12     const std::vector<double> prices{101.0, -1.0, 99.0, 100.0};
13
14     int manual_valid{0};
15     for (const double price : prices) {
16         if (is_valid_price(price)) {
17             ++manual_valid;
18         }
19     }
20
21     const auto named_valid{
22         std::count_if(prices.begin(), prices.end(), is_valid_price)};
23     const auto lambda_valid{std::count_if(
24         prices.begin(), prices.end(),
25         [](const double price) { return price > 0.0; })};
26
27     std::vector<double> valid_prices;
28     std::copy_if(prices.begin(), prices.end(),
29                 std::back_inserter(valid_prices), is_valid_price);
30     std::sort(valid_prices.begin(), valid_prices.end());
31
32     std::vector<double> notionals(valid_prices.size());
33     std::transform(valid_prices.begin(), valid_prices.end(), notionals.begin(),
34                   [](const double price) { return price * 10.0; });
35     const double total_notional{
36         std::accumulate(notionals.begin(), notionals.end(), 0.0)};
37
38     std::cout << "manual_valid=" << manual_valid
39               << " named_valid=" << named_valid
40               << " lambda_valid=" << lambda_valid
41               << " first=" << valid_prices[0]
42               << " total_notional=" << total_notional << '\n';
43 }

```

第一段循环得到独立已知结果 3。std::count_if(first, last, predicate) 对半开区间中的每个元素调用谓词，并返回为真的次数。先传入普通函数 is_valid_price，再把同一规则改写为：

```

1  [](const double price) { return price > 0.0; }

```

开头 [] 是捕获列表；空列表表示不读取周围局部对象。圆括号是参数表，花括号是函数体。本例 lambda 与普通函数都接收一个价格并返回布尔结果。需要阈值时可写 [minimum](double price){ return price >= minimum; }，按值捕获当前阈值；捕获生命周期和可变状态在第 7 章深化。

copy_if 把有效值追加到新 vector，sort 排序，transform 把价格乘以固定数量得到名义金额，accumulate 从 0.0 开始归约。notionals 必须先按输出数量建立大小；把空 vector 的 begin() 交给 transform 写入会越界。算法调用仍需要调用者满足输入、输出和迭代器类别的前置条件。

注 Python 迁移提示：Python 的 sum、sorted、filter 和推导式也能表达操作意图。C++ lambda 的参数和返回在编译期形成具体类型，算法通过迭代器工作；输出算法不会像 Python 列表推导式那样自动替你创建足够的目

标元素。

失败诊断：删除 `std::vector<double> notionals(valid_prices.size())` 中的大小参数，程序仍可能编译，但 `transform` 写入空范围是未定义行为。根因是输出迭代器没有可写位置。修复是预先 `resize`，或像 `copy_if` 一样使用 `std::back_inserter` 明确追加。

立即练习：把有效条件统一改为 `price >= 100.0`。手写、命名函数和 `lambda` 的计数都应变为 2；排序后的首项应为 100，十股名义金额合计应为 2010。三个结果不同步说明你只改了一条规则，而没有完成等价重构。

4.4 文件边界与命令行参数

4.4.1 场景：统计程序需要真实输入文件

`main` 也可接收命令行参数：`argc` 是参数数量，`argv` 保存各参数的 C 字符串指针。`argv[0]` 通常是程序名，本工具要求 `argv[1]` 为 CSV 路径，所以先检查 `argc == 2`。`char* argv[]` 是读取 C 数组参数时常见的写法；数组形参会调整为指针，长度必须由 `argc` 另行提供。

Listing 4.4: 从文件边界调用可复用 CSV 统计函数

```

1  #include <fstream>
2  #include <iomanip>
3  #include <iostream>
4  #include <string>
5  #include <vector>
6
7  #include "quant/ch04/csv_stats.hpp"
8
9  int main(const int argc, char* argv[]) {
10     if (argc != 2) {
11         std::cerr << "error=usage: ch04_csv_stats <market.csv>\n";
12         return 2;
13     }
14
15     std::ifstream input{argv[1]};
16     if (!input) {
17         std::cerr << "error=cannot open input file\n";
18         return 2;
19     }
20
21     std::vector<std::string> symbols;
22     std::vector<double> prices;
23     std::vector<int> quantities;
24     std::string error;
25     if (!quant::ch04::read_market_csv(input, symbols, prices, quantities,
26                                     error)) {
27         std::cerr << "error=" << error << '\n';
28         return 2;
29     }
30
31     const int quantity{quant::ch04::total_quantity(quantities)};
32     double notional{0.0};

```



```

33     if (!quant::ch04::total_notional(prices, quantities, notional)) {
34         std::cerr << "error=internal column length mismatch\n";
35         return 2;
36     }
37     double vwap{0.0};
38     if (quantity != 0) {
39         vwap = notional / quantity;
40     }
41
42     std::cout << std::fixed << std::setprecision(2)
43         << "rows=" << symbols.size() << " total_quantity=" << quantity
44         << " total_notional=" << notional << " vwap=" << vwap << '\n';
45 }

```

`std::ifstream input{argv[1]}`; 打开输入文件。流对象可出现在条件中：`if (!input)` 表示文件未成功打开。程序把启动错误和数据错误写到 `std::cerr` 并返回 2，把正常统计写到 `std::cout` 并返回 0；调用脚本因此能分别消费数据和诊断。

注Python 迁移提示：Python 常写 `with open(path) as input`；C++ 文件流对象也拥有文件句柄，并在对象离开作用域时关闭。第 6 章会把这种确定性清理归纳为 RAII。本章先记住：必须检查打开状态，且不要让解析函数偷偷重新打开另一个路径。

失败诊断：给出不存在的路径时，工具应精确打印 `error=cannot open input file` 并返回 2。根因属于文件边界，不是 CSV 行内容；修复是检查路径、权限和工作目录，而不是修改解析规则。

立即练习：不带参数运行一次，再传入不存在的文件运行一次。两次状态都应为 2，但诊断分别是 `usage` 与 `cannot open`；这证明启动参数错误和外部文件错误没有被混成同一类别。

4.5 CSV 校验：先拒绝坏行，再做统计

4.5.1 场景：逗号文本不等于可信行情

本章刻意定义一个小而明确的 CSV 方言：表头必须精确为 `symbol,price,quantity`，每个数据行恰有三列，不支持引号、转义逗号或多行字段；代码非空，价格必须是有限正数，数量必须是正整数。生产系统可换成熟 CSV 库，但仍需保留这些领域校验。

Listing 4.5: 第 4 章 CSV 与统计函数接口

```

1  #ifndef QUANT_CH04_CSV_STATS_HPP
2  #define QUANT_CH04_CSV_STATS_HPP
3
4  #include <istream>
5  #include <string>
6  #include <vector>
7
8  namespace quant::ch04 {
9
10     bool read_market_csv(std::istream& input, std::vector<std::string>& symbols,
11                         std::vector<double>& prices,
12                         std::vector<int>& quantities, std::string& error);
13
14     int total_quantity(const std::vector<int>& quantities);

```

```

15
16 bool total_notional(const std::vector<double>& prices,
17                    const std::vector<int>& quantities, double& total);
18
19 } // namespace quant::ch04
20
21 #endif

```

`namespace quant::ch04` 是嵌套命名空间的简写，等价于先进入 `quant` 再进入 `ch04`；它让本章快照的名称不与最终项目接口冲突。头文件只声明读取和统计函数，定义仍由实现文件唯一提供。

逐行切分、数值转换、拒绝与统计的完整实现位于 `code/ch04/src/csv_stats.cpp`，并在附录 C 原样列出。它与本章接口、入口共同参与 C++20 构建，不是只存在于正文的伪代码。

`std::getline(input, row)` 每次读取一行且移除换行符；实现还移除 Windows CRLF 留下的单个 `'\r'`，因此同一文件可由 Windows 程序或 WSL 程序读取。`split_row` 用固定长度 `array` 接收三列；`find` 查找逗号，`substr` 复制当前字段。数值转换使用 `istringstream`：先读取目标数值，再尝试读取额外文本；存在 `100abc` 这样的尾随内容就拒绝。`isfinite` 另外拒绝无穷和 NaN。

在第 5 章引入行情记录类型前，`symbols`、`prices`、`quantities` 是三个平行 `vector`。解析函数只在一行完全有效后同时 `push_back`，因此三者长度相等；`total_notional` 仍检查价格与数量列是否等长。格式错误或非 EOF 读取故障都会清空三列并写入 `line N: reason`；调用者不会误用半份文件。

自动测试固定四种公开行为：

1. 正常文件输出 `rows=3 total_quantity=25 total_notional=2005.00 vwap=80.20`；
2. 空代码输出 `error=line 2: empty field: symbol`；
3. 第 3 行非法价格输出 `error=line 3: invalid number: price`；
4. 两列数据输出 `error=line 2: expected 3 columns, observed 2`。

边界回归还使用内存流验证 CRLF 输入、读取中途故障和不等长统计参数；三者不依赖“恰好在当前机器上没出错”。成功路径同时断言状态为 0、标准输出逐字匹配且标准错误为空。

注Python 迁移提示：Python 的 `csv` 模块能处理引号和转义，本章的手写切分器不能；这不是语言能力差异，而是当前接口刻意缩小了输入协议。两种实现都必须在把字符串转成业务对象前检查列数、空字段、数值范围和来源行号。

失败诊断：把数量写成 `10.5` 或 `ten` 会得到 `invalid number: quantity`。直接读取为 `double` 再静默转成整数会隐藏数据错误；修复是按目标字段类型完整消费文本，并拒绝尾随字符与非正数。

立即练习：复制正常文件，把第 3 行数量改成 `ten`。预期标准输出为空、标准错误为 `error=line 3: invalid number: quantity`、退出状态为 2。恢复后再把代码改为空，确认错误类别随首个根因改变。

4.6 从干净目录复现输出任务

独立快照位于 `code/ch04`，不依赖后续回测项目。在 Linux/WSL 仓库根目录运行：

```

1 cmake -S code/ch04 -B build/ch04
2 cmake --build build/ch04
3 ./build/ch04/ch04_csv_stats code/ch04/data/market_rows.csv

```

仓库级行为测试运行：

```

1 cmake -S . -B build-linux
2 cmake --build build-linux
3 ctest --test-dir build-linux -R ch04_ --output-on-failure

```

第一组命令证明章节快照可独立配置、构建并得到章首精确输出；第二组同时运行容器、算法、迭代器、CSV 错误、练习答案与 ASan 证据。Windows 可使用 `build-mingw` 与 `-G "MinGW Makefiles"`，可执行文件带 `.exe`。

4.7 自测、编码任务与面试表达

1. `string`、`vector<double>` 与 `array<string, 3>` 分别表达什么大小和元素类型约束？
2. `size`、`capacity`、`reserve` 与 `resize` 有何区别？
3. 为什么 `[first,last)` 中的 `last` 不能解引用？
4. 哪些 `vector` 修改会使旧迭代器失效？如何恢复？
5. `lambda` 的捕获列表、参数表和函数体分别位于何处？空捕获意味着什么？
6. 手写循环和标准算法各在什么情况下更清楚？算法调用者仍需满足哪些前置条件？
7. 文件打开失败、列数错误和非法数值应如何从状态与诊断上区分？
8. 为什么本章三个平行 `vector` 必须等长？第 5 章应如何消除这个脆弱不变量？

编码任务：按命令行代码汇总正常 CSV。完整答案见 `code/ch04/exercises/symbol_summary_solution.cpp`；AAPL 必须精确输出 `symbol=AAPL rows=2 quantity=15 notional=1505.00`。

问题 4.1 面试检查点 用 90 秒回答：“为什么 `vector` 是行情序列的合理默认容器，什么时候 `push_back` 会让旧迭代器失效？”回答包含连续存储、重新分配、ASan 证据以及 `reserve`/重新取位置两类修复。

4.8 本章小结

`string` 拥有文本，`vector` 管理可变长连续序列，`array` 把固定长度放进类型。迭代器形成半开区间；算法和 `lambda` 只有在范围、输出空间与生命周期前置条件满足时才安全。CSV 是不可信边界，必须校验打开状态、列数、字段和数值并保留行号。下一章会把平行 `vector` 重构为维护不变量的行情记录类型。

第五章 类与行情分析器

内容提要

- ❑ 用 `struct` 和作用域枚举表达公开数据记录
- ❑ 编写成员函数并理解尾随 `const` 与 `this`
- ❑ 用 `class`、访问控制和构造函数维护行情不变
- ❑ 用异常把非法 CSV 转成带行号的稳定诊断量

注 先修知识：已完成第 4 章，会使用 `string`、`vector`、文件流并校验三列 CSV。

Python 迁移边界：Python 的类成员通常可在运行期任意增删；C++ 类型先固定成员与访问边界，并让构造函数建立有效状态。

常见错误与诊断：若程序在 `front()`、除法或统计阶段崩溃，先追查无效对象为何越过构造和输入边界，而不是在每个使用点补零值。

基础篇最后要把第 4 章的三条平行列收束为真正的行情对象，并交付可独立构建的 `ch05_market_analyzer`。固定输入中 AAPL 有两行，程序必须精确输出：

```
1 symbol=AAPL rows=2 first=100.00 last=101.00 return=1.00% low=100.00 high=101.00 trend=up quantity
  =15
```

完成判据还包括：非法价格、数量或空代码以状态 2 退出；错误保留来源行号；从空构建目录配置时不引用后续回测项目。

5.1 从公开记录认识 `struct` 与 `enum class`

5.1.1 场景：先把同一行字段放进一个对象

第 4 章用三个等长 `vector` 保存代码、价格和数量，同一索引共同代表一行。只要漏改一列，表示就会分裂。`struct` 可以把相关字段组合成一个记录，`vector<QuoteRecord>` 的每个元素便是一整行。

Listing 5.1: 公开聚合记录与作用域枚举

```
1 #include <iostream>
2 #include <string>
3
4 enum class Trend { down, flat, up };
5
6 struct QuoteRecord {
7     std::string symbol;
8     double price{};
9     int quantity{};
10 };
11
12 const char* trend_name(const Trend trend) {
13     switch (trend) {
14         case Trend::down:
15             return "down";
16         case Trend::flat:
17             return "flat";
```

```

18     case Trend::up:
19         return "up";
20     }
21     return "unknown";
22 }
23
24 int main() {
25     QuoteRecord quote{"AAPL", 100.0, 10};
26     quote.price = -1.0;
27     const Trend trend{Trend::up};
28     std::cout << "symbol=" << quote.symbol << " price=" << quote.price
29               << " quantity=" << quote.quantity
30               << " trend=" << trend_name(trend) << '\n';
31 }

```

`struct QuoteRecord { ... };` 定义一个新类型。类型定义的右花括号后必须有分号；花括号内部三个声明各自也以分号结束。`struct` 成员默认是公开的，因此 `quote.symbol` 用点号直接访问成员。`QuoteRecord quote{"AAPL", 100.0, 10};` 按声明顺序聚合初始化三个字段。

`enum class Trend { down, flat, up };` 定义三个互斥的命名值。`enum class` 不把 `up` 泄漏到外层作用域，使用时必须写 `Trend::up`；它也不会静默当作整数参与计算。`switch` 对枚举值分派，每个 `case` 后有冒号。本例在每个分支直接 `return`，因此不需要 `break`；若只执行语句而不离开，忘记 `break` 会继续贯穿下一标签。

该程序故意把公开价格改成 `-1`，并稳定输出 `price=-1`。这不是未定义行为，而是一个设计缺口的证据：语法上有效的赋值制造了业务上无效的行情。

注Python 迁移提示： Python 的 `dataclass` 与 C++ 聚合都适合透明记录，但 Python 属性可在运行期重新绑定为不同类型；C++ 字段类型在编译期固定。两者的公开字段都不能自行阻止 `price=-1`，需要在创建或更新边界加入不变量。

失败诊断：把 `Trend::up` 写成裸 `up` 会在编译期得到“名称未声明”一类诊断。根因是作用域枚举要求限定名；修复是恢复 `Trend::up`，不要退回可与整数混用的旧式枚举。

立即练习：把 `Trend::up` 改为 `Trend::down`，先预测末尾精确输出为 `trend=down`，再运行验证。只改变枚举值，不修改字符串转换函数。

5.2 class 把有效状态写进构造边界

5.2.1 场景：行情一旦存在就必须可统计

若每个调用者都要记住“代码非空、价格有限且为正、数量为正”，检查迟早会遗漏。`MarketQuote` 把字段设为私有，只允许构造函数创建对象，并通过只读成员函数观察值。

先看真实头文件中的类声明；这里明确省略头文件保护、`include` 和命名空间上下文，只保留正在学习的类型定义：

Listing 5.2: `MarketQuote` 类声明（省略文件上下文）

```

24 class MarketQuote {
25 public:
26     MarketQuote(std::string symbol, double price, int quantity);
27
28     const std::string& symbol() const;
29     double price() const;

```

```

30     int quantity() const;
31
32 private:
33     std::string symbol_;
34     double price_{};
35     int quantity_{};
36 };

```

构造函数与只读成员函数的真实实现如下；这里同样省略文件开头与命名空间上下文，完整文件收入附录：

Listing 5.3: 构造边界与只读访问（省略文件上下文）

```

9  MarketQuote::MarketQuote(std::string symbol, const double price,
10                          const int quantity)
11      : symbol_{symbol}, price_{price}, quantity_{quantity} {
12      if (symbol_.empty()) {
13          throw std::invalid_argument{"symbol must not be empty"};
14      }
15      if (!std::isfinite(price_) || price_ <= 0.0) {
16          throw std::invalid_argument{"price must be finite and positive"};
17      }
18      if (quantity_ <= 0) {
19          throw std::invalid_argument{"quantity must be positive"};
20      }
21  }
22
23  const std::string& MarketQuote::symbol() const { return symbol_; }
24
25  double MarketQuote::price() const { return price_; }
26
27  int MarketQuote::quantity() const { return quantity_; }

```

下面的可执行程序创建一个有效对象，再验证非法价格确实在构造边界被拒绝：

Listing 5.4: 有效行情与被拒绝行情

```

1  #include <exception>
2  #include <iostream>
3
4  #include "quant/ch05/market.hpp"
5
6  int main() {
7      try {
8          const quant::ch05::MarketQuote valid{"AAPL", 100.0, 10};
9          std::cout << "valid=" << valid.symbol() << '@' << valid.price();
10         const quant::ch05::MarketQuote invalid{"AAPL", -1.0, 10};
11         std::cout << " unexpected=" << invalid.price();
12     } catch (const std::exception& error) {
13         std::cout << " rejected=" << error.what();
14     }
15     quant::ch05::MarketAnalyzer analyzer{"AAPL"};
16     try {

```



```

17     analyzer.add(quant::ch05::MarketQuote{"MSFT", 50.0, 10});
18 } catch (const std::exception& error) {
19     std::cout << " mismatch=" << error.what();
20 }
21     std::cout << '\n';
22 }

```

class 的成员默认私有。类声明中的 public: 与 private: 是访问标签，冒号后直到下一个标签的声明共享该权限。构造函数与类同名、没有返回类型；MarketQuote(std::string symbol, double price, int quantity) 在函数体运行前通过成员初始化列表建立字段。

接口中的 double price() const; 末尾 const 修饰成员函数，承诺调用不会修改该对象的非 mutable 状态。因此 const MarketQuote valid 仍可调用 price()。返回代码时使用 const std::string&，让调用者只读借用类所拥有的字符串；借用不能比对象活得更久。

构造函数发现非法字段就执行 throw std::invalid_argument{...}。对象没有构造成功，调用点跳到匹配的 catch (const std::exception& error); 作用域中已经构造完成的自动对象会在传播过程中销毁，这称为栈展开。捕获后读取 error.what() 得到原因，但只在命令行边界记录一次。

注Python 迁移提示：Python 的 __init__ 也可抛出 ValueError 阻止对象交付；C++ 的关键差异是成员初始化早于构造函数体，且栈展开会确定性销毁已经完成构造的自动对象。第 6 章会把这一机制用于文件和动态资源的 RAII。

故意失败的 code/ch05/failures/private_access.cpp 尝试读取 quote.quantity_。它不进入绿色目标；编译必须非零退出，并匹配 private 或 inaccessible 诊断类别。根因是调用者越过公开接口，修复是调用 quantity()，而不是把字段改回 public。

立即练习：把有效对象的数量从 10 改为 0。预期程序进入 catch 并得到 quantity must be positive; 随后恢复 10，解释为何不需要在每次统计前重复检查数量。

5.3 成员函数维护分析器不变量

5.3.1 场景：只统计一个代码且至少有一行

分析器把目标代码、价格序列和总数量放在同一对象中。完整声明如下，MarketSummary 只是计算完成后的公开快照，MarketAnalyzer 则维护可变状态：

Listing 5.5: 行情值、摘要记录与分析器接口

```

1  #ifndef QUANT_CH05_MARKET_HPP
2  #define QUANT_CH05_MARKET_HPP
3
4  #include <cstdint>
5  #include <string>
6  #include <vector>
7
8  namespace quant::ch05 {
9
10     enum class Trend { down, flat, up };
11
12     struct MarketSummary {
13         std::string symbol;
14         std::size_t rows{};

```

```

15     double first_price{};
16     double last_price{};
17     double return_percent{};
18     double low{};
19     double high{};
20     Trend trend{Trend::flat};
21     int total_quantity{};
22 };
23
24 class MarketQuote {
25 public:
26     MarketQuote(std::string symbol, double price, int quantity);
27
28     const std::string& symbol() const;
29     double price() const;
30     int quantity() const;
31
32 private:
33     std::string symbol_;
34     double price_{};
35     int quantity_{};
36 };
37
38 class MarketAnalyzer {
39 public:
40     MarketAnalyzer(std::string symbol);
41
42     void add(const MarketQuote& quote);
43     MarketSummary summary() const;
44
45 private:
46     std::string symbol_;
47     std::vector<double> prices_;
48     int total_quantity_{};
49 };
50
51 const char* trend_name(Trend trend);
52
53 } // namespace quant::ch05
54
55 #endif

```

构造函数先拒绝空的目标代码。add 只接收代码相同的 MarketQuote；不匹配时抛出 symbol does not match analyzer，所以数量和价格序列永远描述同一标的。

成员函数内可直接写成员名，也可写 this->prices_。this 是指向当前对象的指针，箭头先通过指针找到对象再访问成员。本章实现用 this-> 明确指出 add 修改的是当前分析器；没有名称歧义时省略它通常更简洁。

summary() const 不修改分析器。它先拒绝空价格序列，再用首末价计算简单收益率

$$100 \left(\frac{P_{last}}{P_{first}} - 1 \right),$$

并用 `minmax_element` 得到区间。价格已由 `MarketQuote` 保证为正，因此分母不会为零。末价大于、等于或小于首价时分别产生 `Trend::up`、`flat` 或 `down`。

注Python 迁移提示：Python 方法显式写 `self` 参数；C++ 非静态成员函数隐式获得 `this`。Python 常在实例字典中加入字段，C++ 的私有成员集合属于类型定义，调用者只能通过编译器已声明的公开成员函数操作它。

失败诊断：查询数据中不存在的 TSLA 时，旧实现对空 `vector` 调用 `front()`，WSL 中曾以状态 `-11` 崩溃。根因是成员函数没有先证明“至少一行”的前置条件。当前实现抛出 `no rows for symbol TSLA`，入口捕获后返回 2。

立即练习：把正常数据第二条 AAPL 价格从 101 改为 99。预期 `return=-1.00%`、`low=99.00`、`high=100.00`、`trend=down`；行数和总数量不变。

5.4 多文件 CLI 与异常边界

5.4.1 场景：领域错误必须带回 CSV 行号

`csv_reader.hpp` 只暴露 `read_market_csv(std::istream&)`。实现逐行切分，把文本转换为数值，再调用 `MarketQuote` 构造函数；若构造拒绝字段，读取层补上 line N：后重新抛出。这样类不依赖 CSV，CSV 又不复制类的不变量。

Listing 5.6: 在程序边界捕获一次并输出摘要

```

1  #include <fstream>
2  #include <iomanip>
3  #include <iostream>
4  #include <exception>
5
6  #include "quant/ch05/csv_reader.hpp"
7  #include "quant/ch05/market.hpp"
8
9  int main(const int argc, char* argv[]) {
10     if (argc != 3) {
11         std::cerr << "error=usage: ch05_market_analyzer <market.csv> <symbol>\n";
12         return 2;
13     }
14
15     try {
16         std::ifstream input{argv[1]};
17         if (!input) {
18             std::cerr << "error=cannot open input file\n";
19             return 2;
20         }
21         const auto quotes{quant::ch05::read_market_csv(input)};
22         quant::ch05::MarketAnalyzer analyzer{argv[2]};
23         for (const auto& quote : quotes) {
24             if (quote.symbol() == argv[2]) {
25                 analyzer.add(quote);
26             }
27         }
28         const auto result{analyzer.summary()};
29         std::cout << std::fixed << std::setprecision(2)

```

```

30         << "symbol=" << result.symbol << " rows=" << result.rows
31         << " first=" << result.first_price
32         << " last=" << result.last_price
33         << " return=" << result.return_percent << '%'
34         << " low=" << result.low << " high=" << result.high
35         << " trend=" << quant::ch05::trend_name(result.trend)
36         << " quantity=" << result.total_quantity << '\n';
37     } catch (const std::exception& error) {
38         std::cerr << "error=" << error.what() << '\n';
39         return 2;
40     }
41 }

```

`try { ... }` 包围可能失败的读取、构造和统计；`catch` 位于入口边界，只负责把异常转换为 `error=` 原因与状态 2。正常路径写 `cout`，诊断路径写 `cerr`，测试同时断言另一个流为空。不能写空的 `catch (...)` 后继续统计，否则错误上下文与备份数据都会被隐藏。

固定测试覆盖以下行为：正常 AAPL 摘要；空代码、非法价格和非正数量；错误表头、列数、价格文本与数量文本；不存在的目标代码；`usage` 与文件打开失败。完整读取和分析实现位于 `code/ch05/src/`，并在附录 C 原样给出。

注Python 迁移提示：Python 常在命令行入口捕获 `Exception` 并转换退出码；C++ 同样应在有能力决定协议的边界捕获。不要在每一层重复记录，也不要将文件打不开、CSV 语法错误和对象不变量错误都改写成同一句“分析失败”。

失败诊断：价格文本 `not-a-price` 最初泄漏了标准库实现消息 `stod`。这不是稳定领域契约。修复是在 CSV 层完整消费文本并固定为 `invalid number: price`，再附加行号。

立即练习：把正常文件的第 2 行数量改为 `ten`。预期 `stdout` 为空、`stderr` 为 `error=line 2: invalid number: quantity`、退出状态为 2；恢复后输出任务必须重新逐字匹配。

5.5 从干净目录交付基础篇项目

独立快照位于 `code/ch05`，只包含本章头文件、实现、数据和 CMake 入口，不引用 `project/`。在 Linux/WSL 仓库根目录运行：

```

1 cmake -S code/ch05 -B build/ch05
2 cmake --build build/ch05
3 ./build/ch05/ch05_market_analyzer code/ch05/data/market_rows.csv AAPL

```

从整书根目录复验本章全部例子、失败实验与干净快照：

```

1 cmake -S . -B build-linux
2 cmake --build build-linux
3 ctest --test-dir build-linux -R ch05_ --output-on-failure

```

Windows 原生 MinGW 的目标文件带 `.exe`，生成器可写 `-G "MinGW Makefiles"`；对象、构造、访问控制和异常边界不因平台改变。

5.6 自测、编码任务与面试表达

1. `struct` 与 `class` 的默认访问权限有何区别？二者是否代表不同对象模型？

2. 为什么使用 `Trend::up` 而不是裸 `up`? `switch` 分支何时需要 `break`?
3. 构造函数为何没有返回类型? 成员初始化列表在函数体之前做什么?
4. 尾随 `const` 修饰谁? 它与返回 `const std::string&` 分别约束什么?
5. `this` 指向什么? 点号与箭头访问成员的对象来源有何不同?
6. 为什么 `MarketQuote` 适合构造失败, 而 `MarketSummary` 适合公开字段?
7. 异常传播时哪些对象会被销毁? 为什么只在 CLI 边界记录一次?
8. 私有成员编译失败与非法价格运行期失败分别属于哪个阶段? 如何恢复?

编码任务: 使用现有 `MarketAnalyzer` 构造两条 AAPL 行情, 输出最高价与最低价之差以及趋势。可运行答案位于 `code/ch05/exercises/price_range_solution.cpp`, 精确输出必须为 `symbol=AAPL range=1.00 trend=up`。

问题 5.1 面试检查点 “为什么不把字段全部设为 `public`?” 回答应包含: 公开记录适用于无需维护跨字段不变量的结果; 输入对象由构造函数保证有效; 私有表示允许未来替换存储; 测试通过公开行为而不是读取内部成员。随后用负价格、空分析器崩溃和带行号诊断作为证据。

5.7 本章小结

`struct` 适合透明记录, `enum class` 提供有作用域的有限状态; `class` 通过构造函数、访问控制和成员函数守住不变量。基础篇行情分析器已经能从干净目录读取 CSV、拒绝坏数据并输出可验证摘要。下一章从这些对象出发学习生命周期、复制移动、RAII 与所有权。

第二部分

Modern C++ 对象与组件

第六章 对象生命周期、RAII 与所有权

内容提要

- ❑ 从构造、使用到析构，画出自动对象与动态对象的生命周期
- ❑ 用 RAII 和 Rule of Zero 把文件等资源的释放绑定到对象生命周期
- ❑ 区分值、引用、裸指针，以及左值、右值和移动后的有效状态
- ❑ 在直接值与三种智能指针之间做有理由的所有权选择
- ❑ 识读 C 数组、new/delete 和典型悬空风险，同时坚持现代默认路径
- ❑ 用 AddressSanitizer 取得一次真实的 heap-use-after-free 诊断

注 先修知识：已完成基础篇，能读写函数、引用、const、string/vector、struct/class 和构造函数；关键语法会在首次实验处复习。

Python 迁移边界：Python 名称和垃圾回收不等于 C++ 所有权；最接近 RAII 的 Python 心智模型是显式的 with 边界，而不是等待 __del__。

常见错误与诊断：出现悬空、重复释放或泄漏时，先画“谁创建、谁拥有、谁借用、何时销毁”，再用 sanitizer 取得失败类别。

6.1 本章输出：让所有权状态可以被观察

低延迟系统里，“这个指针应该还活着”不是可靠论证。行情快照、文件句柄、锁和异步任务都要求同一个问题有明确答案：对象从何时存在，到何时销毁，谁有权延长它的寿命？本章最终任务是重写一个所有权选择程序，并得到如下精确输出：

```
direct=direct copied=copy unique_empty=1 unique_owner=unique shared_owners=1 weak_expired=0
weak_after_reset=1
```

它依次证明五件事：直接值的复制互相独立；唯一所有权可以转移；移动后原智能指针为空；弱观察不增加共享计数；最后一个共享所有者释放后，弱观察变为失效。完成判据不是背出四种工具，而是能从业务寿命关系推出选择，并用输出验证状态转换。

6.2 对象生命周期、复制与移动

6.2.1 场景：一次行情缓冲区究竟销毁几次

函数内的自动对象通常在声明处开始生命周期，在离开其花括号作用域时结束。构造成功才意味着对象存在；析构完成后，其存储即使尚未被别处覆盖，也不能继续作为原对象使用。多个自动对象按构造的逆序析构，这让后创建的资源可以先释放。

下面两种初始化分别请求复制构造和移动构造：

```
Trace copied{original};
Trace moved{std::move(original)};
```

前者创建独立对象。std::move 本身不搬运字节，也不销毁对象；它把表达式转换为允许选择移动操作的值类别。移动后的源对象仍必须可析构、可赋值，但除非类型另有保证，其具体值通常未指定。

6.2.2 完整实验与逐行轨迹

下面的程序显式记录构造、复制、移动和析构。它是教学探针，不是业务类型应照抄的设计；真实业务类型优先让成员自己管理资源。

Listing 6.1: 复制、移动与逆序析构轨迹

```

1  #include <iostream>
2  #include <string>
3  #include <utility>
4
5  class Trace final {
6  public:
7      explicit Trace(std::string label) : label_(std::move(label)) {
8          std::cout << "construct=" << label_ << '\n';
9      }
10
11      Trace(const Trace& other) : label_(other.label_ + "-copy") {
12          std::cout << "copy=" << other.label_ << "->" << label_ << '\n';
13      }
14
15      Trace(Trace&& other) noexcept : label_(std::move(other.label_)) {
16          other.label_ = "moved-from";
17          std::cout << "move=" << label_ << '\n';
18      }
19
20      ~Trace() { std::cout << "destroy=" << label_ << '\n'; }
21
22      [[nodiscard]] const std::string& label() const { return label_; }
23
24  private:
25      std::string label_;
26  };
27
28  int main() {
29      Trace original{"quote"};
30      {
31          Trace copied{original};
32          Trace moved{std::move(original)};
33          std::cout << "states=" << original.label() << ',' << copied.label() << ','
34                  << moved.label() << '\n';
35      }
36      std::cout << "after-inner=" << original.label() << '\n';
37  }

```

`original` 先以标签 `quote` 构造。内部作用域复制出 `quote-copy`，随后把 `original` 的字符串资源移动给 `moved`；本实验主动把源标签设为 `moved-from`，所以这一状态可测试。离开内部作用域时，`moved` 后构造、先析构，然后才是 `copied`。最外层结束时，源对象仍能安全析构。

够用的值类别工作模型是：左值表达式有可重复识别的身份，例如已命名对象；纯右值常表示临时计算结果；将亡值表示资源可以被转移的对象。值类别属于表达式，不是“对象永久贴着的标签”。`original` 是对象，

而写下它这个表达式时通常是左值：`std::move(original)` 才得到将亡值表达式。

注Python 迁移提示：Python 的 `b = a` 通常让两个名称指向同一对象；C++ 的 `Trace b{a}` 调用复制构造并产生独立对象。Python 对象何时回收不应依赖某个实现的引用计数时机；C++ 自动对象离开作用域时按语言规则确定性析构。

6.2.3 失败诊断与立即练习

常见错误是把“移动后仍有效”误读成“移动后仍保持原值”。标准容器通常保证移动后可析构和可重新赋值，但不保证仍有原内容。诊断时先看类型契约，再检查是否在移动后读取了业务状态。

立即练习：把内部两次声明交换顺序，先预测构造和析构行的顺序，再运行 `ch06_lifetime_trace`。只改这一处，并解释为什么析构顺序随构造顺序反转。

6.3 RAII：把释放变成结构性结果

6.3.1 场景：提前返回也必须关闭报告文件

文件、锁、socket、线程和临时目录都需要成对操作。若每条控制路径都手写关闭或解锁，遗漏概率会随分支增加。RAII（Resource Acquisition Is Initialization）让资源由对象成员持有，并在对象析构时释放；作用域和栈展开替程序选择正确清理路径。

6.3.2 完整实验：文件流离开作用域后立即可读

Listing 6.2: 文件流的确定性关闭

```

1  #include <cstdio>
2  #include <fstream>
3  #include <iostream>
4  #include <string>
5
6  int main() {
7      std::string path{"ch06_raii_report.txt"};
8      {
9          std::ofstream output{path};
10         output << "symbol,quantity\nAAPL,10\nMSFT,20\n";
11         std::cout << "inside_open=" << static_cast<bool>(output) << '\n';
12     }
13
14     int rows{};
15     {
16         std::ifstream input{path};
17         std::string line;
18         std::getline(input, line);
19         while (std::getline(input, line)) {
20             ++rows;
21         }
22     }
23
24     int removed = std::remove(path.c_str()) == 0;

```

```

25     std::cout << "after_scope_rows=" << rows << " removed=" << removed << '\n';
26 }

```

内部第一对花括号创建 `std::ofstream`。离开作用域时，它的析构函数刷新并关闭文件；下一作用域因此能够立即打开并读取两行数据。输入流也先离开作用域，Windows 才允许后面的 `std::remove` 删除文件。测试得到 `inside_open=1`、`after_scope_rows=2` 与 `removed=1`。

如果一个类型的 `string`、`vector`、文件流和智能指针成员都能自行管理资源，外层类型通常不应手写析构、复制或移动操作，这就是 Rule of Zero。组合可靠值类型，比维护裸句柄、布尔“是否已关闭”标记和多条清理路径更容易验证。

注Python 迁移提示：Python 的 `with open(path) as f:` 是最接近的资源边界：`__exit__` 在离开块时运行。垃圾回收解决不可达对象的回收，不是及时关闭外部资源的跨实现契约；`__del__` 还会受到循环引用、解释器退出和异常处理限制。C++ 自动对象的析构由词法作用域确定，但动态共享对象仍要等最后一个拥有者消失。

6.3.3 失败诊断与立即练习

析构函数不应让异常逃逸；如果栈展开期间又抛出异常，进程会终止。可能失败的提交或关闭应提供显式操作报告错误，析构只做不抛的尽力清理。另一个常见失败是给已经 Rule of Zero 的类型补一个无必要析构，从而意外影响隐式移动操作。

立即练习：在写入作用域中加入一个条件 `return 0;`，预测文件流是否仍会析构。不要永久保留提前返回；用调试输出或单独实验确认后恢复完整程序。

6.4 值、唯一所有权与共享所有权

6.4.1 选择顺序：先问能否直接拥有值

默认选择顺序是：直接值成员；需要可选堆对象或动态多态时用 `unique_ptr`；只有多个参与者确实共同决定寿命时才用 `shared_ptr`；需要观察共享对象但不延寿时用 `weak_ptr`。智能指针不是“更安全的普通指针”，而是把特定所有权规则编码进类型。

6.4.2 完整输出任务

Listing 6.3: 值、唯一、共享与弱观察状态

```

1  #include <iostream>
2  #include <memory>
3  #include <string>
4  #include <utility>
5
6  struct Ledger final {
7      std::string name;
8  };
9
10 int main() {
11     Ledger direct{"direct"};
12     Ledger copied{direct};
13     copied.name = "copy";
14
15     auto unique = std::make_unique<Ledger>(Ledger{"unique"});

```

```

16     auto transferred = std::move(unique);
17
18     auto shared = std::make_shared<Ledger>(Ledger{"shared"});
19     std::weak_ptr<Ledger> observer{shared};
20
21     std::cout << "direct=" << direct.name << " copied=" << copied.name
22               << " unique_empty=" << (unique == nullptr)
23               << " unique_owner=" << transferred->name
24               << " shared_owners=" << shared.use_count()
25               << " weak_expired=" << observer.expired();
26
27     shared.reset();
28     std::cout << " weak_after_reset=" << observer.expired() << '\n';
29 }

```

Ledger copied{direct} 使用 string 的值语义，修改副本名称不影响原对象。make_unique 创建唯一所有者；std::move(unique) 把所有权交给 transferred，原指针变空。make_shared 建立控制块，weak_ptr 不增加所有者计数，因此 shared_owners=1。调用 shared.reset() 后最后一个强所有者消失，observer.expired() 变为真。

读取弱观察时不能先检查 expired() 再稍后解引用，因为并发环境中对象可能在两步之间销毁；应调用 lock() 一次取得临时 shared_ptr，再判断它是否为空。shared_ptr 的控制块操作通常是线程安全的，但被管理对象的字段并不会自动获得同步。

注Python迁移提示：Python 名称共享对象更像隐式共享引用，但没有在类型中区分唯一、共享和弱观察。weakref 与 weak_ptr 都可观察而不延寿；C++ 则要求接口把所有权意图写得更显式。

6.4.3 失败诊断与立即练习

两个对象互相保存 shared_ptr 会形成强引用环，计数永远不能归零；把不负责延寿的一侧改为 weak_ptr。另一个设计味道是所有参数和成员都改成 shared_ptr：这隐藏了自然所有者、增加原子计数成本，也让对象可能活得远超预期。

立即练习：在输出任务中复制一个 shared_ptr 到内部作用域，预测作用域内外的 use_count()，再运行验证。离开作用域后计数应恢复，不要把具体计数当成并发同步机制。

6.4.4 整合补充：把所有权意图写进函数接口

所有权判断不应只存在于注释里。函数签名是第一道约束：按值参数取得自己的副本或被移动进来的值；const T& 表达调用期间只读借用；T& 表达调用期间可变借用；unique_ptr<T> 按值传入通常表示接管唯一所有权；shared_ptr<T> 按值传入则表示函数要成为共同所有者。若函数只在调用期间读取共享对象，仍应优先接收 const T&，不要为方便而增加引用计数。下面只比较声明，省略类型定义与函数体，不是可单独编译的完整程序：

```

double mark_value(const Quote& quote);           // read-only borrow
void normalize(Quote& quote);                     // mutable borrow
void enqueue(std::unique_ptr<Order> order);       // ownership transfer
void retain(std::shared_ptr<Feed> feed);          // shared lifetime

```

Python 的参数名称和常规类型标注不会在语言层面区分借用、转移与共同延寿，调用约定通常依靠文档和对象行为表达；C++ 接口可以让编译器拒绝复制唯一所有者，却仍不能替程序员证明借用期足够长。

返回值也优先表达值语义。现代 C++ 能通过复制消除或移动高效返回对象，调用者得到清楚的所有权。返回裸指针或引用只有在“被返回对象由别处拥有，且寿命明确长于调用者使用期”时才合理；接口文档还必须说明能否为空。切勿返回局部自动对象的指针或引用，因为函数返回时该对象已经析构。

审查一条接口时可以按四问展开：它是否创建对象；谁在调用后拥有对象；借用最长持续到何时；并发任务或回调能否越过该寿命边界。若回答需要“大家小心一点”，类型通常还没有表达真实关系。量化系统里尤其要警惕把栈上行情对象的引用捕获到异步任务：提交函数很快返回，而任务稍后才解引用，形成确定的悬空窗口。

贯穿项目也刻意遵循这条顺序：project/include/quant/types.hpp 让行情、订单、成交和组合快照直接拥有各自的值；回测入口只读借用事件序列与策略，结果则按值返回。项目目前不需要智能指针，这不是遗漏，而是“没有共同动态寿命就不引入共享所有权”的设计证据。

整合练习：只考察“异步保存一份订单”这一处寿命变化。先写按值版本；若订单不可复制，再把同一个参数改为按值接收 `unique_ptr`，并用调用后原指针为空验证所有权已经转移。不要同时引入共享所有权；只有需求明确允许调用者与任务共同延寿时，才另行比较 `shared_ptr` 版本。

6.5 引用、裸指针与底层内存识读

前两节已经建立现代默认路线：先用直接值和 RAII，确有动态寿命再选择智能指针。现在才进入裸指针、C 数组与手动分配；目标是识读遗留接口和诊断内存错误，而不是建立另一套推荐风格。

6.5.1 场景：观察价格不等于拥有价格

引用 `T&` 是已有对象的别名，声明时必须绑定；裸指针 `T*` 保存地址，可以改指向、可以为空。`&price` 取得地址，`*observer` 解引用并访问所指对象，`nullptr` 明确表示没有对象。二者默认都不拥有对象，也不会自动延长被观察对象的生命周期。

6.5.2 完整实验：借用、C 数组与手动释放

Listing 6.4: 非拥有借用与旧式内存语法识读

```

1  #include <iostream>
2
3  void add_half_tick(double& price) { price += 0.5; }
4
5  int main() {
6      double price{100.0};
7      double& alias{price};
8      add_half_tick(alias);
9
10     const double* observer{&price};
11     std::cout << "price=" << price << " alias=" << alias
12               << " observed=" << *observer;
13
14     observer = nullptr;
15     int stack_quotes[3]{100, 101, 102};
16     int* heap_quantity{new int{10}};
17     std::cout << " missing=" << (observer == nullptr)
18               << " stack_last=" << stack_quotes[2]
19               << " heap_quantity=" << *heap_quantity;

```



```

20
21     delete heap_quantity;
22     heap_quantity = nullptr;
23     std::cout << " released=" << (heap_quantity == nullptr) << '\n';
24 }

```

`alias` 与 `price` 是同一对象的两个名字，因此函数通过可变引用把价格从 100 改到 100.5。`observer` 只读观察同一地址；置为 `nullptr` 后，程序先检查再避免解引用。

`int stack_quotes[3]` 是含三个元素的 C 数组，下标有效范围为 0、1、2。它在许多表达式和函数参数中会退化为指向首元素的指针，长度信息不随指针传递；越界不会自动抛出 Python 式异常。现代业务代码默认用 `std::array` 或 `std::vector` 保留大小与值语义，C 数组主要用于识读接口、协议布局和遗留代码。

`new int{10}` 在动态存储期创建整数并返回拥有它的裸指针；`delete heap_quantity` 结束对象生命周期并释放存储。数组形式 `new T[n]` 必须与 `delete[]` 配对，混用释放形式属于未定义行为。这里展示手动配对是为了读懂底层代码，不是鼓励把它作为默认方案。

注Python 迁移提示： Python 名称没有直接的“可空地址并手动解引用”语法。切片和容器通常携带长度；C++ 裸指针不携带长度或所有权。Python 的对象引用也可能让对象继续存活，而 C++ 裸引用和裸指针不会延寿。

6.5.3 失败诊断与立即练习

离开被指对象的作用域后继续解引用会悬空；释放后继续读取是 `use-after-free`；对同一地址再次 `delete` 是 `double-free`。把指针设为空只能防止该变量再次误用，不能修复其他别名。根因分析必须回到唯一的拥有者和销毁时刻。

立即练习：把 `observer = nullptr`；后面增加条件分支，只在非空时读取；分别用空与非空观察者验证。不要直接写 `*observer` 来“看看会发生什么”，未定义行为没有可接受的固定输出。

6.6 失败实验：让 AddressSanitizer 证明悬空

故意失败源码 `code/ch06/failures/use_after_free.cpp` 在 `delete` 之后再次读取同一地址。它与绿色示例隔离，不能进入默认 `all` 目标：

```

1  int* quantity{new int{10}};
2  delete quantity;
3  std::cout << *quantity << '\n'; // intentional use-after-free

```

在 Linux/WSL 中运行：

```

g++ -std=c++20 -O0 -g -fsanitize=address \
    -fno-omit-frame-pointer \
    code/ch06/failures/use_after_free.cpp -o use_after_free
ASAN_OPTIONS=detect_leaks=0:halt_on_error=1 ./use_after_free

```

验收类别是 `heap-use-after-free` 和非零退出，不绑定某一版完整堆栈文本。报告中的“freed by”位置说明存储在哪里释放，“read of size”说明悬空访问发生在哪里。恢复步骤是删除非法读取，并把所有权改为直接值或唯一所有者；只把指针置空不足以修复其他别名。若把非法读取换成第二次 `delete quantity`；，同一所有权错误会表现为 `double-free` 类别。

本项目的 CTest 在 Windows 上通过 WSL GCC 取得真实 ASan 证据；受限环境无法启动 WSL 时测试以明确的 `skip` 状态退出，而发布验收必须在可用的 Linux/WSL 环境实际通过。

6.7 自测、练习与面试表达

1. 对象生命周期的开始和结束分别需要什么事件？为什么存储仍有旧比特不等于对象仍存在？
2. `std::move` 实际做什么？移动后的对象保证什么，又通常不保证什么？
3. 引用、裸指针、`unique_ptr` 和 `shared_ptr` 分别表达何种所有权？
4. C 数组退化后丢失什么信息？`new[]` 为什么必须与 `delete[]` 配对？
5. RAII 如何覆盖异常和提前返回？Rule of Zero 为什么通常比手写析构更稳健？
6. `weak_ptr` 如何打破共享环？为什么“shared”不代表对象字段线程安全？
7. sanitizer 报告 use-after-free 时，如何从分配、释放和非法访问三处恢复生命周期图？

编码练习：

1. 从空文件重建复制/移动轨迹，先写出完整预期输出再运行；答案见 `code/ch06/lifetime_trace.cpp` 与对应 `expected` 文件。
2. 为可空价格观察写出引用版本和指针版本，说明为什么引用版本不能表达“没有价格”；答案见 `code/ch06/borrowing_and_legacy.cpp`。
3. 用文件流作用域写入并读取两行报告，确认资源在第二次打开前已经关闭；答案见 `code/ch06/raii_file.cpp`。
4. 重写本章所有权输出任务并达到章首精确字符串；答案见 `code/ch06/ownership_choices.cpp`。
5. 单独运行 ASan 失败实验，记录非零退出和稳定诊断类别，不把未定义输出当作答案。

问题 6.1 面试检查点 给定“策略异步保存一条行情供稍后读取”，先问谁拥有行情、异步任务最长活多久、能否按值复制，再决定值、唯一所有权或共享所有权。回答必须包含一个悬空失败路径、一个 RAII 清理边界和拒绝滥用 `shared_ptr` 的理由。

6.8 本章小结

生命周期是所有权推理的时间轴：构造建立对象，作用域与拥有者决定析构，引用和裸指针只表达借用。直接值和 Rule of Zero 是默认路径，RAII 把资源释放变成结构性结果；`unique_ptr` 表达可转移的唯一所有权，`shared_ptr` 只用于真实共同寿命，`weak_ptr` 提供不延寿的观察。C 数组与手动分配必须会读、会诊断，但不应取代现代值类型。最后，sanitizer 给出的不是“修复方案”，而是重建错误生命周期图的运行证据。

第七章 STL 与 ranges

内容提要

- ❑ 按访问模式、迭代器稳定性和所有权选择容器
- ❑ 解释 view 的惰性、借用关系与悬空风险
- ❑ 用 ranges 算法和 view 组合行情处理步骤
- ❑ 识别浮点归约对顺序的依赖

注 先修知识：已完成第 4–6 章，会使用 vector、迭代器、lambda、类、引用和 RAII。

Python 迁移边界：Python 的 list comprehension、生成器和 dict 能表达相似管道，但 C++ 容器的存储、迭代器失效和 view 生命周期都必须显式推理。

常见错误与诊断：若管道结果异常，依次检查源容器是否仍存活、view 是否仍借用同一数据、算法是否改变了顺序，以及归约是否依赖浮点结合律。

本章把第五章的行情分析器演化为可独立构建的成交管道。固定输入中 AAPL 有三笔成交，程序必须精确输出：

```
1 symbol=AAPL trades=3 first_sell=102.00 low=100.00 high=102.00 gross=2520.00 net_quantity=15
```

完成判据还包括：非法买卖方向与缺失代码以状态 2 退出；惰性 view、悬空 view 和浮点归约顺序均有可运行证据；code/ch07 可从空构建目录独立配置。

7.1 从访问模式选择容器

7.1.1 场景：时间序列与按代码查询不是同一种访问

成交按到达顺序进入程序，适合由 `vector<Trade>` 连续保存；仓位却需要频繁按代码查询，若每次都从头扫描时间序列，意图和成本都不清楚。下面的程序同时保留时间序列，并用 `map<string, int>` 建立净数量索引：

Listing 7.1: 按访问模式组合 vector、map 与算法

```
1 #include <algorithm>
2 #include <iomanip>
3 #include <iostream>
4 #include <map>
5 #include <vector>
6
7 #include "quant/ch07/trade.hpp"
8
9 int main() {
10     const std::vector<quant::ch07::Trade> trades{
11         {"AAPL", quant::ch07::Side::buy, 10, 100.0},
12         {"MSFT", quant::ch07::Side::buy, 4, 200.0},
13         {"AAPL", quant::ch07::Side::sell, 5, 102.0},
14         {"AAPL", quant::ch07::Side::buy, 10, 101.0},
15     };
16
17     std::map<std::string, int> positions;
18     for (const auto& trade : trades) {
19         int signed_quantity{trade.quantity};
```

```

20     if (trade.side == quant::ch07::Side::sell) {
21         signed_quantity = -trade.quantity;
22     }
23     positions[trade.symbol] += signed_quantity;
24 }
25
26 const auto first_sell =
27     std::ranges::find_if(trades, [](const quant::ch07::Trade& trade) {
28         return trade.side == quant::ch07::Side::sell;
29     });
30 auto sorted{trades};
31 std::ranges::sort(
32     sorted, [](const quant::ch07::Trade& left,
33               const quant::ch07::Trade& right) {
34         return left.price < right.price;
35     });
36
37 std::cout << std::fixed << std::setprecision(2)
38     << "rows=" << trades.size() << " symbols=" << positions.size()
39     << " aapl_position=" << positions.at("AAPL")
40     << " first_sell=" << first_sell->symbol << '@' << first_sell->price
41     << " lowest=" << sorted.front().symbol << '@'
42     << sorted.front().price << " order=";
43
44 std::string separator;
45 for (const auto& entry : positions) {
46     std::cout << separator << entry.first;
47     separator = ',';
48 }
49 std::cout << '\n';
50 }

```

`vector` 拥有元素并提供连续存储、常数时间索引和高效尾部追加，是批量行情的默认候选；扩容仍会使指向旧元素的迭代器、指针和引用失效。`map` 也拥有键和值，按键排序并提供对数时间查找。它的下标运算在键缺失时先插入值初始化的整数，再执行复合赋值；只查询已有键时用 `at`，拼错键会抛出异常而不会悄悄插入。范围循环遍历 `map` 时按键顺序得到 AAPL、MSFT，因此输出末尾为 `order=AAPL,MSFT`。

若只关心平均常数时间查找且不需要键序，可考虑 `unordered_map`；若频繁在两端插入可考虑 `deque`。选择依据是访问与失效规则，不是背诵容器名称。原始 `trades` 保持时间顺序，副本 `sorted` 才交给 `ranges::sort`；这样“首笔卖出”和“最低价格”不会争夺同一个顺序。

`ranges::find_if(trades, predicate)` 直接接收范围，返回指向首个匹配元素的迭代器；没有匹配时等于 `trades.end()`。本例已知存在卖出才使用 `first_sell->price`，项目版本会显式检查末端。

注Python 迁移提示：Python `list` 与 C++ `vector` 都保持顺序，`dict` 与关联容器都按键查询；不同点是 Python 引用通常不因列表扩容失效，而 C++ 迭代器直接关联具体存储。Python 的 `sorted` 返回新列表，C++ 的 `ranges::sort` 原地修改范围，因此需要先决定是否保留原顺序。

失败诊断：若直接排序 `trades` 后再寻找“首笔卖出”，程序仍能编译，却把“首笔”偷换为“排序后的第一笔”。根因是一个容器同时承担时间线和排名两种语义；修复是保留原序列并排序副本。若 `find_if` 可能失败，则在解引用前先与末端比较。

立即练习：把第一笔 AAPL 的代码改为 MSFT。预期 `aapl_position=5`，最低价变为 MSFT@100.00，而行数、代码数、首笔 AAPL 卖出及 `order=AAPL,MSFT` 不变。

7.2 view 是惰性借用，不是新容器

7.2.1 场景：先描述筛选，再决定何时遍历

大量中间 `vector` 会复制元素并分配内存。C++20 `view` 可以先组合“保留哪些元素、如何转换”，真正迭代时才逐个求值：

Listing 7.2: `filter` 与 `transform` 的惰性管道

```

1  #include <iomanip>
2  #include <iostream>
3  #include <ranges>
4  #include <vector>
5
6  int main() {
7      std::vector<double> prices{99.0, 101.0, 80.0};
8      auto selected = prices | std::views::filter([](const double price) {
9          return price >= 100.0;
10         }) |
11         std::views::transform([](const double price) {
12             return price * 2.0;
13         });
14
15     prices[0] = 110.0;
16
17     std::cout << std::fixed << std::setprecision(2) << "selected=";
18     bool first{true};
19     for (const double value : selected) {
20         if (!first) {
21             std::cout << ',';
22         }
23         std::cout << value;
24         first = false;
25     }
26     std::cout << '\n';
27 }
```

管道运算符 `|` 把左侧范围交给右侧 adaptor。`filter` 保存谓词并跳过不满足条件的元素；`transform` 在解引用时计算新值。`selected` 不是新 `vector`，没有复制价格。它借用 `prices`，所以 `view` 创建后把首价从 99 改为 110，稍后的遍历会看到 110，并输出 `selected=220.00,202.00`。

借用也意味着源范围必须覆盖 `view` 的所有使用。`view` 自身按值保存并不延长底层 `vector` 的生命；同时，若源容器发生使迭代器失效的修改，已有 `view` 的遍历同样可能失效。

注Python 迁移提示：Python 生成器表达式同样惰性，但生成器通常持有对源对象的 Python 引用，垃圾回收会延长对象生命。本例由左值 `vector` 建立的 C++ `view` 只借用源容器，不能替它保活数据；其他 `view` 是否拥有底层范围，取决于它包装的 `range` 或 `view`。两者都可能在迭代时看到源数据的后续修改。

故意失败的 `code/ch07/failures/dangling_view.cpp` 从函数返回借用局部 `vector` 的 `view`。它不进入绿色目标；ASan 执行必须非零退出并报告 `stack-use-after-return`、`stack-use-after-scope` 或等价的栈访问类别。根因不是 `filter`，而是源范围先结束生命。修复是让调用者拥有源容器、在源仍存活时消费 `view`，或把结果物化为拥有元素的容器。

立即练习：把过滤阈值从 100 改为 105。首价修改后仍通过，101 被排除，精确输出应变为 `selected=220.00`。

7.3 把算法组合成行情管道

7.3.1 场景：过滤、排序、查找、转换与归约各守一个职责

最终管道接收已解析的 `vector<Trade>` 与目标代码。核心实现如下，CSV 读取和 CLI 只负责边界：

Listing 7.3: 成交筛选、排序、查找、转换与归约

```

1  #include "quant/ch07/pipeline.hpp"
2
3  #include <algorithm>
4  #include <iterator>
5  #include <map>
6  #include <numeric>
7  #include <ranges>
8  #include <stdexcept>
9
10 namespace quant::ch07 {
11
12 TradeSummary summarize_trades(const std::vector<Trade>& trades,
13                               const std::string& symbol) {
14     auto selected = trades | std::views::filter([&symbol](const Trade& trade) {
15         return trade.symbol == symbol;
16     });
17
18     std::vector<Trade> sorted;
19     std::ranges::copy(selected, std::back_inserter(sorted));
20     if (sorted.empty()) {
21         throw std::logic_error{"no trades for symbol " + symbol};
22     }
23     std::ranges::sort(sorted, [](const Trade& left, const Trade& right) {
24         return left.price < right.price;
25     });
26
27     const auto first_sell =
28         std::ranges::find_if(selected, [](const Trade& trade) {
29             return trade.side == Side::sell;
30         });
31     if (first_sell == selected.end()) {
32         throw std::logic_error{"no sell trade for symbol " + symbol};
33     }
34
35     auto notionals = selected | std::views::transform([](const Trade& trade) {
36         return trade.price * static_cast<double>(trade.quantity);
37     });
38     const double gross{
39         std::accumulate(notionals.begin(), notionals.end(), 0.0)};
40

```

```

41     std::map<std::string, int> positions;
42     for (const Trade& trade : trades) {
43         int signed_quantity{trade.quantity};
44         if (trade.side == Side::sell) {
45             signed_quantity = -trade.quantity;
46         }
47         positions[trade.symbol] += signed_quantity;
48     }
49
50     return TradeSummary{symbol,          sorted.size(), first_sell->price,
51                        sorted.front().price, sorted.back().price, gross,
52                        positions.at(symbol)};
53 }
54
55 } // namespace quant::ch07

```

第一步的 `selected` 借用参数 `trades`；函数执行期间参数引用有效。第二步用 `back_inserter(sorted)` 把筛选结果追加到拥有元素的 `vector`，因为排序会修改元素顺序。空结果在调用 `front` 或 `back` 前被拒绝。

`find_if` 在原时间线中找首笔卖出；`transform` 把每笔成交映射为 $price \times quantity$ ；`accumulate` 从 0.0 开始按迭代顺序累加，因此结果类型为 `double`。最后的 `map` 用正买入、负卖出累计每个代码的净数量。每个步骤的对象寿命都能从局部变量和参数边界读出。

注Python 迁移提示：Python 常把这段写成筛选列表、`sorted`、`next` 和 `sum`。C++ `ranges` 把“遍历什么”与“如何处理”分开，但不会自动把 `view` 物化，也不会自动检查末端。需要修改顺序或把结果带出源范围时，显式复制反而是清楚的所有权决定。

失败诊断：把命令行代码改为 `MSFT` 时，过滤能得到一笔买入，但找不到卖出。程序以状态 2 输出 `error=no sell trade for symbol MSFT`，而不是解引用末端迭代器。查询 `TSLA` 则更早得到 `no trades for symbol TSLA`。错误发生在哪个前置条件，诊断就停在哪一层。

立即练习：保持数据不变，只把命令中的 `AAPL` 改为 `MSFT`。验证退出状态为 2、`stdout` 为空、`stderr` 精确包含 `error=no sell trade for symbol MSFT`。

7.4 浮点归约有顺序

7.4.1 场景：可归约不等于满足结合律

算法接口允许累加，不代表浮点加法等同于实数加法。下面只改变遍历顺序：

Listing 7.4: 相同浮点值的两种归约顺序

```

1  #include <algorithm>
2  #include <iostream>
3  #include <numeric>
4  #include <vector>
5
6  int main() {
7      std::vector<double> values{1.0e16, -1.0e16, 1.0};
8      const double forward{std::accumulate(values.begin(), values.end(), 0.0)};
9
10     std::ranges::reverse(values);
11     const double reverse{std::accumulate(values.begin(), values.end(), 0.0)};

```

```

12
13     std::cout << "forward=" << forward << " reverse=" << reverse << '\n';
14 }

```

有限精度下， $10^{16} + (-10^{16})$ 先抵消，再加 1 得到 1；反向遍历时，1 先与 -10^{16} 相加会被舍入，最终得到 0。因此精确输出为 `forward=1 reverse=0`。`accumulate` 的顺序确定，却仍依赖输入顺序；允许重排的并行归约可能产生另一条舍入路径。

注Python 迁移提示：Python 的普通 `float` 通常同样是 IEEE 754 双精度，`sum` 也不能把浮点数变回实数。NumPy 的实现可能使用不同分块或成对求和，因此结果与逐项 Python 循环不同并不自动说明其中一个“错了”；必须先定义容差与数值口径。

失败诊断：这里没有编译错误或 sanitizer 报告；失败的是“加法可任意重排且逐位相同”的假设。修复不应只是固定一次偶然顺序，而应在第 16 章根据数据尺度选择更稳定算法，并用有来源的容差测试。

立即练习：把值的顺序改为 `{1.0, 1.0e16, -1.0e16}`。先预测，再验证正向结果为 0、反向结果为 1；元素集合没有变化。

7.5 独立快照与输出任务

在 WSL 中从项目根目录执行：

```

1 cmake -S code/ch07 -B build/ch07
2 cmake --build build/ch07 -j
3 ./build/ch07/ch07_ranges_pipeline code/ch07/data/trades.csv AAPL

```

输出必须与章首单行逐字一致。根项目的 CTest 还验证非法方向、缺失代码、惰性 `view`、浮点顺序、ASan 故障实验、核心练习与干净快照。Windows 可用同一 CMake 源目录选择 Visual Studio 生成器，运行时按所选配置进入 `Debug` 或 `Release` 子目录；正文的生命周期和算法规则不变。

7.6 自测、编码任务与面试检查点

1. 为什么时间顺序成交默认适合 `vector`，而按代码仓位适合关联容器？
2. `map::operator[]` 与 `at` 在键缺失时有何不同？
3. `ranges::sort` 为什么作用于副本，而 `find_if` 作用于原序列？
4. `view` 的惰性从哪条输出得到证明？它拥有什么、借用什么？
5. 返回借用局部容器的 `view` 为什么悬空？有哪些恢复方案？
6. `back_inserter` 在物化筛选结果时解决什么问题？
7. 为什么 `accumulate` 的初值写 0.0 而不是 0？
8. 相同浮点集合为何能归约出 1 和 0？本章应承诺什么、不承诺什么？

编码任务：过滤固定成交中的 AAPL，按价格降序排序并只输出前两笔。可运行答案位于 `code/ch07/exercises/top_prices_solution.cpp`，精确输出必须为 `top=AAPL@102.00,AAPL@101.00`。

问题 7.1 面试检查点 面试中若被问“`ranges` 是否一定比循环快”，先回答它主要表达组合与惰性，性能必须测量；再说明 `view` 的借用生命周期、物化时机、迭代器失效与浮点归约顺序。能指出这些边界，比背诵 adaptor 名称更可信。

7.7 本章小结

容器选择来自访问、失效和所有权；`ranges` 算法直接处理范围。本章从左值容器建立的 `view` 惰性借用源数据，其他 `view` 的所有权取决于所包装的范围。需要排序、跨边界保存或固定所有权时应显式物化。归约接口不

取消浮点舍入，数值稳定性留到第 16 章继续解决。

第八章 接口与值语义

内容提要

- 从订单、成交与组合失败场景推导公开接口
- 用值语义表达可独立复制的领域事实
- 用组合装配默认路径，并识别运行时多态的适应
- 用边界
- 在修改状态前维护数量、价格、现金和持仓不
- 变量

注 先修知识：已完成第 3–7 章，会使用类、构造函数、引用、异常、容器和 `ranges`。

Python 迁移边界：Python 对象通常通过名字共享引用；C++ 对象默认按值拥有自己的状态，复制会得到独立对象。Python 的鸭子类型可以直接替换策略，C++ 需要具体类型、模板约束或虚函数接口。

常见错误与诊断：若组合结果错误，先检查无效值是否在构造边界被拒绝，再检查“无订单/无成交”是否被当成正常缺席，最后检查组合更新是否在验证完成前修改了状态。

本章把脚本式分析流程演化为四类组件：行情事件是输入事实，策略产生订单，交易所把可成交订单变为成交，组合只接受合法成交并给出快照。固定场景从 10000 现金开始，在 99 元买入 25 股，以 101 元估值，必须精确输出：

```
1 orders=1 fills=1 symbol=AAPL quantity=25 cash=7525.00 equity=10050.00
```

完成判据不只是“能打印”。零数量、非有限价格、超现金买入和超持仓卖出都必须在越过领域边界前失败；策略决策与成交/组合两个公开 seam 由独立行为测试保护；`code/ch08` 能从空构建目录独立配置。

8.1 先写用例，再决定类

8.1.1 场景：一条行情怎样变成持仓

先写成功路径的业务句子，而不是先列类名：当 AAPL 价格不高于 100 且组合为空仓时，策略申请买入 25 股；只有订单代码与行情代码相同且行情数量充足时才成交；组合扣除 $25 \times 99 = 2475$ 现金并增加 25 股。第二条 101 元行情到达时，已有持仓使策略不再下单，最终权益为

$$7525 + 25 \times 101 = 10050.$$

再写失败句子：数量不能为零或负数；价格必须为正且有限；AAPL 订单不能由 MSFT 行情成交；持有 15 股时不能卖 20 股；现金 100 元时不能买入名义金额 110 元的股票。这些句子决定了组件的职责边界。

事实或动作	拥有的知识	不应承担的职责
MarketEvent	代码、价格、可用数量	决定是否交易
Order	代码、方向、申请数量	假装已经成交
Fill	代码、方向、数量、成交价	自己修改组合
Portfolio	现金和本次运行的单代码持仓	猜测策略意图
Strategy	从事件与快照作决策	直接写组合内部表

这不是 `getter/setter` 清单。公开操作来自用例：策略需要读取事件；交易所需要比较订单与事件；组合需要应用成交和产生快照。没有用例要求调用者任意改写成成交价，因此没有 `set_price`。

注 Python 迁移提示：Python 的 `@dataclass` 很适合快速表达这些记录，但默认字段仍可被任意改写，也不会从类型声明自动得到正价格约束。C++ 同样不会自动验证业务规则；区别在于我们可以让构造函数成为唯一建成对象的入口，并把字段保持为私有。Python 也能用只读 `property` 或验证库实现类似契约，关键是边界而不是语言优越性。

失败诊断：如果 `Portfolio` 同时读取 CSV、选择策略、撮合并打印报告，那么任何失败都指向这个大类，测试也必须搭建整条流水线。根因是职责由“程序需要哪些名词”而非具体行为划分。恢复方法是先写成功/失败句子，再把每条规则放到拥有足够信息的最小组件。

立即练习：为“订单数量大于行情可用数量时不成交”补一句用例，并指出规则属于 `SimulatedExchange` 而不是 `Portfolio`。原因是撮合时已经同时拥有订单与行情，组合不应重新推断市场流动性。

8.2 值对象：构造后始终有效

8.2.1 场景：不让非法订单流入后续组件

`Order` 和 `Fill` 是值对象：它们表示事实，拥有自己的字符串和数值，复制后两份对象互不依赖。核心声明位于：

Listing 8.1: 带构造不变量的领域值对象

```

1  #ifndef QUANT_CH08_DOMAIN_HPP
2  #define QUANT_CH08_DOMAIN_HPP
3
4  #include <cmath>
5  #include <cstdint>
6  #include <stdexcept>
7  #include <string>
8  #include <utility>
9
10 namespace quant::ch08 {
11
12     enum class Side : std::uint8_t { buy, sell };
13
14     namespace detail {
15
16         void require_symbol(const std::string& symbol);
17         void require_price(double price);
18         void require_quantity(std::int64_t quantity);
19
20     } // namespace detail
21
22     class MarketEvent final {
23     public:
24         MarketEvent(std::string symbol, double price, std::int64_t quantity)
25             : symbol_(std::move(symbol)), price_(price), quantity_(quantity) {
26             detail::require_symbol(symbol_);
27             detail::require_price(price_);
28             detail::require_quantity(quantity_);
29         }
30
31         const std::string& symbol() const { return symbol_; }
32         double price() const { return price_; }
33         std::int64_t quantity() const { return quantity_; }
34

```

```

35 private:
36     std::string symbol_;
37     double price_;
38     std::int64_t quantity_;
39 };
40
41 class Order final {
42 public:
43     Order(std::string symbol, Side side, std::int64_t quantity)
44         : symbol_(std::move(symbol)), side_(side), quantity_(quantity) {
45         detail::require_symbol(symbol_);
46         detail::require_quantity(quantity_);
47     }
48
49     const std::string& symbol() const { return symbol_; }
50     Side side() const { return side_; }
51     std::int64_t quantity() const { return quantity_; }
52
53 private:
54     std::string symbol_;
55     Side side_;
56     std::int64_t quantity_;
57 };
58
59 class Fill final {
60 public:
61     Fill(std::string symbol, Side side, std::int64_t quantity, double price)
62         : symbol_(std::move(symbol)),
63         side_(side),
64         quantity_(quantity),
65         price_(price) {
66         detail::require_symbol(symbol_);
67         detail::require_quantity(quantity_);
68         detail::require_price(price_);
69     }
70
71     const std::string& symbol() const { return symbol_; }
72     Side side() const { return side_; }
73     std::int64_t quantity() const { return quantity_; }
74     double price() const { return price_; }
75
76 private:
77     std::string symbol_;
78     Side side_;
79     std::int64_t quantity_;
80     double price_;
81 };
82
83 struct PortfolioSnapshot final {

```

```

84     std::string symbol;
85     std::int64_t quantity{};
86     double cash{};
87     double equity{};
88 };
89
90 } // namespace quant::ch08
91
92 #endif

```

构造函数先接收参数，再初始化私有成员，最后验证。空代码、非正数量、非正或非有限价格都会被拒绝，并抛出 `invalid_argument`。因此一旦调用者持有 `Fill`，组合无需在每次读取时再次猜测价格是否为 `NaN`。

`std::isfinite` 很重要：对 `NaN`，`price <= 0.0` 为假，只检查大小会让 `NaN` 漏过。条件必须写成“不是有限数，或者不为正”。无效外部 CSV 如何保留行号和字段上下文留到第 10 章；本章只建立核心对象不能代表非法领域事实。

类没有手写析构、复制构造或复制赋值。`string` 自己管理资源，编译器生成的成员逐一复制正是需要的语义，这称为 *rule of zero*。复制一笔订单不会创建共享可变字段；把副本交给交易所后，原订单仍是独立值。

`PortfolioSnapshot` 则是只读观察结果的简单 `struct`。它不负责被重新送入组合更新，因此公开数据记录比一组机械 `getter` 更直接。选择 `class` 还是 `struct` 不是“复杂与简单”的区别，而是是否需要用访问控制维护构造与修改规则。

注Python 迁移提示：Python 的 `b = a` 通常让两个名字指向同一对象；要得到独立对象需显式复制。C++ 的 `Order b = a;` 默认复制成员，`a` 与 `b` 是两个对象。反过来，`const Order&` 才是借用同一个对象且不复制。判断接口成本时要先分清按值拥有、按引用借用和移动所有权。

失败诊断：若提供 `set_quantity(-5)`，对象可能先有效、后失效，下游每层都被迫防御。根因是修改入口破坏了构造契约。恢复方案通常是创建一个新的合法值，或提供一个名称体现完整业务动作的方法，而不是开放任意字段写入。

立即练习：在 `code/ch08/tests/test_invalid_values.cpp` 中把零数量改为负数，把 `NaN` 改为正无穷。两种构造仍应抛出 `invalid_argument`，测试最终精确输出 `domain invariants ok`。

8.3 正常缺席不是非法对象

8.3.1 场景：策略可以合理地不下单

高于阈值或已有持仓时，“没有订单”是正常业务结果，不应伪造数量为零的订单，也不应抛异常。策略返回 `std::optional<Order>`：有值表示一笔完整合法订单，无值表示本次不行动。

Listing 8.2: 策略接口、optional 与阈值实现

```

1  #ifndef QUANT_CH08_STRATEGY_HPP
2  #define QUANT_CH08_STRATEGY_HPP
3
4  #include <cmath>
5  #include <cstdint>
6  #include <optional>
7  #include <stdexcept>
8
9  #include "quant/ch08/domain.hpp"
10

```

```

11 namespace quant::ch08 {
12
13 class Strategy {
14 public:
15     virtual ~Strategy() = default;
16
17     virtual std::optional<Order> on_event(
18         const MarketEvent& event,
19         const PortfolioSnapshot& portfolio) const = 0;
20 };
21
22 class ThresholdStrategy final : public Strategy {
23 public:
24     ThresholdStrategy(double threshold, std::int64_t order_quantity)
25         : threshold_(threshold), order_quantity_(order_quantity) {
26         if (!std::isfinite(threshold_) || threshold_ <= 0.0) {
27             throw std::invalid_argument{"threshold must be positive and finite"};
28         }
29         if (order_quantity_ <= 0) {
30             throw std::invalid_argument{"order quantity must be positive"};
31         }
32     }
33
34     std::optional<Order> on_event(
35         const MarketEvent& event,
36         const PortfolioSnapshot& portfolio) const override {
37         if (portfolio.symbol != event.symbol()) {
38             throw std::logic_error{"portfolio snapshot symbol does not match event"};
39         }
40         if (event.price() > threshold_ || portfolio.quantity != 0) {
41             return std::nullopt;
42         }
43         return Order{event.symbol(), Side::buy, order_quantity_};
44     }
45
46 private:
47     double threshold_;
48     std::int64_t order_quantity_;
49 };
50
51 } // namespace quant::ch08
52
53 #endif

```

`optional<T>` 要么包含一个 `T`，要么为空。`has_value()` 查询状态；确认有值后，可用 `*order` 或 `order->quantity()` 访问其中对象。`std::nullopt` 明确构造空状态。它不解释“为何没有”，因此适合这里单一而正常的缺席。需要错误原因、外部故障或诊断上下文时，第 10 章会比较其他机制。

`ThresholdStrategy` 的构造参数同样有不变量。若把非法下单数量留到 `on_event` 才判断，错误配置可能潜伏到某条特定行情才出现。配置对象应在建成时就有效。

策略行为测试只从公开 seam 观察结果：低价空仓得到 AAPL 买单，高价与已有持仓都得到空 optional。测试不检查阈值字段叫什么，也不要求内部执行几次比较，所以成员布局可以重构而不改测试。

注Python 迁移提示：Python 常用 None 表示没有订单；optional<Order> 把这条可能性写进静态类型，调用者不能把结果直接当 Order 使用。它仍不等于 Python 的异常：无值是预期分支，非法策略配置才在构造边界失败。

失败诊断：用 Order{"AAPL", buy, 0} 表示“不下单”会与“非法订单”混在一起，而且该对象根本无法通过本章构造函数。恢复方案是让值对象只表达有效事实，用 optional 表达事实可能缺席。

立即练习：把阈值改为 98。测试中的 99 元事件应不再产生订单；先观察策略 seam 以“cheap event should create an order”失败，再同步修改用例或恢复阈值。失败消息证明测试保护的是行为而非内部实现。

8.4 组合是默认装配方式

8.4.1 场景：策略、交易所和组合协作但互不拥有对方内部状态

SimulatedExchange 接收订单和行情，代码不匹配或流动性不足时返回空 optional；否则创建合法成交。Portfolio 拥有现金和仓位表，只通过 apply 修改状态：

Listing 8.3: 撮合与组合不变量

```

1  #ifndef QUANT_CH08_EXECUTION_HPP
2  #define QUANT_CH08_EXECUTION_HPP
3
4  #include <cmath>
5  #include <cstdint>
6  #include <optional>
7  #include <stdexcept>
8  #include <string>
9
10 #include "quant/ch08/domain.hpp"
11
12 namespace quant::ch08 {
13
14 class SimulatedExchange final {
15 public:
16     std::optional<Fill> match(const Order& order,
17                             const MarketEvent& event) const {
18         if (order.symbol() != event.symbol() ||
19             order.quantity() > event.quantity()) {
20             return std::nullopt;
21         }
22         return Fill{order.symbol(), order.side(), order.quantity(), event.price()};
23     }
24 };
25
26 class Portfolio final {
27 public:
28     explicit Portfolio(double initial_cash) : cash_(initial_cash) {
29         if (!std::isfinite(cash_) || cash_ < 0.0) {
30             throw std::invalid_argument{"initial cash must be finite and non-negative"};
31         }
32     }

```



```

32     }
33
34     void apply(const Fill& fill) {
35         const double notional = static_cast<double>(fill.quantity()) * fill.price();
36
37         if (!symbol_.empty() && symbol_ != fill.symbol()) {
38             throw std::logic_error{"portfolio supports one symbol per run"};
39         }
40         if (fill.side() == Side::sell && quantity_ < fill.quantity()) {
41             throw std::logic_error{"sell quantity exceeds position"};
42         }
43         if (fill.side() == Side::buy && cash_ < notional) {
44             throw std::logic_error{"buy notional exceeds cash"};
45         }
46
47         if (symbol_.empty()) {
48             symbol_ = fill.symbol();
49         }
50         if (fill.side() == Side::buy) {
51             quantity_ += fill.quantity();
52             cash_ -= notional;
53         } else {
54             quantity_ -= fill.quantity();
55             cash_ += notional;
56         }
57     }
58
59     PortfolioSnapshot snapshot(const std::string& symbol,
60                               double mark_price) const {
61         if (symbol.empty()) {
62             throw std::invalid_argument{"symbol must not be empty"};
63         }
64         if (!std::isfinite(mark_price) || mark_price <= 0.0) {
65             throw std::invalid_argument{"mark price must be positive and finite"};
66         }
67         if (!symbol_.empty() && symbol_ != symbol) {
68             throw std::logic_error{"snapshot symbol does not match portfolio"};
69         }
70         return PortfolioSnapshot{
71             symbol, quantity_, cash_,
72             cash_ + static_cast<double>(quantity_) * mark_price};
73     }
74
75 private:
76     double cash_;
77     std::string symbol_;
78     std::int64_t quantity_{0};
79 };
80

```

```

81 } // namespace quant::ch08
82
83 #endif

```

`apply` 先读取当前仓位并计算名义金额，然后完成所有可能失败的检查。只有验证通过才写 `positions_` 和 `cash_`。因此超卖或现金不足时，异常离开函数，组合的可观察快照保持不变。本章只验证这个具体性质；异常安全术语和更复杂的回滚策略留到第 10 章。

主程序沿用第 7 章“文件边界只负责读取，核心接收已验证对象”的数据路径：`event_reader.cpp` 把本快照自带的 `events.csv` 转为事件值；随后按值拥有具体 `ThresholdStrategy`、`SimulatedExchange` 和 `Portfolio`。每个对象的生命都覆盖事件循环，所有权不需要智能指针：

Listing 8.4: 由具体组件组合成事件管道

```

1  #include <cstdint>
2  #include <iomanip>
3  #include <iostream>
4  #include <stdexcept>
5
6  #include "quant/ch08/execution.hpp"
7  #include "quant/ch08/event_reader.hpp"
8  #include "quant/ch08/strategy.hpp"
9
10 int main(int argc, char* argv[]) {
11     if (argc != 2) {
12         std::cerr << "usage: ch08_domain_pipeline <events.csv>\n";
13         return 2;
14     }
15
16     try {
17         const auto events = quant::ch08::read_events(argv[1]);
18
19         const quant::ch08::ThresholdStrategy threshold{100.0, 25};
20         const quant::ch08::Strategy& strategy = threshold;
21         const quant::ch08::SimulatedExchange exchange;
22         quant::ch08::Portfolio portfolio{10'000.0};
23         std::size_t order_count = 0;
24         std::size_t fill_count = 0;
25
26         for (const quant::ch08::MarketEvent& event : events) {
27             const auto before = portfolio.snapshot(event.symbol(), event.price());
28             const auto order = strategy.on_event(event, before);
29             if (!order.has_value()) {
30                 continue;
31             }
32             ++order_count;
33             const auto fill = exchange.match(*order, event);
34             if (fill.has_value()) {
35                 portfolio.apply(*fill);
36                 ++fill_count;
37             }

```

```

38     }
39
40     const quant::ch08::MarketEvent& last = events.back();
41     const auto result = portfolio.snapshot(last.symbol(), last.price());
42     std::cout << std::fixed << std::setprecision(2)
43         << "orders=" << order_count << " fills=" << fill_count
44         << " symbol=" << result.symbol
45         << " quantity=" << result.quantity << " cash=" << result.cash
46         << " equity=" << result.equity << '\n';
47 } catch (const std::exception& error) {
48     std::cerr << "error=" << error.what() << '\n';
49     return 2;
50 }
51 }

```

这就是组合：较大行为由现成对象协作得到，而不是通过继承把交易所、策略和组合塞进同一父类。默认先选具体成员和值成员，因为依赖、生命和调用成本在代码中直接可见。只有真实需求要求运行时替换一种能力时，才引入动态接口。

成交/组合 seam 测试手算两笔成交。99 元买 25 股后现金 7525；101 元卖 10 股后现金 8535、持仓 15、权益 10050。它还验证错误代码与流动性不足都不成交、超卖与第二个代码都在修改前被拒绝。策略 seam 另外拒绝事件与快照代码不一致。测试只调用 match、apply 与 snapshot，不读取私有状态。本章快照明确限定一次运行一个代码；多资产总权益需要一组完整估值价格，留到第 17 章统一设计，不能只拿最后一个代码的价格冒充总权益。

注Python 迁移提示： Python 可把策略对象作为参数传入循环，只要它有所需方法；对象生命由引用计数或垃圾回收协助管理。C++ 的具体局部对象让所有权和销毁点完全可见，引用参数只借用它们。不要为了模仿 Python 的可替换性，提前把每个组件都放进 shared_ptr。

失败诊断：若 apply 先扣现金、随后才发现超卖，异常会留下半更新状态。根因不是异常本身，而是把修改放在完整验证之前。恢复方法是先算候选结果并验证，再一次提交相关字段。

立即练习：运行 code/ch08/exercises/sell_down_solution.cpp，解释为何买入 25、卖出 10 后精确输出 quantity=15 cash=8535.00 equity=10050.00。再把卖出数量改为 26，程序应在产生输出前以状态不一致失败。

8.5 什么时候才需要运行时多态

8.5.1 场景：启动后按配置选择一种策略

若可执行文件启动时读取配置，在阈值策略和持有策略中选择一个，而事件循环不应写两套分支，就需要通过同一运行时接口调用不同具体类型。Strategy 使用虚函数表达这种契约：

以下只摘出两个成员声明，省略了所属类、头文件和命名空间上下文；完整可编译定义就是前文引入的 strategy.hpp：

```

1 virtual std::optional<Order> on_event(
2     const MarketEvent&, const PortfolioSnapshot&) const = 0;
3
4 std::optional<Order> on_event(
5     const MarketEvent&, const PortfolioSnapshot&) const override;

```

= 0 使成员成为纯虚函数，基类不能单独实例化；通过 `Strategy&` 或指针调用时，运行期选择实际覆盖函数。基类析构函数为虚函数，保证未来通过基类指针拥有派生对象时销毁完整对象。`override` 要求签名确实覆盖基类函数，`final` 禁止继续派生。

下面的完整程序让同一个 `creates_order(const Strategy&)` 分别接收阈值策略和持有策略，精确输出 `threshold=1 hold=0`。这里引用不拥有策略；调用者必须保证实际对象仍存活。

Listing 8.5: 通过基类引用运行时替换策略

```

1  #include <iostream>
2  #include <optional>
3
4  #include "quant/ch08/strategy.hpp"
5
6  class HoldStrategy final : public quant::ch08::Strategy {
7  public:
8      std::optional<quant::ch08::Order> on_event(
9          const quant::ch08::MarketEvent&,
10         const quant::ch08::PortfolioSnapshot&) const override {
11         return std::nullopt;
12     }
13 };
14
15 bool creates_order(const quant::ch08::Strategy& strategy) {
16     const quant::ch08::MarketEvent event{"AAPL", 99.0, 1'000};
17     const quant::ch08::PortfolioSnapshot flat{"AAPL", 0, 10'000.0, 10'000.0};
18     return strategy.on_event(event, flat).has_value();
19 }
20
21 int main() {
22     const quant::ch08::ThresholdStrategy threshold{100.0, 25};
23     const HoldStrategy hold;
24     std::cout << "threshold=" << creates_order(threshold)
25               << " hold=" << creates_order(hold) << '\n';
26 }

```

动态分派有成本：对象通常携带虚表关联信息，调用可能是间接调用，接口演化还影响 ABI；更重要的是所有实现都必须服从同一个运行时契约。若类型在编译时已知，直接组合最简单。若需要对许多类型复用算法且愿意在编译期实例化，第 9 章再引入模板与 `concepts`；不要把“可替换”自动等同于“必须继承”。

故意失败的 `code/ch08/failures/wrong_override.cpp` 把基类的 `const double&` 参数误写为 `double&`，却标记 `override`。编译器必须报告该函数没有覆盖基类成员。去掉 `override` 可能让错误潜伏为新重载，并使派生类仍是抽象类；正确修复是让签名完全一致。

立即练习：在 `HoldStrategy::on_event` 参数末尾删除成员函数的 `const`，保留 `override`。预测并验证编译失败；恢复 `const` 后输出重新变为 `threshold=1 hold=0`。

8.6 独立快照与验收命令

在 WSL 中从项目根目录执行：

```
1 cmake -S code/ch08 -B build/ch08
```

```

2 cmake --build build/ch08 -j
3 ctest --test-dir build/ch08 --output-on-failure
4 ./build/ch08/ch08_domain_pipeline code/ch08/data/events.csv

```

最后一行必须与章首输出逐字一致。根项目还运行 8 项 ch08 检查：策略 seam、成交/组合 seam、主流水线、运行时替换、非法值、核心练习、错误覆盖诊断以及从空目录配置的快照。Windows 可选择 Visual Studio 生成器；多配置生成器的可执行文件通常位于 Debug 或 Release 子目录，接口与值语义规则不变。

8.7 自测、编码任务与面试检查点

1. 为什么先写成功和失败用例，比先列 getter/setter 更能确定接口？
2. Order 与 Fill 的哪些不变量由构造函数维护？为什么只检查 `price <= 0` 不够？
3. 值语义复制与 `const T&` 借用在所有权和成本上有何不同？
4. 为什么“不下单”应是空 optional，而不是数量为零的 Order？
5. 撮合失败和非法成交对象分别属于正常缺席还是契约错误？
6. `Portfolio::apply` 为什么先验证再修改？哪项测试观察了失败后的状态？
7. 组合、具体值成员与运行时多态分别适合什么需求？
8. `virtual`、`=0`、`override`、`final` 各提供什么约束？

编码任务：从 10000 现金开始，应用 99 元买入 25 股和 101 元卖出 10 股两笔成交，再以 101 元估值。可运行答案位于 `code/ch08/exercises/sell_down_solution.cpp`；输出必须精确为 `quantity=15 cash=8535.00 equity=10050.00`。扩展题是尝试应用一笔 MSFT 成交，验证程序在任何现金或数量改变前以“单次运行只支持一个代码”的状态错误拒绝它。

问题 8.1 面试检查点 被问“什么时候用继承”时，先说默认用值语义和组合表达明确所有权；只有调用方必须在运行时通过同一契约替换实现时，才考虑抽象接口。随后说明虚析构、`override`、对象生命周期和间接调用成本，并指出编译期已知类型将在第 9 章用模板方案比较。

8.8 本章小结

接口来自用例和失败边界，而不是成员字段清单。值对象在构造后保持有效，optional 表达正常缺席，组合让策略、撮合与组合各守职责。组合状态在验证完成后才修改；运行时多态只服务于真实的运行时替换需求。到此，行情分析器已经演化为可独立构建、行为受测且不变量明确的领域组件快照。

第九章 模板、concepts 与多态

内容提要

- ❑ 从只接受一种策略的普通函数演化出函数模板
- ❑ 用 `requires` 表达式和 `concept` 给泛型接口命名并约束能力
- ❑ 读懂模板参数、实参推导、实例化与类模板语法
- ❑ 比较静态多态与运行时多态的适用场景及工程代价

注 先修知识：已完成第 3–8 章，会使用函数、类、optional、值语义、组合和虚函数接口。

Python 迁移边界：Python 函数天然可以接收任何具有所需方法的对象；C++ 模板也按表达式能力工作，但会在编译期为实际类型检查并实例化代码。Python Protocol 更接近 `concept`，而抽象基类更接近虚函数接口。

常见错误与诊断：模板错误先找最早的“哪项表达式无效”或“哪项约束不满足”；运行结果错误则回到策略决策 `seam`，不能把“编译通过”误当作业务语义正确。

本章保持第 8 章的领域边界与同一公开行为：事件仍验证非空代码、正有限价格和正数量，订单仍验证非空代码和正数量；99 元、空仓的 AAPL 事件使阈值策略下单，持有策略不下单。最终独立快照必须精确输出：

```
1 threshold_orders=1 hold_orders=0
```

完成判据还包括：普通函数与函数模板给出相同行为；无约束模板和 `concept` 约束分别产生稳定的简化诊断；静态与动态路径都得到买单；`code/ch09` 可从空构建目录独立配置。

9.1 从普通函数开始

9.1.1 场景：先让一种策略工作

第 8 章的事件循环只需要调用策略的 `on_event`。最直接的第一版函数把具体类型写进参数：

Listing 9.1: 只接受阈值策略的普通函数

```
1 #include <iostream>
2 #include <optional>
3 #include <string>
4
5 #include "quant/ch09/strategies.hpp"
6
7 std::optional<quant::ch09::Order> decide_threshold(
8     const quant::ch09::ThresholdStrategy& strategy,
9     const quant::ch09::MarketEvent& event,
10    const quant::ch09::PortfolioSnapshot& portfolio) {
11    return strategy.on_event(event, portfolio);
12 }
13
14 std::string order_label(const std::optional<quant::ch09::Order>& order) {
15     if (order.has_value()) {
16         return "buy";
17     }
18     return "none";
19 }
```

```

19 }
20
21 int main() {
22     const quant::ch09::ThresholdStrategy strategy{100.0, 25};
23     const quant::ch09::MarketEvent event{"AAPL", 99.0, 1'000};
24     const quant::ch09::PortfolioSnapshot flat{"AAPL", 0, 10'000.0, 10'000.0};
25     const auto order = decide_threshold(strategy, event, flat);
26     std::cout << "ordinary=" << order_label(order) << '\n';
27 }

```

`decide_threshold` 不是模板。它只接受 `ThresholdStrategy` 的只读引用，调用方传入 `HoldStrategy` 会在重载选择阶段失败。这个限制在只有一种策略时完全合理：签名清楚、诊断直接、实现放在源文件也容易保持 ABI 边界。

程序用固定事件和空仓快照调用它，精确输出 `ordinary=buy`。先保存这条可运行行为，再泛化接口；否则直接面对 `concept` 最终代码，读者无法判断每个语法元素解决了什么重复。

注Python 迁移提示： Python 的函数通常不声明具体参数类型，只要传入对象在运行到该行时具有 `on_event` 就能继续。C++ 普通函数把类型要求写在签名里，因此编译器在调用前就能拒绝其他类型。两者都可以从最具体实现开始，不应为了“以后可能有很多策略”预先设计复杂层次。

失败诊断：若调用 `decide_threshold(hold, event, flat)`，编译器会报告无法把 `HoldStrategy` 绑定到 `const ThresholdStrategy&`。根因不是持有策略缺少方法，而是函数签名点名了另一种类型。若确实需要复用同一算法，下一步才引入模板。

立即练习：把事件价格从 99 改为 101，其他代码不变。输出应变为 `ordinary=none`；若仍为 `buy`，先检查阈值比较而不是模板语法。

9.2 函数模板：把类型变成参数

9.2.1 场景：阈值策略和持有策略共享一次调用逻辑

普通函数的函数体并不依赖阈值策略的私有字段，只要求参数能执行 `on_event(event, portfolio)`。可以把具体类型换成模板参数：

Listing 9.2: 从普通函数演化出的函数模板

```

1  #include <iostream>
2  #include <optional>
3  #include <string>
4
5  #include "quant/ch09/strategies.hpp"
6
7  template <typename Strategy>
8  std::optional<quant::ch09::Order> decide(
9      const Strategy& strategy, const quant::ch09::MarketEvent& event,
10     const quant::ch09::PortfolioSnapshot& portfolio) {
11     return strategy.on_event(event, portfolio);
12 }
13
14 std::string order_label(const std::optional<quant::ch09::Order>& order) {
15     if (order.has_value()) {
16         return "buy";

```



```

17     }
18     return "none";
19 }
20
21 int main() {
22     const quant::ch09::MarketEvent event{"AAPL", 99.0, 1'000};
23     const quant::ch09::PortfolioSnapshot flat{"AAPL", 0, 10'000.0, 10'000.0};
24     const quant::ch09::ThresholdStrategy threshold{100.0, 25};
25     const quant::ch09::HoldStrategy hold;
26
27     std::cout << "threshold=" << order_label(decide(threshold, event, flat))
28               << " hold=" << order_label(decide(hold, event, flat))
29               << '\n';
30 }

```

`template <typename Strategy>` 声明接下来的函数是模板，并引入类型参数 `Strategy`。函数参数写 `const Strategy&`；调用 `decide(threshold, ...)` 时，编译器从第一个实参推导 `Strategy` 为 `ThresholdStrategy`，调用持有策略时则推导为 `HoldStrategy`。

模板本身是生成规则。只有使用某组模板实参时才形成对应的函数特化，这个过程称为实例化。本程序至少需要两个实例：一个接受阈值策略，一个接受持有策略。它们共享源码写法，却是不同的编译期函数实体，精确输出 `threshold=buy hold=none`。

编译器实例化时必须看到模板定义，不能像普通非模板函数那样只在头文件留声明、把通用定义藏进一个无人可见的源文件。常见做法是把模板定义放在头文件；大型项目也可显式实例化已知类型，但会限制可用类型集合并增加构建设计成本。

注Python 迁移提示：Python 只保留一份函数对象，运行时对不同对象执行相同字节码；C++ 模板通常为实际类型分别实例化机器代码。Python 类型提示不会自动生成新函数，C++ 模板参数也不是运行时变量，不能在程序执行中突然换成另一种类型。

失败诊断：传入整数 42 时，模板推导成功，因为任何类型都能先成为 `Strategy`；错误直到实例化函数体中的 `strategy.on_event` 才出现。根因是模板参数没有说明所需能力，不是推导失败。

立即练习：删除 `HoldStrategy` 调用并重新构建。输出只剩阈值结果，同时编译器不再需要持有策略对应的 `decide` 实例；源码模板仍然存在。

9.3 concept: 给所需能力命名

9.3.1 场景：把“像策略”写成可检查契约

本章策略不仅要有同名方法，返回类型还必须恰好是 `optional<Order>`。C++20 concept 可以给这组编译期要求命名：

Listing 9.3: `requires` 表达式与受约束的类模板

```

1 #ifndef QUANT_CH09_STRATEGY_RUNNER_HPP
2 #define QUANT_CH09_STRATEGY_RUNNER_HPP
3
4 #include <cstdint>
5 #include <concepts>
6 #include <optional>
7 #include <utility>

```

```

8  #include <vector>
9
10 #include "quant/ch09/domain.hpp"
11
12 namespace quant::ch09 {
13
14 template <typename Strategy>
15 concept MarketStrategy = requires(const Strategy& strategy,
16                                   const MarketEvent& event,
17                                   const PortfolioSnapshot& portfolio) {
18     { strategy.on_event(event, portfolio) }
19     -> std::same_as<std::optional<Order>>;
20 };
21
22 template <MarketStrategy Strategy>
23 class StrategyRunner final {
24 public:
25     explicit StrategyRunner(Strategy strategy) : strategy_(std::move(strategy)) {}
26
27     std::size_t count_orders(
28         const std::vector<MarketEvent>& events,
29         const PortfolioSnapshot& portfolio) const {
30         std::size_t count = 0;
31         for (const MarketEvent& event : events) {
32             if (strategy_.on_event(event, portfolio).has_value()) {
33                 ++count;
34             }
35         }
36         return count;
37     }
38
39 private:
40     Strategy strategy_;
41 };
42
43 } // namespace quant::ch09
44
45 #endif

```

第一行模板参数仍允许候选类型进入检查。concept `MarketStrategy = requires (...)` 定义一个布尔编译期谓词；圆括号中的 `strategy`、`event` 和 `portfolio` 只是供要求表达式使用的局部占位，不会构造运行时对象。

花括号中的复合要求检查调用表达式有效，箭头后的 `std::same_as<std::optional<Order>>` 检查表达式类型完全一致。concept 检查的是语法与类型能力：它不能证明策略一定只在低价下单，也不能证明数量符合风控规则。

类模板声明把 `MarketStrategy` 用作类型参数约束。以 `ThresholdStrategy` 作为 `StrategyRunner` 的显式模板实参时，尖括号中的类型决定成员 `strategy_` 的真实类型。两个不同策略会得到两个不同的运行器特化，彼此不能直接赋值。

构造函数按值接收策略并移动到成员，运行器拥有策略对象。`count_orders` 负责遍历一批事件、调用所持策略并汇总订单数；调用方提供同一持仓快照，因而这里刻意不模拟成交后的仓位变化。类模板既固定一种编译期策略，也封装批处理职责，不是只转发一个成员函数的空包装。

注Python 迁移提示： Python 的 `typing.Protocol` 也能描述“具有某方法并返回某类型”的结构化能力，但主要由静态检查器使用，运行时不一定强制。C++ `concept` 直接参与模板候选选择；不满足时该重载或特化不可用。两者都不能证明方法的业务语义，只能约束可表达的接口形状。

失败诊断：若策略返回 `Order` 而不是 `optional<Order>`，调用表达式本身有效，但 `same_as` 要求失败。修复应先判断领域契约：若“无订单”是正常结果，就修改策略返回 `optional`；不要为了让 `concept` 通过而放宽成与业务不一致的任意类型。

立即练习：把 `HoldStrategy::on_event` 返回类型暂时改为 `bool`，并把函数体改成 `return false;`。构建持有策略对应的运行器特化时，应报告 `MarketStrategy` 约束不满足；恢复原返回类型与函数体后，静态策略 `seam` 重新绿色。

9.4 约束改善入口诊断，不证明运行结果

9.4.1 场景：同样把整数误当策略

`code/ch09/failures/unconstrained_strategy.cpp` 与 `code/ch09/failures/constrained_strategy.cpp` 都故意把 42 传给泛型 `decide`。它们与绿色目标隔离，只由编译失败测试运行。

在 WSL 的项目根目录可直接触发两种失败：

```
1 g++ -std=c++20 -c code/ch09/failures/unconstrained_strategy.cpp
2 g++ -std=c++20 -c code/ch09/failures/constrained_strategy.cpp
```

两条命令都应以非零状态退出：第一条暴露函数体中的成员访问错误，第二条在 `concept` 入口报告约束不满足。实验结束后无需改动绿色文件；若在练习中换过候选类型，恢复原故障文件或改传满足接口的类型即可回到可构建状态。

无约束版本先实例化函数体，再报告整数没有 `on_event` 成员。真实模板库中，这类错误可能嵌套许多层。受约束版本在候选入口检查 `EventStrategy`，诊断类别变为“constraints not satisfied”，并可继续指出 `requires` 中哪条表达式无效。测试只匹配类别，不锁定 GCC 与 Clang 的整段措辞。

`concept` 改善的是错误发生位置和接口文档，并不替代行为测试。一个 `AlwaysBuyStrategy` 完全可以满足返回类型，却在任何价格下买入。`code/ch09/tests/test_static_strategy.cpp` 仍通过公开策略 `seam` 验证：阈值策略给出一笔数量 25 的 AAPL 订单，持有策略没有订单。

注Python 迁移提示： Python 静态检查器也能在调用前指出对象不满足 `Protocol`；若不运行检查器，错误可能到执行方法访问时才出现。C++ `concept` 检查属于编译本身，但它同样只看到声明的能力，无法替代 `pytest` 中的业务样例。

失败诊断：看到长模板诊断时，从最靠前的用户代码调用点和第一条未满足要求开始，不要从最末尾的库实现逐字倒读。先把参与类型写出来，再核方法的参数 `cv/ref` 限定和精确返回类型。

立即练习：在受约束失败示例中为整数换成一个含 `int on_event(const Event&) const` 的小类。约束应通过；随后把返回类型改为 `double`，观察 `same_as<int>` 失败。

9.5 静态与动态多态的真实取舍

9.5.1 场景：类型何时确定

第 8 章的虚函数接口允许启动时读取配置，再通过 `Strategy&` 调用所选实现；本章模板要求调用点编译时知道具体类型。下面用完全相同的事件和阈值比较两条路径：

Listing 9.4: 相同策略行为的静态与动态路径

```

1  #include <iostream>
2  #include <cstdint>
3  #include <optional>
4  #include <string>
5  #include <vector>
6
7  #include "quant/ch09/strategy_runner.hpp"
8  #include "quant/ch09/strategies.hpp"
9
10 class RuntimeStrategy {
11 public:
12     virtual ~RuntimeStrategy() = default;
13     virtual std::optional<quant::ch09::Order> on_event(
14         const quant::ch09::MarketEvent& event,
15         const quant::ch09::PortfolioSnapshot& portfolio) const = 0;
16 };
17
18 class RuntimeThreshold final : public RuntimeStrategy {
19 public:
20     RuntimeThreshold(double threshold, std::int64_t quantity)
21         : strategy_(threshold, quantity) {}
22
23     std::optional<quant::ch09::Order> on_event(
24         const quant::ch09::MarketEvent& event,
25         const quant::ch09::PortfolioSnapshot& portfolio) const override {
26         return strategy_.on_event(event, portfolio);
27     }
28
29 private:
30     quant::ch09::ThresholdStrategy strategy_;
31 };
32
33 std::string order_label(const std::optional<quant::ch09::Order>& order) {
34     if (order.has_value()) {
35         return "buy";
36     }
37     return "none";
38 }
39
40 std::string count_label(std::size_t order_count) {
41     if (order_count == 1) {

```

```

42     return "buy";
43 }
44 return "none";
45 }
46
47 int main() {
48     const quant::ch09::MarketEvent event{"AAPL", 99.0, 1'000};
49     const std::vector<quant::ch09::MarketEvent> events{event};
50     const quant::ch09::PortfolioSnapshot flat{"AAPL", 0, 10'000.0, 10'000.0};
51     const quant::ch09::StrategyRunner<quant::ch09::ThresholdStrategy> static_path{
52         quant::ch09::ThresholdStrategy{100.0, 25}};
53     const RuntimeThreshold runtime_object{100.0, 25};
54     const RuntimeStrategy& dynamic_path = runtime_object;
55
56     std::cout << "static=" << count_label(static_path.count_orders(events, flat))
57             << " dynamic=" << order_label(dynamic_path.on_event(event, flat))
58             << '\n';
59 }

```

两条路径都必须输出 `buy`，说明选择分派机制不应改变领域行为。动态路径通过虚函数表关联的间接调用选择覆盖函数，能把实现隐藏在稳定基类接口之后，也适合插件或运行时配置。静态路径的具体类型进入模板实例，编译器有机会内联并跨调用优化，不需要基类继承关系。

“静态一定更快”不是可靠结论。虚分派成本可能被解析、网络或数值计算淹没；模板内联也可能扩大指令体积。性能必须在第 14 章用目标工作负载测量。本章能确定的工程权衡是：

问题	静态多态/模板	动态多态/虚函数
具体类型何时知道	编译期	运行期可替换
接口检查	<code>concept</code> 与实例化	基类虚函数签名
代码生成	可能按类型重复	通常共享调用路径
实现可见性	模板定义常在头文件	可隐藏在源文件/库后
ABI 边界	头文件变化常触发重编译	稳定接口可隔离实现
调用形态	可直接调用、可内联	通常间接虚调用

模板会增加解析和实例化工作；类型组合多时，编译时间与代码体积可能上升。暴露模板定义也让调用方依赖更多头文件细节。虚接口则承担对象生命、虚析构、间接调用和 ABI 演化约束。选择来自部署与替换需求，不能只凭 “modern” 标签。

注Python 迁移提示： Python 常在运行时替换策略对象，天然接近动态分派；Numba、Cython 或专门化 JIT 又可能根据具体类型生成代码，接近静态专门化。C++ 把选择显式放在模板或虚接口中，因此类型确定时机与构建成本更容易在源码中审查。

失败诊断： 若为了允许一个运行时配置而实例化所有策略组合并写巨大 `variant` 分支，模板复杂度可能超过收益；反过来，若热循环中的策略集合在编译时固定，却强制所有对象经共享指针和虚接口访问，也可能增加生命管理负担。先写替换时机，再选机制。

立即练习： 把 `RuntimeStrategy&` 改为具体 `RuntimeThreshold&`。输出不变，但调用点不再展示运行时替换能力；这说明 “用了继承类” 本身不等于调用发生了动态分派。

9.6 独立快照与验收命令

最终快照用受 `concept` 约束的类模板保存阈值策略和持有策略，对两条固定事件计数：

Listing 9.5: 有约束的泛型策略运行器快照

```

1 #include <iostream>
2 #include <vector>
3
4 #include "quant/ch09/strategy_runner.hpp"
5 #include "quant/ch09/strategies.hpp"
6
7 int main() {
8     const std::vector<quant::ch09::MarketEvent> events{
9         {"AAPL", 99.0, 1'000}, {"AAPL", 101.0, 800}};
10    const quant::ch09::PortfolioSnapshot flat{
11        "AAPL", 0, 10'000.0, 10'000.0};
12    const quant::ch09::StrategyRunner<quant::ch09::ThresholdStrategy> threshold{
13        quant::ch09::ThresholdStrategy{100.0, 25}};
14    const quant::ch09::StrategyRunner<quant::ch09::HoldStrategy> hold{
15        quant::ch09::HoldStrategy{}};
16
17    std::cout << "threshold_orders=" << threshold.count_orders(events, flat)
18              << " hold_orders=" << hold.count_orders(events, flat) << '\n';
19 }
```

在 WSL 中从项目根目录执行：

```

1 cmake -S code/ch09 -B build/ch09
2 cmake --build build/ch09 -j
3 ctest --test-dir build/ch09 --output-on-failure
4 ./build/ch09/ch09_constrained_pipeline
```

最后一行必须与章首逐字一致。根项目还验证普通函数、函数模板、静态策略 `seam`、静态/动态对照、两种编译诊断、练习答案与干净快照。`concept` 只负责编译期能力，输出测试负责运行期语义，两类证据不可互相替代。

9.7 自测、编码任务与面试检查点

1. 普通函数的具体参数何时比模板更合适？
2. `template <typename Strategy>` 中类型参数的作用域是什么？调用时如何推导？
3. 什么是模板实例化？为什么模板定义通常需要对调用方可见？
4. 类模板 `StrategyRunner<ThresholdStrategy>` 中尖括号实参决定了什么？
5. `requires` 表达式中的局部参数会不会创建运行时对象？
6. `same_as<optional<Order>>` 检查什么，又不能证明什么？
7. 无约束与受约束失败的首要诊断类别有何不同？
8. 静态和动态多态在替换时机、编译成本、代码体积和 ABI 上如何取舍？

编码任务：实现一个最大持仓策略，空仓时可以买 25 股，持仓已达 50 股时不再下单，并把它交给同一个 `StrategyRunner`。完整答案位于 `code/ch09/exercises/maximum_position_solution.cpp`，精确输出必须为 `flat=buy full=none`。

问题 9.1 面试检查点 回答“模板与虚函数怎么选”时，先说明具体类型在编译期还是运行期确定，再比较 `concept` 接口、实例化与内联机会、虚分派、代码体积、编译时间和 ABI。最后强调 `concept` 只检查能力，业务正确性仍由同一策略 `seam` 的手算行为测试保护。

9.8 本章小结

泛型设计从可运行的普通函数开始。函数模板把类型变成编译期参数，实例化为实际类型生成函数；类模板保存具体策略。`concept` 用 `requires` 表达式把能力要求移到接口入口，却不证明业务语义。静态与动态多态服务于不同的类型确定时机，两者的性能和工程成本都必须结合真实系统选择。

第十章 错误处理与可观测性

内容提要

- ❑ 区分正常缺席、可恢复输入错误、外部失败与程序员错误
- ❑ 使用 optional、variant、异常与 assert 表达不同契约
- ❑ 通过候选状态和提交点提供强异常保证
- ❑ 在系统边界只记录一次错误，并保留可聚合指标

注 先修知识：已完成第 4、5、8 章，会读取 CSV、使用 optional、构造有效领域对象并观察组合快照。

Python 迁移边界：Python 的 None、返回错误对象、raise 和 assert 分别有相近角色；C++ 还要求调用方从函数类型看见一部分失败契约，并明确对象在异常后的状态。

常见错误与诊断：先问失败属于正常分支、输入问题、环境问题还是不可能状态；再检查错误是否保留行号与字段、是否被重复记录、对象状态是否半更新。

本章把第 8 章的成交与组合边界扩展为一个可恢复 CSV 管道。三行数据中有两行合法、一行数量字段损坏；程序继续处理后续合法行，并精确输出：

```
1  accepted=2 rejected=1 committed=2 rolled_back=0 quantity=25 cash=7515.00 equity=9990.00
```

标准错误只出现一条:

```
1 error line=3 field=quantity message=expected a positive integer
```

完成判据还包括：缺文件以状态 2 退出并只记录一次；混合代码批次失败后组合快照逐字段不变；错误注入练习必须用行为断言证明回滚，而不是只写“应该安全”。

10.1 先给失败分类，再选择机制

10.1.1 场景：四种“不成功”并不是同一件事

找不到 **MSFT** 最新价可能只是当前数据集中没有该代码；坏 CSV 行是调用方需要检查的可恢复结果；文件系统拒绝打开属于环境失败；负持仓则违反组合内部不变量。若全部返回 `bool`，上下文会丢失；若全部抛异常，正常缺席也会变成昂贵而喧闹的控制流。

Listing 10.1: 同一程序中的四种错误机制

[illegible]

```

14  assert(!symbol.empty());
15  for (auto position = fills.rbegin(); position != fills.rend(); ++position) {
16      if (position->symbol() == symbol) {
17          return position->price();
18      }
19  }
20  return std::nullopt;
21 }
22
23 void require_readable_file(const std::string& path) {
24     std::ifstream input{path};
25     if (!input) {
26         throw std::runtime_error{"cannot open input file"};
27     }
28 }
29
30 std::string optional_label(const std::optional<double>& value) {
31     if (value.has_value()) {
32         return "value";
33     }
34     return "none";
35 }
36
37 std::string exception_label(bool caught) {
38     if (caught) {
39         return "caught";
40     }
41     return "missed";
42 }
43
44 int main() {
45     const std::vector<quant::ch10::Fill> fills{
46         {"AAPL", quant::ch10::Side::buy, 10, 100.0}};
47     const auto missing = latest_price(fills, "MSFT");
48     const auto malformed =
49         quant::ch10::parse_fill_row("AAPL,buy,oops,100.0", 7);
50     std::string error_field{"missing"};
51     const auto* parse_error =
52         std::get_if<quant::ch10::ParseError>(&malformed);
53     if (parse_error != nullptr) {
54         error_field = parse_error->field;
55     }
56
57     bool external_caught = false;
58     try {
59         require_readable_file("definitely-missing.csv");
60     } catch (const std::runtime_error&) {
61         external_caught = true;
62     }

```

```

63
64     std::cout << "optional=" << optional_label(missing)
65               << " explicit=" << error_field
66               << " exception=" << exception_label(external_caught)
67               << " assertion=internal-only\n";
68 }

```

`std::optional<double>` 表示“有价格或正常缺席”，没有错误消息。解析结果把 `Fill` 与 `ParseError` 作为 `variant` 的两个备选类型，并用较短的 `FillParseResult` 作为别名。`variant` 在任一时刻只持有一个备选，返回类型明确要求调用方处理成功或错误。`holds_alternative<T>` 可以检查当前类型；`get_if<T>` 接收 `variant` 地址，匹配时返回指向该备选的指针，否则返回空指针。示例先检查指针再读取字段，不会因读取错误备选而抛 `bad_variant_access`。

文件打不开由 `runtime_error` 表达，因为更上层边界可能统一重试、换源或终止。异常从抛出点沿调用栈寻找匹配的 `catch`，途中已构造的局部对象会析构；`catch` 通常写 `const std::exception&`，既保留动态错误消息，也避免复制。

`assert(!symbol.empty())` 只声明开发者必须满足的内部前置条件。`Release` 构建定义 `NDEBUG` 时断言通常不执行，所以不能用它检查 CSV、网络消息、命令行参数或资金是否充足。外部数据无论构建类型都可能无效，必须走返回值或异常。

失败类别	首选机制	调用方责任
正常缺席	<code>optional</code>	分支处理无值
可恢复输入错误	<code>variant</code> 中的 <code>ParseError</code>	检查并保留上下文
无法就地处理的外部失败	<code>exception</code>	在有策略的边界捕获
程序员破坏内部不变量	<code>assert</code>	<code>Debug</code> 期尽早暴露

注Python 迁移提示： Python 的 `None` 接近 `optional`，`Union[Fill, ParseError]` 接近 `variant`，`raise` 接近异常。Python `assert` 同样可被优化选项移除，也不适合校验用户输入。区别在于 C++ `variant` 的备选集合进入静态类型，调用方不能把错误对象当 `Fill` 使用而不被编译器阻止。

失败诊断：如果 `optional` 无值却不知道原因，说明“正常缺席”可能被误用来承载错误；如果每一层 `catch` 后只写日志再重新抛出，同一故障会形成重复日志。先重新分类，而不是追加更多字符串。

立即练习：先把最新价查询的代码参数暂时改为空字符串。`Debug` 构建应在 `assert` 处终止；再配置独立 `Release` 构建：

```

1 cmake -S code/ch10 -B build/ch10-release -DCMAKE_BUILD_TYPE=Release
2 cmake --build build/ch10-release -j
3 ./build/ch10-release/ch10_error_choices

```

断言可能因 `NDEBUG` 被移除而继续运行。这正说明 `assert` 不能校验外部输入。恢复 `MSFT` 后，输出重新为 `optional=none`。

10.2 显式错误值保留输入上下文

10.2.1 场景：解析器知道字段，边界知道处置策略

解析器最接近原始文本，最清楚错误发生在哪一行、哪个字段；它却不知道业务应该跳过、停止还是隔离文件。因此 `ParseError` 保存事实，不在底层输出日志：

Listing 10.2: 成功成交或带上下文错误的显式结果

```

1  #ifndef QUANT_CH10_CSV_READER_HPP
2  #define QUANT_CH10_CSV_READER_HPP
3
4  #include <cstdint>
5  #include <string>
6  #include <variant>
7
8  #include "quant/ch10/domain.hpp"
9
10 namespace quant::ch10 {
11
12 struct ParseError final {
13     std::size_t line{};
14     std::string field;
15     std::string message;
16 };
17
18 using FillParseResult = std::variant<Fill, ParseError>;
19
20 FillParseResult parse_fill_row(const std::string& row, std::size_t line);
21
22 } // namespace quant::ch10
23
24 #endif

```

using FillParseResult = ... 创建类型别名，不生成新运行时对象。ParseError 拥有行号、字段和消息；调用方移动或保存它时不会依赖输入缓冲区的生命周期。实现位于 code/ch10/src/csv_reader.cpp，依次验证列数、side、quantity、price，再构造带不变量的 Fill。

测试从公开解析入口验证已知字面量：合法行必须保留 AAPL 与数量 10；损坏数量的错误行号必须为 3，字段必须为 quantity，并带固定消息。预期来自手写场景，不重新实现解析算法。

注Python 迁移提示： Python 常返回 (value, error) 或抛自定义异常。若元组允许两项同时为空或同时有值，就出现无效状态；variant 保证恰有一个备选。Rust 的 Result 更直接表达同一思想，C++20 可用 variant 建立轻量版本。

失败诊断：若错误只写 invalid input，运维无法定位文件；若把整行原文无条件写入日志，又可能泄露敏感字段。保留结构化的行号、字段名和稳定类别，原始值只在数据治理允许时记录。

立即练习：把坏行的 side 改为 hold。测试应得到 field=side 与 expected buy or sell；恢复后重新运行输入 seam。

10.3 强异常保证：失败前后公开状态相同

10.3.1 场景：批次第二笔失败，第一笔不能残留

异常安全讨论的是异常发生后对象仍承诺什么。基本保证要求对象可析构且不变量成立；强保证进一步要求操作像事务一样，要么全部成功，要么公开状态不变；不抛保证则承诺操作不会抛异常。不是每个函数都需要强保证，但组合批次非常适合：半批成交会产生难以解释的仓位。

Listing 10.3: 候选状态、验证与无异常提交点

```

1  #include "quant/ch10/portfolio.hpp"
2
3  #include <cassert>
4  #include <cmath>
5  #include <limits>
6  #include <stdexcept>
7  #include <string>
8  #include <utility>
9  #include <vector>
10
11 namespace quant::ch10 {
12
13 Portfolio::Portfolio(double initial_cash) : cash_(initial_cash) {
14     if (!std::isfinite(cash_) || cash_ < 0.0) {
15         throw std::invalid_argument{"initial cash must be finite and non-negative"};
16     }
17 }
18
19 void Portfolio::apply_batch(const std::vector<Fill>& fills) {
20     Portfolio candidate{*this};
21     for (const Fill& fill : fills) {
22         candidate.apply_one(fill);
23     }
24     swap(candidate);
25 }
26
27 PortfolioSnapshot Portfolio::snapshot(const std::string& symbol,
28                                     double mark_price) const {
29     detail::require_symbol(symbol);
30     detail::require_price(mark_price);
31     if (!symbol_.empty() && symbol_ != symbol) {
32         throw std::logic_error{"snapshot symbol does not match portfolio"};
33     }
34     assert(quantity_ >= 0);
35     return PortfolioSnapshot{
36         symbol, quantity_, cash_,
37         cash_ + static_cast<double>(quantity_) * mark_price};
38 }
39
40 void Portfolio::apply_one(const Fill& fill) {
41     const double notional = static_cast<double>(fill.quantity()) * fill.price();
42     if (!std::isfinite(notional)) {
43         throw std::overflow_error{"fill notional must be finite"};
44     }
45     if (!symbol_.empty() && symbol_ != fill.symbol()) {
46         throw std::logic_error{"portfolio supports one symbol per run"};
47     }
48     if (fill.side() == Side::sell && quantity_ < fill.quantity()) {
49         throw std::logic_error{"sell quantity exceeds position"};

```

```

50     }
51     if (fill.side() == Side::buy && cash_ < notional) {
52         throw std::logic_error{"buy notional exceeds cash"};
53     }
54     if (fill.side() == Side::buy &&
55         quantity_ > std::numeric_limits<std::int64_t>::max() - fill.quantity()) {
56         throw std::overflow_error{"buy quantity exceeds portfolio range"};
57     }
58
59     std::int64_t next_quantity{};
60     double next_cash{};
61     if (fill.side() == Side::buy) {
62         next_quantity = quantity_ + fill.quantity();
63         next_cash = cash_ - notional;
64     } else {
65         next_quantity = quantity_ - fill.quantity();
66         next_cash = cash_ + notional;
67     }
68     if (!std::isfinite(next_cash)) {
69         throw std::overflow_error{"portfolio cash must remain finite"};
70     }
71
72     if (symbol_.empty()) {
73         symbol_ = fill.symbol();
74     }
75     quantity_ = next_quantity;
76     cash_ = next_cash;
77     assert(quantity_ >= 0);
78     assert(std::isfinite(cash_) && cash_ >= 0.0);
79 }
80
81 void Portfolio::swap(Portfolio& other) noexcept {
82     using std::swap;
83     swap(cash_, other.cash_);
84     symbol_.swap(other.symbol_);
85     swap(quantity_, other.quantity_);
86 }
87
88 } // namespace quant::ch10

```

`apply_batch` 先复制当前 `Portfolio` 得到 `candidate`，逐笔只修改候选对象。任一 `apply_one` 抛出时，`candidate` 在栈展开中销毁，原对象从未改变。全部成功后，`swap` 交换数值和字符串状态；该成员标为 `noexcept`，构成清楚的提交点。

提交前还必须排除算术本身破坏不变量：买入数量先与固定宽度整数上限比较，名义金额和下一现金都必须保持 `finite`。测试先建立接近 `INT64_MAX` 的仓位，再注入一股买单；实现必须抛出 `overflow_error`，且异常前后的公开快照不变。不能依赖 `Debug assert` 在溢出后补救，因为有符号溢出本身是未定义行为。

这不是“catch 后手工撤销”。撤销代码也可能漏字段或再次失败；先构造完整新状态再提交，更容易审查。代价是复制时间与内存，因此大型订单簿可改用预验证、写前日志或专门事务结构，但必须由同一公开快照测

试保护语义。

`code/ch10/tests/test_atomic_portfolio.cpp` 先建立 10 股 AAPL，再注入“买 5 股 AAPL 后买 1 股 MSFT”的批次。测试捕获 `logic_error`，并逐字段比较异常前后 `quantity`、`cash`、`equity`。它不检查 `candidate` 或私有成员，所以实现可替换。

注Python 迁移提示：Python 也可先 `candidate = copy.deepcopy(state)`，验证后再替换引用；数据库事务则把提交点交给数据库。C++ 的差异在于析构与 `noexcept` 能把栈展开和提交边界写入类型与实现，资源对象会在异常路径自动清理。

失败诊断：看到异常后现金已扣、仓位未加，说明修改发生在所有可能失败检查之前。列出函数内每个可能抛出的操作，移动到候选状态阶段，最后只保留不会失败的提交动作。

立即练习：把失败批次第二笔改成合法 AAPL 买单。此时不应抛异常，快照应一次增加 6 股；随后恢复错配代码，确认强保证测试重新绿色。

10.4 日志、指标与追踪各回答什么

10.4.1 场景：一条坏行只产生一条可行动记录

日志解释一次具体事件，指标聚合大量事件的数量或分布，追踪把一次请求跨组件的阶段用关联标识连接起来。三者互补：日志不适合每秒扫描求错误率；指标不能告诉你第 3 行哪个字段坏了；没有 `trace id` 的跨服务日志难以还原一次请求路径。

Listing 10.4: 恢复边界统一记录错误并累计指标

```

1  #include "quant/ch10/pipeline.hpp"
2
3  #include <cstdint>
4  #include <exception>
5  #include <istream>
6  #include <ostream>
7  #include <stdexcept>
8  #include <string>
9  #include <variant>
10 #include <vector>
11
12 #include "quant/ch10/csv_reader.hpp"
13
14 namespace quant::ch10 {
15 namespace {
16
17 void remove_trailing_carriage_return(std::string& row) {
18     if (!row.empty() && row.back() == '\r') {
19         row.pop_back();
20     }
21 }
22
23 void record_error(std::ostream& log, const ParseError& error) {
24     log << "error line=" << error.line << " field=" << error.field
25         << " message=" << error.message << '\n';
26 }

```



```

27
28 void record_portfolio_error(std::ostream& log,
29                             ProcessingMetrics& metrics,
30                             std::size_t line,
31                             const std::exception& exception) {
32     record_error(log, ParseError{line, "portfolio", exception.what()});
33     ++metrics.rows_rejected;
34     ++metrics.batches_rolled_back;
35 }
36
37 } // namespace
38
39 ProcessingMetrics process_fill_csv(std::istream& input,
40                                   std::ostream& log,
41                                   Portfolio& portfolio) {
42     ProcessingMetrics metrics;
43     std::string row;
44     if (!std::getline(input, row)) {
45         throw std::runtime_error{"line 1: expected symbol,side,quantity,price"};
46     }
47     remove_trailing_carriage_return(row);
48     if (row != "symbol,side,quantity,price") {
49         throw std::runtime_error{"line 1: expected symbol,side,quantity,price"};
50     }
51
52     std::size_t line = 1;
53     while (std::getline(input, row)) {
54         ++line;
55         ++metrics.rows_seen;
56         remove_trailing_carriage_return(row);
57         const FillParseResult result = parse_fill_row(row, line);
58         if (std::holds_alternative<ParseError>(result)) {
59             record_error(log, std::get<ParseError>(result));
60             ++metrics.rows_rejected;
61             continue;
62         }
63
64         try {
65             portfolio.apply_batch({std::get<Fill>(result)});
66             ++metrics.rows_accepted;
67             ++metrics.batches_committed;
68         } catch (const std::logic_error& exception) {
69             record_portfolio_error(log, metrics, line, exception);
70         } catch (const std::overflow_error& exception) {
71             record_portfolio_error(log, metrics, line, exception);
72         }
73     }
74     if (input.bad()) {
75         throw std::runtime_error{"line " + std::to_string(line + 1) +

```

```

76         ": input read failure"};
77     }
78     return metrics;
79 }
80
81 } // namespace quant::ch10

```

解析器不记录，Portfolio 也不记录；process_fill_csv 是知道“跳过坏行并继续”策略的边界，所以只在这里调用 record_error。解析错误增加 rejected；组合异常还增加 rolled back。accepted 与 committed 分开命名，避免未来出现“解析成功但批次回滚”时含义混乱。

热路径每行都拼接字符串会分配内存并放大延迟。这里仅在错误分支格式化日志，成功路径只增加计数器。真实并发系统还要考虑原子计数、采样、异步队列容量和丢弃指标；不能为了观测而让日志阻塞交易线程，也不能静默丢弃而无计数。

异常只在有恢复或终止策略的边界捕获。code/ch10/src/main.cpp 负责文件打开和进程退出，因此缺文件在那里记录一次 fatal context=... 并返回 2；中间层不会 catch、记录、再抛出同一异常。

注Python 迁移提示： Python logging 的 extra 字段、Prometheus counter 和 OpenTelemetry span 分别对应结构化日志、指标与追踪。关键原则与语言无关：稳定字段便于聚合，关联 ID 贯穿边界，错误归属层只记录一次。

失败诊断：若同一个 trace id 出现三条完全相同堆栈，搜索 catch-and-throw；若错误率升高却没有日志，检查采样与队列丢弃指标；若日志量拖慢回测，测量成功路径是否仍格式化消息。

立即练习：在输入末尾加入一行 MSFT,buy,1,420.0。解析会成功，但组合边界应记录 field=portfolio, rejected 与 rolled back 各增加 1；AAPL 状态不变。

10.5 独立快照与验收命令

Listing 10.5: 进程边界捕获外部失败并输出稳定摘要

```

1  #include <exception>
2  #include <fstream>
3  #include <iomanip>
4  #include <iostream>
5  #include <stdexcept>
6  #include <string>
7
8  #include "quant/ch10/pipeline.hpp"
9
10 int run(const std::string& path) {
11     std::ifstream input{path};
12     if (!input) {
13         throw std::runtime_error{"cannot open input file"};
14     }
15
16     quant::ch10::Portfolio portfolio{10'000.0};
17     const auto metrics = quant::ch10::process_fill_csv(input, std::cerr, portfolio);
18     const auto snapshot = portfolio.snapshot("AAPL", 99.0);
19     std::cout << std::fixed << std::setprecision(2)
20               << "accepted=" << metrics.rows_accepted
21               << " rejected=" << metrics.rows_rejected

```

```

22         << " committed=" << metrics.batches_committed
23         << " rolled_back=" << metrics.batches_rolled_back
24         << " quantity=" << snapshot.quantity << " cash=" << snapshot.cash
25         << " equity=" << snapshot.equity << '\n';
26     return 0;
27 }
28
29 int main(int argc, char* argv[]) {
30     if (argc != 2) {
31         std::cerr << "fatal context=cli message=expected one CSV path\n";
32         return 2;
33     }
34     try {
35         return run(argv[1]);
36     } catch (const std::exception& exception) {
37         std::cerr << "fatal context=" << argv[1] << " message=" << exception.what()
38             << '\n';
39         return 2;
40     }
41 }

```

在 WSL 的项目根目录执行：

```

1 cmake -S code/ch10 -B build/ch10
2 cmake --build build/ch10 -j
3 ctest --test-dir build/ch10 --output-on-failure
4 ./build/ch10/ch10_recovery_pipeline code/ch10/data/fills.csv
5 ./build/ch10/ch10_recovery_pipeline missing.csv

```

第一条运行命令的标准输出与标准错误必须和章首逐字一致；第二条应只在标准错误打印：

```

1 fatal context=missing.csv message=cannot open input file

```

并以状态 2 退出。状态码让 shell、调度器和 Python subprocess 在不解析自然语言时也能判断失败。

10.6 自测、编码任务与面试检查点

1. optional 无值与 ParseError 的语义边界是什么？
2. variant 如何避免“value 与 error 同时存在”的无效状态？
3. 为什么 assert 不能校验 CSV 或命令行参数？Release 构建有何变化？
4. 栈展开期间局部 RAII 对象会发生什么？为什么按 const 引用捕获？
5. 基本保证、强保证与不抛保证分别承诺什么？
6. candidate 加 noexcept swap 如何形成提交点？代价是什么？
7. 日志、指标和追踪分别回答哪类问题？
8. 为什么错误应在有处置策略的边界记录一次？

编码任务：向一个已有 AAPL 持仓注入“两笔组成的失败批次”，第一笔合法，第二笔超卖；捕获异常后必须用公开快照断言 quantity、cash、equity 全部不变，并精确输出 injected=1 rolled_back=1 unchanged=1。完整答案位于 code/ch10/exercises/error_injection_solution.cpp。

问题 10.1 面试检查点 回答“交易系统怎么做错误处理”时，不要只说 `try/catch`。先分类正常缺席、可恢复错误、外部失败和内部 `bug`；再说明错误上下文、异常安全级别与提交点；最后给出日志只记一次、指标聚合、追踪关联和热路径成本边界。

10.7 本章小结

错误机制表达不同契约：`optional` 是正常缺席，`variant` 是显式可恢复结果，异常把无法就地处理的失败传播到策略边界，`assert` 暴露程序员破坏的不变量。强异常保证通过候选状态与无异常提交点保护组合；日志、指标和追踪在合适边界提供一次、可聚合且成本受控的证据。

第三部分

可靠工程

第十一章 CMake 与依赖

内容提要

- ❑ 区分配置、生成、构建与测试，读懂源目录和构建目录
- ❑ 准确选择 PRIVATE、PUBLIC 与 INTERFACE
- ❑ 用库、应用、示例和测试组成 target-based CMake 构建图
- ❑ 从干净目录复现 Debug、Sanitized 与 Release 配置

注 先修知识：已完成第 1、3、5 章，能编译多文件程序并理解头文件与链接。

Python 迁移边界：Python 的 `pyproject.toml` 主要描述包和工具；CMake 描述原生目标、编译要求与目标之间的依赖边。

常见错误与诊断：先确认失败发生在配置、生成、编译、链接、测试哪一步，再检查目标图；不要用全局 `include` 路径掩盖缺失依赖。

11.1 本章输出：从源码树复现一个目标图

量化程序从一个可执行文件增长为公共计算库、命令行应用、研究示例和行为测试后，继续复制编译选项与源文件列表会产生漂移。本章把一个 VWAP 汇总器组织成六个正常目标：`ch11_project_options` 保存共同构建要求，`ch11_market` 提供静态库，报告程序、示例、测试和练习答案只声明自己对该库的依赖。另有一个 `EXCLUDE_FROM_ALL` 故障目标，仅供显式触发链接实验，不属于正常构建。

从项目根目录执行以下命令；`build/ch11-debug` 必须在开始前不存在：

```
cmake -S code/ch11 -B build/ch11-debug -DCMAKE_BUILD_TYPE=Debug
cmake --build build/ch11-debug -j
ctest --test-dir build/ch11-debug --output-on-failure
./build/ch11-debug/ch11_market_report
```

最后一条命令精确输出：

```
rows=3 quantity=25 notional=2525.00 vwap=101.00
```

完成判据不只是“我的机器能编译”：全新构建树能配置并构建全部目标；CTest 运行库的公开行为；应用无需自己声明头文件目录或 C++ 标准；移除链接边会稳定停在链接阶段；Debug、Sanitized 和 Release 使用彼此独立的构建树。

11.2 配置不是编译：先读四个阶段

11.2.1 场景：为什么旧可执行文件会制造假绿

`cmake -S ... -B ...` 先读取 `CMakeLists.txt`、检测编译器和依赖，再生成 Ninja 或 Make 等底层构建文件。它通常不编译 C++。`cmake --build ...` 才按依赖图调用编译器与链接器；`ctest --test-dir ...` 运行已经登记且已经构建的测试。若只运行最后一步，旧二进制可能让错误源码看起来仍然通过。

`CMakeCache.txt` 保存编译器路径、选项和检测结果，所以“删除一个目标文件”不等于干净配置。验证可复现性时应换用从未存在的构建目录；切换编译器、生成器或 sanitizer 时也优先使用新目录，而不是反复修改同一个缓存。

注Python 迁移提示：Python 直接执行源码时，安装与运行边界常被解释器隐藏；原生项目把配置、生成、编译、链接和测试明确分开。CMake 类似“生成构建系统的项目模型”，不是编译器，也不是等价于 pip 的包管理器。先判断命令停在哪个阶段，才能决定检查 CMake 目标、C++ 源码、链接库还是运行行为。

立即练习：只运行配置命令，确认构建目录中已有缓存和生成文件但还没有 `ch11_market_report`；再构建该单一目标，观察静态库先于应用生成；最后运行 CTest，解释为何未构建测试目标时测试可能显示“找不到可执行文件”。

11.3 用目标表达库、应用、测试与示例

先观察重构顺序。第一步若只有报告程序，常见写法是把应用和实现源文件都塞给一个 `executable`：

```
add_executable(ch11_market_report
    app/main.cpp src/market_summary.cpp)
target_include_directories(ch11_market_report PRIVATE include)
```

它能构建，却没有可供示例和测试复用的库。第二步抽出 `ch11_market`，把实现源文件和公共 `include` 要求移到库目标；报告程序删除重复源文件，只保留一条链接边：

```
add_library(ch11_market STATIC src/market_summary.cpp)
target_include_directories(ch11_market PUBLIC include)
add_executable(ch11_market_report app/main.cpp)
target_link_libraries(ch11_market_report PRIVATE ch11_market)
```

第三步按相同形状增加 `example` 与 `test`：各自拥有入口源码，各自 `PRIVATE` 链接库；测试再用 `add_test` 登记。最后才把多目标共同的 C++20、警告和 sanitizer 要求收进 `INTERFACE options` 目标。每一步都删除旧目标上的重复属性，并立即从空构建目录验证，避免最终清单看似整齐却无法解释依赖从何而来。

下面的完整清单就是独立章节快照的构建入口：

Listing 11.1: 第 11 章完整 target-based CMake 项目

```
1 cmake_minimum_required(VERSION 3.20)
2 project(ch11_cmake_targets LANGUAGES CXX)
3
4 if(CMAKE_SOURCE_DIR STREQUAL PROJECT_SOURCE_DIR AND
5     NOT CMAKE_BUILD_TYPE AND NOT CMAKE_CONFIGURATION_TYPES)
6     set(CMAKE_BUILD_TYPE Debug CACHE STRING "Build type" FORCE)
7 endif()
8
9 option(CH11_ENABLE_SANITIZERS "Enable AddressSanitizer and UBSan" OFF)
10
11 include(CTest)
12
13 add_library(ch11_project_options INTERFACE)
14 target_compile_features(ch11_project_options INTERFACE cxx_std_20)
15 if(MSVC)
16     target_compile_options(ch11_project_options INTERFACE /W4 /permissive-)
17 else()
18     target_compile_options(ch11_project_options INTERFACE
19         -Wall -Wextra -Wpedantic)
20     if(CH11_ENABLE_SANITIZERS)
```



```

21     target_compile_options(ch11_project_options INTERFACE
22         -fsanitize=address,undefined -fno-omit-frame-pointer)
23     target_link_options(ch11_project_options INTERFACE
24         -fsanitize=address,undefined)
25     endif()
26 endif()
27
28 add_library(ch11_market STATIC src/market_summary.cpp)
29 target_include_directories(ch11_market PUBLIC include)
30 target_link_libraries(ch11_market PUBLIC ch11_project_options)
31 target_compile_definitions(ch11_market PRIVATE CH11_BUILDING_LIBRARY=1)
32
33 add_executable(ch11_market_report app/main.cpp)
34 target_link_libraries(ch11_market_report PRIVATE ch11_market)
35
36 add_executable(ch11_market_example examples/market_example.cpp)
37 target_link_libraries(ch11_market_example PRIVATE ch11_market)
38
39 add_executable(ch11_market_tests tests/test_market_summary.cpp)
40 target_link_libraries(ch11_market_tests PRIVATE ch11_market)
41 add_test(NAME ch11_market_summary_behavior COMMAND ch11_market_tests)
42
43 add_executable(ch11_notional_solution exercises/notional_report_solution.cpp)
44 target_link_libraries(ch11_notional_solution PRIVATE ch11_market)
45
46 add_executable(ch11_missing_target_link_failure EXCLUDE_FROM_ALL
47     failures/missing_target_link.cpp)
48 target_link_libraries(ch11_missing_target_link_failure PRIVATE
49     ch11_project_options)
50
51 add_test(NAME ch11_market_report_runs COMMAND ch11_market_report)
52 set_tests_properties(ch11_market_report_runs PROPERTIES
53     PASS_REGULAR_EXPRESSION
54     "rows=3 quantity=25 notional=2525.00 vwap=101.00")
55 add_test(NAME ch11_market_example_runs COMMAND ch11_market_example)
56 set_tests_properties(ch11_market_example_runs PROPERTIES
57     PASS_REGULAR_EXPRESSION "example-quantity=4 example-vwap=100")
58 add_test(NAME ch11_notional_solution_runs COMMAND ch11_notional_solution)
59 set_tests_properties(ch11_notional_solution_runs PROPERTIES
60     PASS_REGULAR_EXPRESSION "exercise-notional=200.00")

```

`add_library` 与 `add_executable` 创建有名称的目标。源文件、头文件目录、语言标准、编译定义和链接项随后附着到具体目标，不写成影响整个目录的全局状态。应用、示例、测试都只通过 `target_link_libraries(... ch11_market)` 建立一条边；它们不复制 `src/market_summary.cpp`，因此库的实现只有一个编译实体。

`include(CTest)` 提供 `BUILD_TESTING` 约定并启用测试登记；`add_test` 描述运行期行为，但不会自动创建或构建可执行文件。测试本身仍是普通目标，区别只在于它被 `CTest` 调度。

公共头文件展示了库消费者真正需要的 API：

Listing 11.2: 由库目标公开的行情汇总接口

```

1  #ifndef QUANT_CH11_MARKET_SUMMARY_HPP
2  #define QUANT_CH11_MARKET_SUMMARY_HPP
3
4  #include <cstddef>
5  #include <cstdint>
6  #include <span>
7
8  namespace quant::ch11 {
9
10 struct MarketRow {
11     double price;
12     std::int64_t quantity;
13 };
14
15 struct MarketSummary {
16     std::size_t rows;
17     std::int64_t quantity;
18     double notional;
19     double vwap;
20 };
21
22 [[nodiscard]] MarketSummary summarize(std::span<const MarketRow> rows);
23
24 } // namespace quant::ch11
25
26 #endif

```

该接口使用 `std::span`，所以消费者也必须以 C++20 编译。正确做法不是要求每个应用记住 `-std=c++20`，而是让库目标传播这一使用要求。

11.4 PRIVATE、PUBLIC 与 INTERFACE 是传播方向

三个关键字必须针对“某一项目标属性”理解，而不是给整个库贴标签：

关键字	当前目标使用	链接当前目标的消费者使用
PRIVATE	是	否
PUBLIC	是	是
INTERFACE	否	是

`include/` 中的头文件是公共接口，所以 `target_include_directories(ch11_market PUBLIC include)` 既让库实现找到头文件，也让应用找到它。`ch11_project_options` 没有源码、不会生成库文件；它是 **INTERFACE** 目标，只携带 C++20、警告和可选 sanitizer 要求。市场库以 **PUBLIC** 链接它，应用因而沿目标边获得这些要求。

相反，`CH11_BUILDING_LIBRARY` 只用于编译库实现，故声明为 **PRIVATE**。应用源码包含一个反向检查：

Listing 11.3: 消费者使用公共接口且拒绝私有定义泄漏

```

1  #include <iomanip>
2  #include <iostream>

```

```

3  #include <vector>
4
5  #include "quant/ch11/market_summary.hpp"
6
7  #ifndef CH11_BUILDING_LIBRARY
8  #error "a PRIVATE compile definition leaked into the application target"
9  #endif
10
11 int main() {
12     const std::vector<quant::ch11::MarketRow> rows{
13         {100.0, 10}, {101.0, 5}, {102.0, 10}};
14     const auto summary = quant::ch11::summarize(rows);
15
16     std::cout << std::fixed << std::setprecision(2)
17         << "rows=" << summary.rows
18         << " quantity=" << summary.quantity
19         << " notional=" << summary.notional
20         << " vwap=" << summary.vwap << '\n';
21 }

```

若把该编译定义误改为 PUBLIC，应用中的 #error 会在编译阶段触发。这是传播行为的直接证据，不需要读取 CMake 内部属性。真实可安装库还会用 BUILD_INTERFACE 和 INSTALL_INTERFACE 区分源码树与安装后的头文件位置；本章快照只在源码树构建，暂不引入安装规则。

失败诊断：将报告目标的 `target_link_libraries` 行删除。头文件可能先因 `include` 路径不再传播而编译失败；若手工加全局 `include` 路径，编译可能通过，最终仍因 `summarize` 没有链接定义而失败。这说明“加一个 -I”不能修复缺失的目标边。

仓库还隔离了一个最小链接实验：

Listing 11.4: 声明存在但没有目标提供定义的故障样例

```

1  #include <cstdint>
2
3  std::int64_t load_market_quantity();
4
5  int main() {
6      return load_market_quantity() == 25 ? 0 : 1;
7  }

```

```

g++ -std=c++20 code/ch11/failures/missing_target_link.cpp \
-o build/missing-target-link

```

源文件能通过编译，但链接器报告 `undefined reference`；恢复方法是链接提供定义的目标，而不是在声明后再写一份临时实现。

立即练习：把 `ch11_market` 对 `ch11_project_options` 的 PUBLIC 暂改为 PRIVATE，在全新构建树中观察消费者能否继续使用 `std::span`；恢复后再把私有编译定义改为 PUBLIC，确认失败位置和第一条稳定诊断不同。

11.5 Debug、Sanitized 与 Release 各自回答什么

单配置生成器通过 `CMAKE_BUILD_TYPE` 选择一组配置。Debug 保留调试信息并适合逐步定位；Sanitized 在 Debug 基础上增加 AddressSanitizer 与 UndefinedBehaviorSanitizer，用运行期开销换取内存和 UB 证据；Release 启用优化并常定义 `NDEBUG`，适合最终行为与性能验收，不能代替前两者。

```
cmake -S code/ch11 -B build/ch11-debug -DCMAKE_BUILD_TYPE=Debug
cmake --build build/ch11-debug -j
ctest --test-dir build/ch11-debug --output-on-failure

cmake -S code/ch11 -B build/ch11-sanitized \
  -DCMAKE_BUILD_TYPE=Debug -DCH11_ENABLE_SANITIZERS=ON
cmake --build build/ch11-sanitized -j
ctest --test-dir build/ch11-sanitized --output-on-failure

cmake -S code/ch11 -B build/ch11-release -DCMAKE_BUILD_TYPE=Release
cmake --build build/ch11-release -j
ctest --test-dir build/ch11-release --output-on-failure
```

三个目录并存，使配置差异可审计，也避免缓存串味。Sanitizer 编译选项与链接选项都挂在 `INTERFACE` 目标上：只加编译选项可能在最终链接 sanitizer runtime 时失败。MSVC 分支使用 `/W4 /permissive-`；本章的 sanitizer 开关只对 GCC/Clang 主线生效。

多配置生成器（如 Visual Studio）通常在一次配置中生成 Debug 与 Release，`CMAKE_BUILD_TYPE` 会被忽略；构建和测试时写 `cmake --build build --config Release` 与 `ctest --test-dir build -C Release`。MinGW Makefiles 通常仍是单配置生成器，沿用 `CMAKE_BUILD_TYPE` 和 GCC 风格选项，可执行文件带 `.exe`；它不能与 MSVC 编译出的不兼容 ABI 库随意混用。平台改变了选择配置的命令位置和产物后缀，没有改变目标、属性和依赖边的模型。

11.6 外部依赖也应表现为目标

现代包提供的 CMake 配置通常暴露命名空间目标。以下只是语法示意，本章不下载 `fmt`：

```
find_package(fmt 10 CONFIG REQUIRED)
target_link_libraries(ch11_market_report PRIVATE fmt::fmt)
```

`fmt::fmt` 应携带自己的头文件目录、编译定义及传递链接项；消费者无需猜测安装路径。固定可接受的版本范围、记录工具链和依赖来源，才能使“我的缓存里能找到”变成可复现配置。若使用 `FetchContent`，还需固定提交或发布版本，并明确网络与离线策略；不要默认追踪上游主分支。

故障诊断按顺序进行：配置期找不到包，检查工具链文件、前缀路径和版本；编译期找不到头文件，检查是否链接了正确 imported target 及其可见性；链接期缺符号，检查库目标和 ABI/配置是否匹配；运行期找不到动态库，再检查部署路径。把所有路径塞进全局 `include_directories` 或 `link_directories` 会让无关目标偶然成功，并隐藏真实依赖。

立即练习：在纸上为“CSV 库、回测库、命令行应用、单元测试”画目标图。逐条标出公共头文件、仅实现使用的编译定义、消费者必须继承的 C++ 标准和外部库，随后为每条属性选择 `PRIVATE`、`PUBLIC` 或 `INTERFACE`；若某属性没有明确消费者，先不要传播。

11.7 独立快照、输出任务与验收

本章遵循 ADR 0001，把 `code/ch11` 保留为可独立配置的阶段快照。整书根项目通过 `add_subdirectory` 复用同一目标声明；自动验收还会删除旧测试构建树，从空目录分别构建普通与 `Sanitized` 配置下的全部正常目标、运行该快照自己的 `CTest`，再核对报告程序输出。这样 PDF 清单、读者命令与 CI 使用同一份 `CMake` 事实源。

编码任务：新增一个 `ch11_notional_solution` 可执行目标。源码只包含公共头文件并链接 `ch11_market`，不得复制库源文件或手写 `include` 路径。用两行行情 `{99.0,1}` 与 `{101.0,1}` 精确输出：

```
exercise-notional=200.00
```

从全新 `Debug` 构建树构建该单一目标并运行，再构建全项目和运行 `CTest`。完整源码见附录；目标声明应只需要 `add_executable` 与一条 `PRIVATE` 链接边。

自测：

1. 配置、生成、构建和测试分别读取什么、产生什么，哪一步调用编译器？
2. 为什么存在旧可执行文件不能证明当前源码可从零构建？
3. 库的公共头文件目录为何是 `PUBLIC`，而库实现宏为何是 `PRIVATE`？
4. `INTERFACE` 目标既不编译源码，为何仍能影响应用的编译和链接？
5. `add_test` 为什么不能代替 `add_executable`？
6. 缺 `include` 路径与缺函数定义分别最可能在哪个阶段暴露？
7. `Debug`、`Sanitized` 和 `Release` 各自提供什么证据，为什么使用独立构建树？
8. 单配置与多配置生成器选择 `Release` 的命令差异是什么，哪些目标模型不变？

问题 11.1 面试检查点 解释 `PUBLIC`、`PRIVATE`、`INTERFACE` 时，先说它们控制某项使用要求是否作用于当前目标及消费者，再给出本章头文件目录、C++20 要求和私有宏三个例子。若被问“怎样让 `CMake` 项目可复现”，回答干净构建树、明确编译器/生成器/配置、目标化依赖、固定外部版本和可运行验收，而不是只说“提交 `CMakeLists.txt`”。

11.8 本章小结

`CMake` 的核心产物是目标图。目标拥有源码和构建属性，依赖边传播真实使用要求；`PRIVATE`、`PUBLIC`、`INTERFACE` 描述传播方向。配置、构建和测试是不同阶段，旧缓存不能替代干净复现。`Debug`、`Sanitized` 与 `Release` 回答不同问题，外部依赖也应以版本化目标进入图。只要应用、测试和示例都通过链接库获得所需信息，项目结构就能随量化代码增长而保持可解释、可验证。

第十二章 测试与调试工具

内容提要

- ❑ 把需求写成不依赖实现细节的行为测试，并完成一次红—绿循环
- ❑ 在 Debug 构建中用 GDB 从失败输出追到错误表达式
- ❑ 用编译器警告、静态分析、ASan 与 UBSan 各自回答合适的问题
- ❑ 识别 Release 下消失的 assert，让测试始终执行真实检查

注 先修知识：已完成第 6、10、11 章，理解生命周期、错误分类、CMake 目标与构建配置。

Python 迁移边界：pytest 异常通常仍由解释器管理；C++ 还可能在没有语言级异常的情况下发生未定义行为，所以需要编译器、调试器和 sanitizer 共同取证。

常见错误与诊断：先保存最小复现、退出码和行为差异，再按症状选择工具；工具报告的是证据，不自动证明业务根因。

12.1 本章输出：一份可复核的故障诊断报告

量化系统最危险的失败不一定崩溃。手续费符号写反仍会生成看似合理的净值；悬空指针可能在测试机上“碰巧可用”；只含 assert 的测试会在 Release 中安静通过。本章交付的不是一串调试命令，而是一份包含六项证据的诊断报告：症状与期望、最小复现命令、失败层级、工具输出中的稳定类别、根因、修复后的回归测试。

仓库已有四条公开行为 seam，可作为测试先例：project/tests/test_csv_reader.cpp 保护行情输入，project/tests/test_strategy.cpp 保护策略决策，project/tests/test_execution_portfolio.cpp 保护成交与组合，project/tests/test_backtest.cpp 保护端到端手算账本。它们只观察输入、返回值、状态和输出，不断言某个私有成员或某次内部调用，因此重构实现时仍有价值。

本章新增一个更小的成交/组合 seam。初始现金为 1000，以 100 买入 2 份并支付 1 元手续费，标记价格为 110；手算得到现金 799、权益 1019。完整测试如下：

Listing 12.1: Release 中仍执行真实分支检查的组合行为测试

```
1  #include <cmath>
2  #include <iomanip>
3  #include <iostream>
4
5  #include "quant/ch12/portfolio.hpp"
6
7  int main() {
8      quant::ch12::Portfolio portfolio{1'000.0};
9      portfolio.buy(100.0, 2, 1.0);
10     const auto snapshot = portfolio.snapshot(110.0);
11
12     const bool correct = snapshot.quantity == 2 &&
13                          std::abs(snapshot.cash - 799.0) < 1e-12 &&
14                          std::abs(snapshot.equity - 1'019.0) < 1e-12;
15     if (!correct) {
16         std::cerr << "portfolio fee behavior mismatch\n";
17         return 1;
18     }
```

```

19
20     std::cout << std::fixed << std::setprecision(2)
21         << "portfolio-fee-ok cash=" << snapshot.cash
22         << " equity=" << snapshot.equity << '\n';
23 }

```

测试使用 `if (!correct)` 返回非零状态。它不是为了绕开测试框架，而是让最小快照不引入外部依赖，同时展示测试程序最基本的协议：零表示通过，非零表示失败，标准错误流解释差异。完整快照可独立验证：

```

cmake -S code/ch12 -B build/ch12-debug -DCMAKE_BUILD_TYPE=Debug
cmake --build build/ch12-debug -j
ctest --test-dir build/ch12-debug --output-on-failure
./build/ch12-debug/ch12_portfolio_tests

```

预期输出严格为：

```
portfolio-fee-ok cash=799.00 equity=1019.00
```

12.2 从行为开始：一次红—绿—整理循环

12.2.1 先写能失败的账本事实

测试驱动开发的第一步不是提前设计所有类，而是选择一个可观察事实。若实现把手续费加回现金，错误程序会报告 `observed-cash=801 expected-cash=799` 并以状态 2 退出：

Listing 12.2: 红灯：手续费符号写反的最小逻辑错误

```

1  #include <iomanip>
2  #include <iostream>
3
4  double cash_after_buy(double cash, double price, int quantity, double fee) {
5      const double notional = price * static_cast<double>(quantity);
6      return cash - notional + fee;
7  }
8
9  int main() {
10     const double observed = cash_after_buy(1'000.0, 100.0, 2, 1.0);
11     std::cerr << std::fixed << std::setprecision(0)
12         << "observed-cash=" << observed << " expected-cash=799\n";
13     return observed == 799.0 ? 0 : 2;
14 }

```

这条红灯有三个质量条件。第一，它因目标行为尚未满足而失败，不是因源文件不存在或测试本身无法编译。第二，期望值来自独立手算 $1000 - 100 \times 2 - 1 = 799$ ，不是复制实现表达式。第三，输出同时给出 `observed` 与 `expected`，失败后无需先打开测试源码猜测。

最小修复只把 `+ fee` 改为 `- fee`，并保留同一个行为检查：

Listing 12.3: 绿灯：修复公式并保留回归检查

```

1  #include <iomanip>
2  #include <iostream>

```



```

3
4 double cash_after_buy(double cash, double price, int quantity, double fee) {
5     const double notional = price * static_cast<double>(quantity);
6     return cash - notional - fee;
7 }
8
9 int main() {
10     const double observed = cash_after_buy(1'000.0, 100.0, 2, 1.0);
11     if (observed != 799.0) {
12         std::cerr << "cash update is still wrong\n";
13         return 1;
14     }
15     std::cout << std::fixed << std::setprecision(0)
16               << "fixed-cash=" << observed << '\n';
17 }

```

绿色之后才整理重复代码：真实快照把组合状态与输入检查封装进 Portfolio，测试只调用公开的 buy 与 snapshot。整理不能改变 799 与 1019 这两个外部事实。若重构后测试先红，先恢复行为再继续整理，不要同时扩大改动面。

注Python迁移提示：pytest 的 `assert observed == expected` 会由 pytest 改写并生成丰富差异；C++ 标准 `assert` 是可被 NDEBUD 删除的调试宏。GoogleTest、Catch2 等框架提供不会随 Release 消失的检查，但本书最小快照用显式分支展示底层协议。两种语言相同的原则是：先断言公开行为，再重构实现；不要对私有调用顺序写脆弱测试。

立即练习：把正确实现中的费用暂改为 0，先写下你预测的现金和权益，再运行测试。确认失败来自数值差异后恢复代码；若你先改期望值让测试变绿，就失去了手续费需求的独立 oracle。

12.3 Release 陷阱: assert 不是测试框架

`assert(condition)` 适合表达程序员认为不可能被合法输入破坏的内部不变量。条件为假时，Debug 程序终止并报告位置；定义 NDEBUD 后，整个表达式通常不再求值。下面的故障测试故意让实际成交数为 1、期望为 2，却只用 `assert` 检查：

Listing 12.4: 只在 Debug 有效的断言陷阱

```

1 #include <cassert>
2 #include <iostream>
3
4 int main() {
5     const int actual_fills = 1;
6     const int expected_fills = 2;
7     assert(actual_fills == expected_fills);
8
9     #ifdef NDEBUD
10         std::cout << "assertion-check-disappeared\n";
11     #endif
12 }

```

分别显式构建这个隔离目标：

```
cmake -S code/ch12 -B build/ch12-debug -DCMAKE_BUILD_TYPE=Debug
```

```
cmake --build build/ch12-debug --target ch12_assert_only_release
./build/ch12-debug/ch12_assert_only_release

cmake -S code/ch12 -B build/ch12-release -DCMAKE_BUILD_TYPE=Release
cmake --build build/ch12-release --target ch12_assert_only_release
./build/ch12-release/ch12_assert_only_release
```

Debug 运行会被断言终止；Release 却输出 `assertion-check-disappeared` 并返回成功。更隐蔽的错误是把有副作用的操作写进 `assert(place_order())`：Release 中连下单调用也会消失。因此断言表达式应无副作用，测试判定必须来自测试框架或显式检查。

本章自动验收还会从全新 Release 构建树运行 `ch12_portfolio_tests`。其错误分支没有被预处理器删除，错误费用仍会令进程返回 1。这才是“Release 测试通过”的真实含义，而不是“测试可执行文件返回了 0”。

12.4 逻辑错误：用 GDB 检查数据流

逻辑错误通常不会触发 sanitizer，因为每次内存访问都可能合法，类型转换也符合语言规则。正确顺序是先行为测试稳定复现，再在带调试信息的低优化构建中检查数据流：

```
g++ -std=c++20 -O0 -g code/ch12/failures/logic_bug.cpp \
    -o build/ch12-logic-bug
gdb -q build/ch12-logic-bug
(gdb) break cash_after_buy
(gdb) run
(gdb) info args
(gdb) next
(gdb) info locals
(gdb) finish
```

本机 WSL 的实际证据可压缩成三行：

```
cash=1000 price=100 quantity=2 fee=1
notional=200
Value returned is 801
```

输入和中间乘积正确，差异首次出现在返回表达式，根因范围便从“组合系统不对”缩小到手续费符号。

常用 GDB 动作可按问题组织：`break` 在怀疑边界停住，`run` 复现，`next` 越过函数调用，`step` 进入函数，`print expr` 检查表达式，`bt` 查看调用栈，`frame n` 切换栈帧，`watch expr` 在值改变时暂停。不要从入口逐行走完整程序；先让失败测试把断点推到最小边界。

优化会内联函数、删除变量或重排指令，GDB 可能显示 `optimized out`。这不说明调试器坏了。先在 `-O0 -g` 重现正确性问题；若故障只在 Release 出现，再保留 `-g` 并逐级增加优化，比较行为与生成配置。禁止通过“Debug 没问题”推断 Release 正确。

Windows 上 Visual Studio 调试器可完成断点、局部变量和调用栈检查；MinGW 的 GDB 命令与 WSL 主线接近。界面不同，但证据链仍是失败行为、可疑边界、输入、中间状态和首次错误值。

12.5 编译器警告与静态分析：不运行也能发现什么

警告分析单个翻译单元和部分跨语句事实。GCC/Clang 主线建议从以下集合开始：

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -c source.cpp
```

`-Wall` 并不等于“所有警告”，`-Wextra` 补充常见可疑构造，`-Wpedantic` 提醒非标准扩展。受控 CI 可以在清洁后使用 `-Werror` 防止新增警告；直接把第三方头文件和多编译器差异全部升级为错误，容易让工具链升级变成无关阻塞。MSVC 对应起点是 `/W4 /permissive-`。

静态分析会沿可能的控制流追踪资源和状态。下面的最小程序语法正确，却在同一路径释放同一地址两次：

Listing 12.5: 静态分析目标：同一路径双重释放

```
1 int main() {
2     auto* price = new double{100.0};
3     delete price;
4     delete price;
5 }
```

本机 GCC 13 可直接运行：

```
g++ -std=c++20 -fanalyzer -c \
    code/ch12/failures/double_delete.cpp
```

稳定证据类别是 `double free/use after delete`；具体措辞和行号会随编译器版本变化。`clang-tidy` 能补充现代化、可读性和缺陷检查，但它不是本章复现的隐藏前置依赖：未安装时先用已有编译器警告和 GCC analyzer。静态分析会有误报，也看不到真实运行数据；先检查报告路径是否可达，再决定修复、抑制或调整规则，并在代码审查中记录理由。

立即练习：删除第一次 `delete` 后复跑 analyzer，确认类别消失；再把裸资源改为 `std::unique_ptr<int>`。解释为什么所有权类型既减少错误路径，也比局部抑制警告提供更强设计证据。

12.6 Sanitizer: 让未定义行为留下运行证据

C++ 的越界、释放后使用和有符号溢出属于未定义行为。程序可能崩溃、输出旧值、污染别处，也可能看似正常；“运行十次都没事”不是正确性证据。Sanitizer 在编译时插入检查并在运行时报告类别，适合用能覆盖缺陷路径的小输入触发。

12.6.1 ASan: 越界与悬空

第一个程序分配两个 `double`，却写入下标 2 指向的第三个位置：

Listing 12.6: ASan 目标：堆越界访问

```
1 #include <memory>
2
3 int main() {
4     auto prices = std::make_unique<double[]>(2);
5     prices[0] = 100.0;
6     prices[1] = 101.0;
7     prices[2] = 102.0;
8     return prices[0] > 0.0 ? 0 : 1;
9 }
```

```
g++ -std=c++20 -O0 -g -fsanitize=address \
    -fno-omit-frame-pointer code/ch12/failures/out_of_bounds.cpp \
    -o build/ch12-out-of-bounds
./build/ch12-out-of-bounds
```

验收类别是 `heap-buffer-overflow`。报告通常还给出访问大小、分配栈和越界地址相对缓冲区的位置。先读第一条错误及其用户代码栈帧，不要被后续影子字节表淹没。

第二个程序保存动态对象地址，释放后又解引用：

Listing 12.7: ASan 目标：释放后使用

```
1 int main() {
2     auto* price = new double{100.0};
3     delete price;
4     return *price > 0.0 ? 0 : 1;
5 }
```

这里的稳定类别是 `heap-use-after-free`。ASan 给出的读取位置、释放位置和分配位置共同构成生命周期证据。修复不是“把指针设为 `nullptr`”这么简单：若别处还保存副本，仍可能悬空；优先让对象按值存在、由容器拥有，或用 `unique_ptr` 表达唯一所有权。

12.6.2 UBSan：合法内存上的非法运算

有符号整数越过可表示上限不是保证回绕：

Listing 12.8: UBSan 目标：有符号整数溢出

```
1 #include <limits>
2
3 int main() {
4     volatile int quantity = std::numeric_limits<int>::max();
5     quantity += 1;
6     return quantity;
7 }
```

```
g++ -std=c++20 -O0 -g -fsanitize=undefined \
    -fno-omit-frame-pointer code/ch12/failures/signed_overflow.cpp \
    -o build/ch12-overflow
./build/ch12-overflow
```

预期类别是 `signed integer overflow`。量化代码中的数量乘价格、时间单位换算和累计计数都应在运算前检查范围；把溢出后的结果再转成更宽类型已经太迟。若确实需要模运算，使用无符号类型并在接口中明确语义，不要依赖有符号回绕。

ASan 和 UBSan 应在 `Debug/Sanitized` 配置中编译并链接相同选项；它们增加内存与时间开销，不用于性能结论。MSVC 近年支持 `AddressSanitizer`，但选项和覆盖范围与 GCC/Clang 不同；本章自动故障类别测试在不支持的编译器上显式跳过，而不是伪造通过。

注Python 迁移提示：Python 列表越界通常抛出 `IndexError`，整数自动扩展，普通对象由运行时维持生命周期；这会降低一类内存错误，却不会阻止 NumPy/C 扩展、ctypes 或原生库中的 UB。C++ sanitizer 更像“带观测器的特殊构建”，不是异常处理机制：发现错误后应修复所有权、边界或范围，并留下行为回归测试。

12.7 症状决定工具，工具不能代替根因

症状	首选证据	本章稳定类别或观察
业务数值错误	行为测试 + GDB	801 与手算 799 首次分歧
堆越界	ASan	heap-buffer-overflow
释放后使用	ASan	heap-use-after-free
有符号溢出	UBSan	signed integer overflow
资源路径可疑	静态分析	double free/use after delete
只在 Release 假绿	显式检查 + 配置对照	assert 被 NDEBUG 删除

编译错误先读第一条根因诊断，链接错误检查目标边，确定性崩溃用 sanitizer 和调用栈，数据竞争留给第 15 章的 ThreadSanitizer，性能慢留给第 14 章的 benchmark/profiler。随意把所有工具同时打开会增加噪声，也可能让 sanitizer 开销改变时序；一次实验应写清它回答的单一问题。

实际故障循环可固定为六步：保存失败输入与命令；用最小公开行为稳定复现；按错误阶段和症状选择工具；记录稳定类别及首个用户栈帧；提出能解释全部证据的根因；做最小修复并增加回归测试。若修复只让症状消失却不能解释证据，诊断仍未完成。

12.8 输出任务：诊断而不是照抄命令

从 code/ch12/failures/logic_bug.cpp 开始，不先查看练习答案。提交一份短报告并满足以下验收：

1. 写出独立手算的预期现金以及程序实际输出、退出码。
2. 用 Debug 信息编译，在 cash_after_buy 入口停住，记录四个参数、notional 和返回值。
3. 解释为什么 ASan/UBSan 不应报告这个逻辑错误，为什么行为 oracle 会报告。
4. 指出首次产生差异的表达式，只修改根因，不修改期望值。
5. 增加不依赖 assert 的回归检查，在 Debug 与 Release 都运行。
6. 最终程序精确输出 fixed-cash=799，报告同时保留复现命令与限制。

完整修复见附录：先提交证据再对照答案。若报告只有“运行 GDB、发现 bug、已修复”，不算完成，因为其他人无法复核你观察过什么，也无法判断修复是否碰巧掩盖症状。

自测：

1. 行为测试为何应断言公开事实，而不是私有成员或内部调用顺序？
2. 红灯为什么必须因目标行为不满足而失败，期望值又为何要独立计算？
3. assert 在 Release 中可能发生什么，为什么条件中不能有副作用？
4. GDB 显示参数和中间值正确、返回值错误时，根因范围如何缩小？
5. 编译器警告、静态分析与 sanitizer 分别需要多少运行信息？
6. ASan 的分配、释放、错误访问三条栈分别回答什么？
7. 为什么有符号溢出后再转宽类型不能修复溢出？
8. 一份可复核诊断报告至少应保存哪些证据？

问题 12.1 面试检查点 被问“怎样调试 C++ 崩溃”时，不要只背 GDB 命令。先说明如何用输入、配置、退出码和公开行为复现，再按症状选择调用栈、ASan、UBSan 或静态分析，最后用根因解释证据并留下回归测试。若被问测试为何在 Release 假绿，明确指出 NDEBUG 可删除标准 assert，真实测试检查必须在所有配置执行。

12.9 本章小结

可靠诊断从可观察行为开始。行为测试提供 oracle，GDB 展开具体数据流，警告与静态分析检查未运行路径，ASan 和 UBSan 把部分未定义行为变成稳定类别。每种工具都有盲区：逻辑错误不会因 sanitizer 自动现形，Release 成功也可能只是断言消失。保存复现、选择最小工具、解释全部证据、最小修复并留下回归测试，才是一条能用于量化求职项目和真实工作的调试闭环。

第四部分

量化性能与互操作

第十三章 数据布局、缓存与分配

内容提要

- ❑ 用 `sizeof`、`alignof` 与 `offsetof` 区分语言保证和本机观察
- ❑ 在相同 checksum 下比较 AoS 与 SoA 搬运的字节集合
- ❑ 观察 `vector::reserve` 与 `arena` 对分配、移动和生命周期的影响
- ❑ 把 `false sharing` 写成带缓存行假设的实验，而不是固定硬件真理

注 先修知识：已完成第 6、7、12 章，理解对象生命周期、`vector` 失效规则与 `sanitizer`。

Python 迁移边界：`NumPy` 的 `dtype`、`shape`、`strides` 和 `contiguous` 属性显式描述数组布局；C++ 对象布局还受成员顺序、对齐、填充和实现选择影响。

常见错误与诊断：先验证两种布局产生相同结果，再记录编译器、目标架构、数据规模和布局事实；若结果不同，性能数字全部作废。

13.1 本章输出：正确性受控的布局报告

“SoA 更快”“arena 没有分配”“64 字节一定是缓存行”都不是可以脱离条件复述的结论。本章先交付一份布局报告，再允许修改项目快照。报告至少包含：源码与命令、编译器和架构、数据规模、两种实现的同一 checksum、`sizeof/alignof` 等观察、模型假设以及没有测量的内容。计时留给第 14 章。

从项目根目录运行：

```
cmake -S code/ch13 -B build/ch13 -DCMAKE_BUILD_TYPE=Release
cmake --build build/ch13 -j
ctest --test-dir build/ch13 --output-on-failure
./build/ch13/ch13_object_layout
./build/ch13/ch13_aos_soa
./build/ch13/ch13_preallocation
./build/ch13/ch13_arena
./build/ch13/ch13_false_sharing_layout
```

本机 WSL2、x86-64、GCC 13.3 的一次实际输出如下；缩进续行仅用于排版，程序每项实际输出一行：

```
layout-facts-ok sizeof-quote=24 alignof-quote=8
  field-bytes=17 padding-bytes=7 offsets=0,8,16
aos-soa-ok rows=5 checksum=1496.00
  modeled-aos-bytes=160 modeled-soa-hot-bytes=80
preallocation-ok rows=32 checksum=3696.00
  unreserved-moves=31 reserved-moves=0
arena-ok rows=3 checksum=608.00 buffer-bytes=4096
false-sharing-layout-ok assumed-line-bytes=64
  slot-size=64 slot-stride=64 separated-by-assumption=true
```

换一台机器后数字可以变化；*-ok 只说明实验自己的正确性条件成立。报告不能把 24、8、31 或 64 当作 C++ 标准承诺。

13.2 对象表示：成员之外还有填充

13.2.1 规则与可观察实验

对象必须从满足其对齐要求的地址开始。`sizeof(T)` 给出数组中相邻两个 `T` 的步长，包含成员、内部填充和尾部填充；`alignof(T)` 给出对齐要求。对 `standard-layout` 类型，`offsetof(T, member)` 可观察成员相对对象起点的字节偏移。

Listing 13.1: 记录对象大小、对齐、成员字节与偏移

```

1  #include <cstdint>
2  #include <cstdint>
3  #include <iostream>
4  #include <type_traits>
5
6  struct Quote final {
7      double price;
8      std::int64_t quantity;
9      char venue;
10 };
11
12 int main() {
13     constexpr std::size_t field_bytes =
14         sizeof(double) + sizeof(std::int64_t) + sizeof(char);
15     const bool valid = std::is_standard_layout_v<Quote> &&
16         sizeof(Quote) >= field_bytes &&
17         offsetof(Quote, price) < offsetof(Quote, quantity) &&
18         offsetof(Quote, quantity) < offsetof(Quote, venue);
19     if (!valid) {
20         std::cerr << "layout invariants were not satisfied\n";
21         return 1;
22     }
23
24     std::cout << "layout-facts-ok sizeof-quote=" << sizeof(Quote)
25         << " alignof-quote=" << alignof(Quote)
26         << " field-bytes=" << field_bytes
27         << " padding-bytes=" << sizeof(Quote) - field_bytes
28         << " offsets=" << offsetof(Quote, price) << ','
29         << offsetof(Quote, quantity) << ',' << offsetof(Quote, venue)
30         << '\n';
31 }

```

三个成员本身在本机共 17 字节，`Quote` 却占 24 字节。填充让 `Quote[]` 中每个元素都正确对齐，它不是“编译器浪费的神秘空间”。成员声明顺序受保证，具体偏移、基本类型大小和尾部填充仍与 ABI/实现有关；不要把本机输出写进跨平台文件格式或网络协议。

立即练习：把 `venue` 移到两个大成员之间，在全新构建树中记录大小与偏移。若大小改变，只能写“此编译器和 ABI 下改变了”；随后恢复原顺序并确认 `checksum` 类测试不受布局观察影响。

注Python 迁移提示：NumPy 的结构化 dtype 也可能有偏移与对齐，`itemsize` 类似数组步长；普通 Python 对象还包含运行时元数据和间接引用，`sys.getsizeof` 通常不含它引用的全部对象。两边都不能只把字段标称大小

相加就声称得到真实工作集。

13.3 连续访问与无关字段：AoS 还是 SoA

Array of Structures (AoS) 把一条记录的全部字段放在一起, 适合逐条使用完整记录; Structure of Arrays (SoA) 把同一字段放进连续数组, 适合只扫描少数“热字段”。本实验计算同一名义金额, AoS 的记录还带时间戳和 venue, 而 SoA 热循环只读取价格与数量:

Listing 13.2: AoS 与 SoA 先比较结果, 再记录字段搬运模型

```

1  #include <stdint>
2  #include <iomanip>
3  #include <iostream>
4  #include <vector>
5
6  struct Quote final {
7      double price;
8      std::int64_t quantity;
9      std::int64_t timestamp;
10     char venue;
11 };
12
13 int main() {
14     const std::vector<Quote> rows{
15         {99.0, 2, 1, 'X'}, {100.0, 3, 2, 'X'}, {101.5, 4, 3, 'Y'},
16         {98.0, 5, 4, 'Y'}, {102.0, 1, 5, 'X'}};
17     const std::vector<double> prices{99.0, 100.0, 101.5, 98.0, 102.0};
18     const std::vector<std::int64_t> quantities{2, 3, 4, 5, 1};
19
20     double aos_checksum = 0.0;
21     for (const Quote& row : rows) {
22         aos_checksum += row.price * static_cast<double>(row.quantity);
23     }
24
25     double soa_checksum = 0.0;
26     for (std::size_t index = 0; index < prices.size(); ++index) {
27         soa_checksum += prices[index] * static_cast<double>(quantities[index]);
28     }
29
30     if (aos_checksum != 1'496.0 || soa_checksum != aos_checksum) {
31         std::cerr << "layout variants changed the calculation\n";
32         return 1;
33     }
34
35     const std::size_t modeled_aos_bytes = rows.size() * sizeof(Quote);
36     const std::size_t modeled_soa_hot_bytes =
37         rows.size() * (sizeof(double) + sizeof(std::int64_t));
38     std::cout << std::fixed << std::setprecision(2)
39         << "aos-soa-ok rows=" << rows.size()

```

```

40         << " checksum=" << aos_checksum
41         << " modeled-aos-bytes=" << modeled_aos_bytes
42         << " modeled-soa-hot-bytes=" << modeled_soa_hot_bytes << '\n';
43     }

```

两个 checksum 都是 1496，因而布局没有改变业务语义。modeled-*-bytes 只是源码层模型：AoS 按 `rows * sizeof(Quote)` 计算完整记录字节，SoA 按两个热数组的元素大小计算。它不是硬件计数器读数，没有证明 CPU 恰好搬运这些字节，也没有计入预取、缓存行、TLB、向量化或额外指针。

若下一步操作同时需要价格、数量、时间和 venue，AoS 可能避免从多条数组收集字段；若只扫描价格，SoA 可能减少无关字段进入工作集。选择来自访问模式，不来自容器名字。第 14 章会在相同数据、交替顺序和 checksum 保护下测量；本章不从复杂度相同推断速度相同，也不从字节模型直接推断耗时比例。

立即练习：为两种循环都增加 venue 过滤，列出现在实际读取的字段。先保持 checksum 相同，再更新“模型字节”；不要计时，先解释为什么原来的冷热字段判断已经变化。

13.4 预分配：减少搬迁，不等于增加元素

`vector::reserve(n)` 只保证容量至少为 `n`，`size()` 不变；`resize(n)` 会真正创建或销毁元素。容量不足时，`vector` 申请新存储，把现有元素复制或移动过去，再释放旧存储；指针、引用和迭代器因而失效。

`code/ch13/experiments/preallocation.cpp` 用带移动计数的值类型构造两组相同 32 行。保留容量的一组在本次实现上已有元素移动数为 0，未保留的一组为 31；两组 checksum 都是 3696。测试只要求结果相同且预留组不搬迁已有元素，不要求“31”跨标准库稳定，因为 `vector` 的增长策略没有固定倍率保证。

核心语法只有两步：

```

std::vector<TrackedQuote> quotes;
quotes.reserve(expected_rows); // capacity >= expected_rows, size still 0
quotes.emplace_back(100.0);    // now size becomes 1

```

预分配需要可信上界；把攻击者提供的行数直接用于 `reserve` 可能造成巨大分配。过度预留也会提高峰值内存和工作集。若保存了 `data()` 或元素地址，仍需保证后续不超过容量；`reserve` 不是永久地址承诺。

立即练习：`reserve(3)` 后插入 3 个元素，记录实际 `capacity()` 和移动计数；继续插入到 `size()` 超过该容量，再记录两者。容量一旦改变，先前元素地址便全部失效，不要读取或比较旧指针。

13.5 Arena 与 std::pmr：分配策略不是所有权策略

大量同寿命小对象可从 arena 顺序取存储并批量释放。`std::pmr` 容器把资源从容器类型中解耦。本章用栈上 4096 字节缓冲构造 `monotonic_buffer_resource`，`upstream` 设为 `null_memory_resource()`；容量不足会明确抛出。

关键构造如下，完整可运行源码在 `code/ch13/experiments/arena.cpp`：

```

std::array<std::byte, 4'096> storage{};
std::pmr::monotonic_buffer_resource arena{
    storage.data(), storage.size(), std::pmr::null_memory_resource()};
std::pmr::vector<Quote> quotes{&arena};
quotes.reserve(3);

```

arena 必须比使用它的容器和对象活得久。批量释放存储也不等于可以跳过对象析构：让容器正常离开作用域，元素析构仍会运行。若先销毁资源却保留指向其中的指针，指针立刻悬空。隔离的失败实验正返回 arena 内地址：

Listing 13.3: 故意失败：资源销毁后继续解引用 arena 地址

```

1 #include <memory_resource>
2 #include <vector>
3
4 double* escaped_arena_price() {
5     std::pmr::monotonic_buffer_resource arena;
6     std::pmr::vector<double> prices{&arena};
7     prices.push_back(100.0);
8     return prices.data();
9 }
10
11 int main() {
12     double* price = escaped_arena_price();
13     return *price > 0.0 ? 0 : 1;
14 }

```

从项目根目录触发下列隔离测试：它会以 `-fsanitize=address` 单独编译并运行失败样例：

```

cmake -S . -B build/ch13-asan -DCMAKE_BUILD_TYPE=Debug
ctest --test-dir build/ch13-asan -R ch13_arena_dangling \
    --output-on-failure

```

失败阶段是资源销毁后的解引用，稳定类别为 `heap-use-after-free`；地址和调用栈不是契约。恢复方法是把消费者留在资源作用域、复制逃逸值，或让拥有资源的对象同时拥有使用者。

立即练习：复制样例为 `arena_copy.cpp`，让函数在 `arena` 存活时按值返回 `double`。用 `g++ -std=c++20 -fsanitize=address arena_copy.cpp -o arena_copy` 编译并直接运行 `./arena_copy`，要求正常退出且值为 100；不要调用上面期望失败的 `CTest`。

13.6 False sharing: 先声明缓存行假设

不同原子变量若落在同一一致性块，两个线程写入仍可能让缓存所有权来回转移，即 `false sharing`。变量无 C++ 数据竞争，却可能有硬件开销；原子与并发测量留到第 15 章，本章只检查布局假设：

```

constexpr std::size_t experiment_line_bytes = 64;
struct alignas(experiment_line_bytes) CounterSlot {
    std::atomic<std::uint64_t> value{};
};

```

程序打印 64 字节假设、slot 大小和地址步长。`alignas(64)` 只保证满足源码的对齐请求，不证明缓存行就是 64，也不排除对象共享更大的硬件块。`std::hardware_destructive_interference_size` 可作线索，仍须记录环境并实验验证。

立即练习：把假设改为 128，记录 `sizeof` 和 `stride` 的变化。解释为什么内存占用已经确定增加，却仍不能在多线程测量时声称吞吐提高。

13.7 有证据后才进入章节快照

AoS/SoA 实验先证明价格与数量的两列计算保持 1496，再显示本工作负载不读取 `timestamp` 与 `venue`。只有得到这两项证据后，章节快照才采用列式价格/数量数组完成阈值扫描；它没有修改最终回测引擎，也没有声称列式布局适合所有阶段。

Listing 13.4: 第 13 章快照：列式热字段行情扫描

```

1 #include <cstdint>
2 #include <iomanip>
3 #include <iostream>
4 #include <vector>
5
6 int main() {
7     const std::vector<double> prices{99.0, 100.0, 101.5, 98.0, 102.0};
8     const std::vector<std::int64_t> quantities{2, 3, 4, 5, 1};
9     constexpr double threshold = 100.0;
10
11     std::size_t selected = 0;
12     double notional = 0.0;
13     for (std::size_t index = 0; index < prices.size(); ++index) {
14         if (prices[index] >= threshold) {
15             ++selected;
16             notional += prices[index] * static_cast<double>(quantities[index]);
17         }
18     }
19
20     std::cout << std::fixed << std::setprecision(2)
21               << "quote-scan-ok rows=" << prices.size()
22               << " selected=" << selected << " notional=" << notional << '\n';
23 }

```

初始实现使用 `price > 100`，把恰好等于阈值的行情错误排除，输出 `selected=2 notional=508.00`。公开输出测试先红；最小修复为 `>=`，最终精确输出：

```
quote-scan-ok rows=5 selected=3 notional=808.00
```

输出任务：从空目录构建 `code/ch13`，保存五个实验输出和工具链环境；说明每个数字是语言保证、实现观察还是实验假设；再运行行情扫描并用独立手算解释 808。完整核心答案见附录。

自测：

1. `sizeof`、`alignof` 与 `offsetof` 分别观察什么，哪些数字不能写进协议？
2. 17 字节成员为何可能组成 24 字节对象，尾部填充对数组有什么作用？
3. AoS 与 SoA 的选择为什么取决于访问字段，而不是谁“理论上更快”？
4. `checksum` 不同的两种布局为什么没有资格比较性能？
5. `reserve` 与 `resize` 对 `size`、对象构造和地址失效有何差异？
6. 为什么预分配既可能减少搬迁，也可能增加峰值内存风险？
7. `arena` 资源、PMR 容器和逃逸指针必须满足什么生命周期顺序？
8. `alignas(64)` 为什么不能证明硬件缓存行固定为 64 字节？

问题 13.1 面试检查点 被问“怎样优化数据布局”时，先描述访问模式和正确性 `oracle`，再记录对象表示与工作集，提出 AoS/SoA 或预分配假设，最后用第 14 章的方法测量。主动说明 ABI、标准库增长策略、`arena` 生命周期和缓存行假设的边界；不要用一个 `sizeof` 或单次耗时给出跨机器结论。

13.8 本章小结

数据布局决定对象怎样占据地址空间，访问模式决定哪些字节可能进入工作集，分配策略决定存储取得与回收的节奏。`reserve` 和 `arena` 能减少某些分配或搬迁，却引入容量估计与生命周期责任；对齐可构造 `false-sharing` 实验，却不能替硬件作保证。始终先守住 `checksum` 和公开行为，再记录环境、假设与限制，项目改造才有可审计的依据。

第十四章 Benchmark 与 profiling

内容提要

- ❑ 把“更快”改写成可反驳假设和正确性受控的实验
- ❑ 区分中位数、IQR、吞吐与尾延迟回答的问题
- ❑ 用预热、重复样本、交替顺序和结果消费减少常见测量偏差
- ❑ 用 profiler 和 Amdahl 定律选择端到端优化优先级

注 先修知识：已完成第 12、13 章，能够构建 Release 目标并解释 checksum 与布局实验。

Python 迁移边界：Python 的 timeit、cProfile 和 NumPy 经验有帮助，但 C++ 优化器、编译配置和原生 profiler 改变了证据边界。

常见错误与诊断：先写正确性 oracle 与环境，再收集原始样本；若结果不同或工作负载不代表目标系统，全部性能结论作废。

14.1 本章输出：一份可被推翻的性能报告

“SoA 比 AoS 快”不是实验假设，因为它没有限定程序、输入、机器、编译器、配置和指标。可检验版本是：在本章固定的 100 万行热字段扫描、GCC 13.3、x86-64、Release 配置下，SoA 的单进程批处理耗时中位数低于 AoS；两者必须先得到同一手算 checksum。

从项目根目录构建独立实验：

```
cmake -S code/ch14 -B build/ch14-release -DCMAKE_BUILD_TYPE=Release
cmake --build build/ch14-release -j
ctest --test-dir build/ch14-release --output-on-failure
./build/ch14-release/ch14_layout_benchmark
```

合格报告保存六项：假设；完整命令；编译器、架构、配置与数据规模；未经挑选的原始样本；正确性和统计摘要；结论与限制。先写判负条件，例如 checksum 不同、机器发生明显负载干扰或样本不足；不能看完数字后再改变成功标准。

14.2 微基准协议：先保护语义，再开始计时

code/ch14/experiments/layout_benchmark.cpp 固定 100 万行，使每 200 行的手算名义金额为 920716，总 oracle 为 4603580000。程序在计时前分别预热 3 次，收集 21 对样本；偶数轮先测 AoS，奇数轮先测 SoA，以减少固定“第一个/第二个”位置偏差。每轮还按运行期样本编号轮换起点，两个布局仍按相同次序处理全部元素。下面只是核心片段，容器构造、计时函数、统计和输出见完整源码：

```
for (int sample = 0; sample < samples; ++sample) {
    const auto start_index = (sample + 1) * 7919ULL % rows;
    if (sample % 2 == 0) {
        const auto [aos_sum, aos_us] = measure(run_aos, start_index);
        const auto [soa_sum, soa_us] = measure(run_soa, start_index);
        checksums_match = checksums_match && aos_sum == soa_sum;
    } else {
        // 同样测量并检查，但先 SoA 后 AoS。
    }
}
```



```
}
```

“消费结果”不是随手加 `volatile`。本例让返回值参与 `oracle` 和最终输出，轮换起点又让扫描依赖运行期样本编号；计时函数还用 GCC/Clang 或 MSVC 的编译器屏障把本轮乘加留在两次读钟之间，I/O 则在计时区外。屏障是编译器相关的测量细节，并非 C++ 对优化的通用禁令；仍须记录编译器与配置，必要时检查生成代码。预热能减少首次页错误、冷代码等一次性影响，却不保证缓存状态代表生产流量。

本机 WSL2、GCC 13.3、x86-64 的一次 Release 运行得到。以下是格式化摘录，保留程序真实字段名；教材页内省略了两组 `raw-us` 的 21 个具体值，正式实验报告不得省略：

```
benchmark-ok rows=1000000 samples=21 warmups=3
  alternating=true checksum-match=true rotating-start=true
environment compiler=GCC 13.3.0 architecture=x86-64 configuration=Release
  ndebug=true clock=steady_clock
aos-raw-us=[21 values omitted] median-us=750.00 iqr-us=287.00
soa-raw-us=[21 values omitted] median-us=725.00 iqr-us=151.00
checksum-guard=220971839999.68 conclusion=environment-specific
```

完整程序输出两组各 21 个原始样本。这里的中位数相差 25 微秒，但样本范围重叠，而且只运行了一个进程实例；证据只支持“这次实验通过正确性门并观察到小差异”，不支持“所有机器上 SoA 快 3.3%”。下一步应保存原始输出，交替启动多个独立进程，并调查异常样本，而不是删除不喜欢的点。

立即练习：把测量顺序改成永远先 AoS，重新运行三个独立进程；比较两种协议的中位数与 IQR。即使方向未变，也要解释为什么交替顺序仍是更可信的设计。

14.3 分布与指标：一个数字回答不了四个问题

`code/ch14/experiments/statistics_report.cpp` 使用两组 8 个固定的串行请求延迟。两组中位数均为 10 微秒，却得到：

```
stable median-us=10.00 iqr-us=0.50 p99-us=12
  throughput-ops-per-s=97560.98
bursty median-us=10.00 iqr-us=37.50 p99-us=100
  throughput-ops-per-s=34782.61
```

中位数把一半样本放在两侧，适合抵抗少量极端值，却不描述抖动；本章把上下各半样本的中位数定义为 Q_1, Q_3 ，IQR 为 $Q_3 - Q_1$ 。 p_{99} 关注尾部体验，但 8 个样本的 nearest-rank p_{99} 只是最大值，远不足以估计真实 99 分位。吞吐按“单线程、请求串行完成”模型用 $n/\sum t_i$ 计算；并行、批处理或流水线系统不能直接用延迟倒数替代吞吐。

批处理基准的“一批耗时 p_{99} ”也不是单条行情尾延迟。先说清观测单位、并发度和时间窗口，再选指标。报告至少给样本数和原始分布；只报 best-of-N 会系统性选择幸运噪声，只报平均数又可能掩盖长尾。

注Python 迁移提示：Python 的 `timeit` 也会重复运行并通常报告多次结果，但不能自动证明输入、线程数和底层 BLAS 状态相同；`cProfile` 是确定性调用 profiler，开销模型不同于采样 profiler。NumPy 调用把工作移进原生库时，应同时记录 Python 边界和原生热点，不能只看 Python 行级时间。

立即练习：为 bursty 样本再增加 100 个 10 微秒请求，重新计算中位数、IQR 和 nearest-rank p_{99} 。解释为什么分位数会跳变，以及原来的 8 个样本为何不能支持尾延迟 SLA。

14.4 Profiler 先定位，再用 Amdahl 约束收益

微基准回答一个隔离假设，profiler 回答端到端时间落在哪里。code/ch14/experiments/profile_workload.cpp 保留 decode_stage、score_stage 和 aggregate_stage 三个阶段；本机 WSL 没有 perf，但有 GNU gprof，实际复现命令是：

```
cd build/ch14-release
g++ -std=c++20 -O2 -g -pg ../../code/ch14/experiments/profile_workload.cpp \
-o profile_workload_pg
./profile_workload_pg
gprof ./profile_workload_pg gmon.out | head -n 12
```

这次报告以 0.01 秒为一个采样 tick，共把 0.11 秒归到 score_stage，其余阶段显示 0。它说明人工工作负载的候选热点位置，却不证明百分比精确，更不代表真实回测；应延长运行、重复独立进程，并在目标生产形态上复测。可用 perf 时，perf record -g -- executable 与 perf report 能提供采样调用栈，但权限、内核和符号质量也必须写进报告。

把 profiler 的阶段时间放回端到端预算。若总计 100 ms，热点为 60 ms，局部加速 4 倍，新预算是 $40 + 60/4 = 55$ ms，而不是 25 ms：

Listing 14.1: 把热点比例转换为整体收益上限

```
1 #include <iomanip>
2 #include <iostream>
3
4 int main() {
5     constexpr double baseline_total_ms = 100.0;
6     constexpr double hotspot_ms = 60.0;
7     constexpr double local_speedup = 4.0;
8
9     constexpr double unaffected_ms = baseline_total_ms - hotspot_ms;
10    constexpr double predicted_total_ms =
11        unaffected_ms + hotspot_ms / local_speedup;
12    constexpr double overall_speedup =
13        baseline_total_ms / predicted_total_ms;
14    constexpr double ceiling_speedup = baseline_total_ms / unaffected_ms;
15
16    std::cout << std::fixed << std::setprecision(2)
17        << "amdahl-ok total-ms=" << baseline_total_ms
18        << " hotspot-ms=" << hotspot_ms
19        << " fraction=" << hotspot_ms / baseline_total_ms
20        << " local-speedup=" << local_speedup
21        << " predicted-total-ms=" << predicted_total_ms
22        << " overall-speedup=" << overall_speedup
23        << " ceiling-speedup=" << ceiling_speedup << '\n';
24 }
```

输出整体加速 1.82 倍；即使热点无限快，未受影响的 40 ms 仍把上限锁在 2.5 倍。立即练习：把热点份额改为 20%，局部加速仍为 4 倍，先手算整体收益再运行程序；说明为什么“优化最慢函数”仍可能不值得工程成本。

14.5 失败实验：结果不同，性能比较立即作废

下面的候选实现把 `>=` 偷换为 `>`，漏掉价格恰为阈值的行情。它可能少做工作，但不再具有比较资格：

Listing 14.2: 故意失败：checksum 揭示语义漂移

```

1  #include <cstdint>
2  #include <cstdint>
3  #include <iostream>
4  #include <vector>
5
6  int main() {
7      const std::vector<double> prices{99.0, 100.0, 101.5, 98.0, 102.0};
8      const std::vector<std::int64_t> quantities{2, 3, 4, 5, 1};
9      constexpr double threshold = 100.0;
10
11     double baseline = 0.0;
12     double candidate = 0.0;
13     for (std::size_t index = 0; index < prices.size(); ++index) {
14         if (prices[index] >= threshold) {
15             baseline += prices[index] * static_cast<double>(quantities[index]);
16         }
17         if (prices[index] > threshold) { // Deliberate semantic drift.
18             candidate += prices[index] * static_cast<double>(quantities[index]);
19         }
20     }
21
22     if (baseline != candidate) {
23         std::cerr << "benchmark-invalid checksum-mismatch baseline=" << baseline
24                 << " candidate=" << candidate << '\n';
25         return 2;
26     }
27     return 0;
28 }
```

从根目录隔离触发：

```

cmake -S . -B build/ch14-debug -DCMAKE_BUILD_TYPE=Debug
ctest --test-dir build/ch14-debug -R ch14_checksum_mismatch \
    --output-on-failure
```

失败发生在运行期正确性门，退出码为 2，稳定类别是 `benchmark-invalid checksum-mismatch`；508 与 808 分别是错误候选和基线结果。恢复步骤是在副本中恢复 `>=`、直接编译运行并要求退出 0，再回到正向微基准；不要把修复后的副本交给“期望失败”的负向 CTest。

14.6 从原始数字到有限结论

同一进程内的 21 个样本能显示短时抖动，却共享进程启动、地址布局、系统背景负载和频率状态，不能冒充 21 次完整独立实验。这叫伪重复风险。更可信的下一层证据是启动多个独立进程，每个进程保留完整样本；若目标部署跨机器，还应在目标硬件类别上分层重复，而不是把所有数字混成一个平均值。

本机一次观察中，SoA 与 AoS 中位数相差约 3.3%。这个差异小于任一版本一次明显异常样本的幅度，并且两组区间重叠。合格结论可以写成：

在记录环境的一次 Release 进程中，两种实现均通过 4603580000 的 checksum；观察到 SoA 中位数略低，但现有证据不足以确认稳定且具有实际意义的收益，因此暂不改变项目实现，下一步进行多进程复测并检查硬件计数器。

“实际意义”也应在看结果前定义。例如只有端到端吞吐至少提高 5%、内存不超预算且尾延迟不退化，改造成本才合理；5% 只是示例，真实阈值来自容量计划和业务 SLO。统计显著不自动等于值得上线，未达统计显著也不代表系统一定相同。

把报告组织为可审计链条：

1. **观察：**保存原始样本、环境、checksum 和 profiler 表，不先解释原因；
2. **有限主张：**只覆盖实际测量的工作负载、机器与配置，并写出不确定性；
3. **工程决定：**采用、拒绝或继续实验，同时列出收益门槛和回滚条件；
4. **后续验证：**说明要增加的独立进程、数据集、硬件事件或端到端指标。

可直接复制的最小报告骨架如下；原始样本过长时应作为文本附件保存，正文链接附件，不能用省略号代替：

```
hypothesis:                                invalidate-if:
command:                                    environment: compiler / CPU / OS / configuration
workload: rows / fields / concurrency / warmups / order
oracle: expected / observed / exit status
raw-samples: attached path, no cherry-picking
summary: sample count / median / IQR / throughput or tail
profile-command-and-hotspot:
amdahl-budget:
decision-and-practical-threshold:
limitations-and-next-test:
```

14.7 输出任务与自测

输出任务：提交布局实验的假设、命令、环境、oracle、全部原始样本、中位数/IQR、结论与限制；附 profiler 命令和热点表，并代入 Amdahl 模型。不得只交截图、最快一次或脱离环境的百分比。

自测：

1. 为什么 oracle 必须先通过，输出 checksum 又为何不能禁止所有合法优化？
2. 预热、重复样本和交替顺序分别减少什么偏差，又不能保证什么？
3. 中位数相同的两组样本为何仍可能具有完全不同的工程风险？
4. IQR、吞吐与 p_{99} 各依赖什么口径，为什么 8 个样本不足以支持尾延迟 SLA？
5. 微基准与 profiler 分别回答什么问题？
6. 热点占 60%、局部加速 4 倍时，为什么整体只有约 1.82 倍？

问题 14.1 面试检查点 被问“怎样证明优化有效”时，按假设、oracle、Release 环境、采样分布、profiler 热点、Amdahl 上限和限制回答；没有独立复测或目标硬件数据，就主动缩小结论。

14.8 本章小结

Benchmark 是受控反驳实验，不是产生漂亮数字的脚本；profiler 是选择优化位置的证据，不是自动修复器。先守住业务结果，再保存环境与分布，最后把局部热点放回端到端预算，性能结论才可审计、可复现、可推翻。

第十五章 并发、任务与背压

内容提要

- ❑ 从所有权和 happens-before 判断并发程序是否正确
- ❑ 比较顺序、jthread 与 async 的结果和失败边界
- ❑ 用 mutex 维护复合不变量，用有界队列表达背压
- ❑ 设计可停止、可关闭、可传播异常且无需 sleep 的协议

注 先修知识：已完成第 6、12、14 章，能够推理对象寿命、运行 sanitizer，并先验证结果再测量。

Python 迁移边界：Python 线程、concurrent.futures 与 queue.Queue 提供相似外形；C++ 没有替共享对象自动加锁，数据竞争直接属于未定义行为。

常见错误与诊断：先画对象所有者、读写者和停止边；若只能靠延时“等线程差不多启动”，协议尚未建立。

15.1 本章输出：同一结果，三种执行模型

量化流水线常把行情批次分片计算，但“开两个线程”不是完成判据。code/ch15/experiments/pipeline_compare.cpp 先用 8 条整数分行情建立顺序 oracle，再让两个 std::jthread 和两个 std::async 任务计算同一分区。独立手算为 201100 分；三条路径不一致时，任何吞吐比较立即作废。

从项目根目录构建独立快照：

```
cmake -S code/ch15 -B build/ch15-debug -DCMAKE_BUILD_TYPE=Debug
cmake --build build/ch15-debug -j
ctest --test-dir build/ch15-debug --output-on-failure
./build/ch15-debug/ch15_pipeline_compare
```

稳定输出是：

```
pipeline-ok rows=8 sequential-cents=201100
thread-cents=201100 task-cents=201100 checksum-match=true
```

这里验证的是结果，不是“并发更快”。8 条数据远小于线程启动和调度成本，发布构建也不能让微小工作自动获得收益。立即练习：只把最后一条数量从 2 改为 3，先手算新 oracle 211150，再修改常量并要求三条路径同时通过。

15.2 所有权与 happens-before

线程启动后可以与创建者交错执行。若两个线程访问同一内存位置、至少一个是写入，并且冲突访问之间没有 happens-before，就形成 data race；C++ 对此给出的不是“偶尔少算”，而是未定义行为。先问谁拥有数据、谁只读、谁写入，再选同步工具。

线程版把只读 quotes 借给两个工作线程，每个线程只写 partials 的不同数组元素：

```
// 省略类型和 scan_range 定义；完整程序见源码。
std::array<std::int64_t, 2> partials{};
{
    std::jthread first{[&] { partials[0] = scan_range(quotes, 0, middle); }};
    std::jthread second{[&] {
        partials[1] = scan_range(quotes, middle, quotes.size());
```

```
    }  
} // 两个 jthread 析构并 join, 之后才读取 partials。
```

数组元素是不同内存位置；作用域结束时 `jthread` 请求停止并等待线程结束。线程完成同步于成功返回的 `join`，所以线程内写入 `happens-before` 之后的求和。若改用 `std::thread`，仍必须在所有控制流上 `join` 或 `detach`；一个仍 `joinable` 的 `thread` 析构会终止进程。`detach` 又会把引用捕获的寿命责任交给调用者，本章不把它当默认方案。

注Python 迁移提示：CPython 的 GIL 不等于业务对象的复合操作具有事务性，也不代表 C++ 移植后仍安全。Python 名称捕获常延长对象引用；C++ 的 `[&]` 只是借用，任务若越过局部对象寿命便会悬空。两边都应明确所有权和完成协议。

立即练习：把线程作用域后的求和移动到作用域内部，解释它为何可能与写入并发；修复不是加 `sleep`，而是恢复明确的 `join` 边。

15.3 任务、future 与异常传播

线程接口表达“在哪个线程运行”，任务接口更接近“计算一个结果”。本章显式传入 `std::launch::async`，避免实现选择 `deferred` 后到 `get` 才在调用线程执行。`future::get()` 等待完成、取得值，并把任务中保存的异常重新抛给调用者；每个 `future` 的结果只能 `get` 一次。

`code/ch15/experiments/protocol_demo.cpp` 让裸 `jthread` 抛出 `runtime_error`。线程入口捕获后写入 `promise`，调用方再于 `get()` 处捕获。若任由异常逃出入口函数，进程会终止；异常不会自动穿过线程边界。

失败可由“在线程入口直接 `throw`”触发；诊断类别是 `terminate`，根因是跨越了线程异常边界。修复是在入口内 `catch(...)`，用 `promise::set_exception(std::current_exception())` 写入共享状态，再由控制线程的 `future` 统一处理。Python 的 `concurrent.futures.Future.result()` 同样会等待并重抛任务异常；但 C++ 的启动策略、捕获寿命与裸线程终止规则都不同，不能照搬 Python 的引用直觉。

立即练习：在 `pipeline_compare.cpp` 的任务版移除显式 `std::launch::async`，打印任务与主调方线程 ID；仍须得到 `checksum-match=true`，并记录实现实际选择，但不要把一次观察写成标准保证。

15.4 mutex、条件变量与复合不变量

有界队列的状态不是一个整数，而是 `{items, closed}`：关闭后不得再写，关闭前容量不得超限，关闭后已有元素仍可排空。`code/ch15/include/quant/ch15/bounded_queue.hpp` 用同一 `std::mutex` 保护这组字段；每次检查和修改都在一段临界区内完成。

阻塞读取的核心如下，省略了类声明、写入、关闭和查询成员：

```
std::unique_lock lock{mutex_};  
const bool ready = not_empty_.wait(lock, stop, [this] {  
    return closed_ || !items_.empty();  
});  
if (!ready || stop.stop_requested()) return stopped_result;  
if (items_.empty()) return closed_result;  
T value = std::move(items_.front());  
items_.pop_front();  
not_full_.notify_one();
```

`wait` 在睡眠时释放 `mutex`，被唤醒后重新取得锁并检查谓词；谓词同时处理伪唤醒。当前协议规定停止优先：一旦 `stop` 已请求，等待操作返回 `stopped`，不再消费缓冲元素。`close` 则拒绝新写入，但允许消费者排空现有元素，之后返回 `closed`。

`std::atomic` 适合一个独立值，例如只作观测的 `processed` 计数。即使 `cash` 和 `position` 各自是 `atomic`，也不能让读者得到同一时刻的二字段快照；更新之间仍可被观察。复合不变量继续用一把 `mutex` 或单所有者消息传递。若 `relaxed` 原子只计数，它也不发布另一对象的内容，不能替代队列同步。

若把谓词检查移到锁外，可用两个生产者同时通过“尚未满”的检查来触发越界；TSan 可能报告竞态，而容量断言报告的是不变量破坏。根因是检查与修改不再原子，修复是让同一 `mutex` 覆盖整段事务。Python 的 `queue.Queue(maxsize=...)` 也把 `mutex` 与条件等待封装在队列里；C++ 模板还必须明确元素移动、对象寿命与停止优先级，解释器锁不会替普通 C++ 容器同步。

立即练习：把 `closed_` 改为 `atomic` 却让 `deque` 保持原样，画出“读到已关闭但仍未看见最后一项”的错误推理；再解释为什么统一锁更直接。

15.5 缓冲满就是业务协议

无界队列把下游变慢转化为内存增长；有界队列必须在满时作决定。常见策略是阻塞生产者、拒绝并让上游重试、丢弃最新/最旧、合并更新或溢写磁盘。选择取决于行情是否可丢、顺序要求、延迟预算和恢复成本，不能只写“使用无锁队列”。

`code/ch15/tests/bounded_queue_behavior.cpp` 固定容量 2，前两次 `try_push` 返回 `accepted`，第三次返回 `full`；排空后关闭，后续写入返回 `closed`：

```
queue-ok capacity=2 accepted=2 full=1 drained-sum=30 closed=true
```

`full` 只是队列事实，调用方才决定策略。输出任务采用 `drop-newest`，并且每个被提供的元素必须计入 `accepted` 或 `dropped`，不能静默消失。立即练习：把容量改为 3，五个输入应得到 `accepted=3`、`dropped=2`、`drained-sum=60`。

若监控只看到内存上升或尾延迟恶化，触发方法是让生产速率持续高于消费速率；诊断应记录队列深度、`full` 次数与丢弃率，根因是没有可观测的容量协议，修复才是按业务选择阻塞、拒绝或丢弃。

Python `Queue.put` 的 `block` 参数与 `Full` 异常提供相似机制，但也不会替应用选择能否丢行情、如何计数和怎样恢复。

15.6 启动、停止与关闭协议

可靠关闭需要顺序而不是延时：先创建共享状态；启动工作线程；用 `promise/future` 确认入口已执行；停止时请求 `stop` 并唤醒等待；生产结束时 `close`；最后 `join`；主线程再销毁队列。工作线程内部捕获异常并写入 `promise`，控制线程在 `future` 边界决定记录、重试或终止。

协议实验同时覆盖空队列消费者和满队列生产者。主线程看到 `started future` 后立即请求停止；无论工作线程已经进入 `wait`，还是刚要调用它，`stop-aware` 谓词都返回稳定状态，不需要 `sleep`：

```
protocol-ok started=true consumer-stop=true producer-stop=true
exception-propagated=true queues-closed=true
```

停止请求不是强制杀线程；代码必须在阻塞点或循环边界协作检查。若任务正在无法取消的外部调用中，请求也不能保证立即结束。立即练习：去掉 `close` 但保留 `join`，指出哪一类不接收 `stop token` 的消费者可能永久等待，并给出显式终止条件。

这个永久等待可由“空队列、无 `stop-aware wait`、生产者退出且未 `close`”稳定触发；诊断类别是 `join` 卡住，根因是消费者没有终止事件，修复是把 `stop` 或 `close` 纳入等待谓词并负责唤醒。Python `threading.Event` 也只是协作信号，`Thread` 没有安全的强制取消；`Event` 本身不会自动唤醒任意队列等待，同样需要完整关闭协议。

15.7 失败实验：让 ThreadSanitizer 建立证据

下面两个线程由 `barrier` 同步起跑，随后写同一个普通整数。`barrier` 只建立起跑前后的阶段边，不为两条递增循环彼此排序；程序也没有用 `sleep` 猜测交错。

Listing 15.1: 故意失败：未同步的共享写入

```

1  #include <barrier>
2  #include <iostream>
3  #include <thread>
4
5  int main() {
6      constexpr int increments_per_thread = 100'000;
7      int shared_counter = 0;
8      std::barrier start_line{3};
9
10     const auto increment = [&] {
11         start_line.arrive_and_wait();
12         for (int index = 0; index < increments_per_thread; ++index) {
13             ++shared_counter; // Intentional data race: two unsynchronized writers.
14         }
15     };
16
17     std::jthread first{increment};
18     std::jthread second{increment};
19     start_line.arrive_and_wait();
20     first.join();
21     second.join();
22
23     std::cout << "invalid-racy-counter=" << shared_counter << '\n';
24 }
```

从根目录隔离运行：

```

cmake -S . -B build/ch15-root -DCMAKE_BUILD_TYPE=Debug
ctest --test-dir build/ch15-root -R ch15_data_race \
    --output-on-failure
```

本机 GCC 13.3 的稳定类别是 `WARNING: ThreadSanitizer: data race`，报告把两次冲突访问都指向递增行。某些 WSL 地址布局会先报 `unexpected memory mapping`；测试驱动只在遇到该宿主限制时用 `setarch` 为当前进程关闭 ASLR 后重试。计数恰好等于 200000 也不能证明安全，因为 UB 不以“这次结果看起来对”自证。

恢复方案取决于语义：若只需一个计数，可用 `atomic`；若字段组成不变量，用 `mutex`；若可分区，像顺序基线那样让每线程写私有 `partial`，`join` 后归并通常最清楚。修复后再用 TSan 复测，但“本次未报告”仍不等于覆盖了所有调度路径。

15.8 测量开销，不追求无锁炫技

并发实验仍遵守第 14 章证据链：先比对 `oracle`，再记录 `Release` 环境、核心数、数据规模、线程数和原始样本。单批耗时需包含还是排除线程池启动必须写明；队列还应报告阻塞时间、满次数、丢弃率和尾延迟。若工作量太小、内存带宽已饱和或竞争集中，并发可能更慢。

顺序、线程和任务版本只能在相同输入及结果下比较。不要从这个 8 行正确性样本推断吞吐，也不要使用 `hardware_concurrency()` 返回值当成可用核心或最佳线程数保证。先测端到端瓶颈，再决定是否值得引入并发状态空间。

15.9 输出任务与自测

输出任务：提交 `code/ch15` 的 `clean-build` 命令与四项证据：三种执行模型都得到 201100；容量 2 的 `drop-newest` 报告完整计数；启动、双向停止、关闭和异常传播协议图；`ThreadSanitizer` 的稳定竞态类别。另写一段测量计划，说明 `oracle`、样本、线程数、队列指标和停止条件，不要求实现无锁结构。

自测：

1. `data race` 与一般“先后顺序不确定”有什么区别？
2. `join` 建立了哪条 `happens-before` 边，引用捕获还要求什么寿命？
3. 为什么两个 `atomic` 字段不是一个一致快照？
4. 条件变量为什么必须与谓词和 `mutex` 一起使用？
5. 队列返回 `full` 后，哪些背压策略可能合理，依据是什么？
6. `request_stop`、`close` 与 `join` 各解决什么问题？
7. `raw thread` 与 `future` 的异常边界有什么不同？
8. 为什么正确性样本不能证明并发版本更快？

问题 15.1 面试检查点 被问“怎样设计行情处理并发管道”时，先给所有权和 `happens-before` 图，再说明有界容量、满策略、停止/关闭/异常协议及正确性 `oracle`；最后才讨论测量与线程数。主动说明 `atomic` 不能自动维护复合状态，也不以“无锁”作为目标。

15.10 本章小结

并发正确性来自所有权与明确同步，不来自一次正确输出；背压和停止是接口协议，不是队列实现细节。先保留顺序 `oracle`，用 `join`、`mutex`、`future` 和 `stop-aware` 等待建立可推理边界，再用 `sanitizer` 与受控测量补证据，才能安全地把并发带入量化系统。

第十六章 数值稳定性与 Python 互操作

内容提要

- 从二进制表示解释浮点比较失败，并给容差写出来源
- 区分金额、收益率和方差的表示与舍入责任
- 用补偿求和与在线方差保护可复现的统计口径
- 设计明确 dtype、连续性、GIL 和所有权的批量 pybind11 边界

注 先修知识：已完成第 5、12、14 章，能够维护类不变量、写行为测试并以正确性 oracle 约束测量。

Python 迁移边界：Python float 与 C++ double 通常同为 binary64，但 NumPy 数组还携带 dtype、shape、strides 与所有者；跨语言后这些元数据必须显式验收。

常见错误与诊断：看到末位差异先查表示、运算顺序与口径；看到扩展崩溃先查 dtype、连续性和寿命，不用“浮点误差”概括所有问题。

16.1 本章输出：先验证算法，再跨边界

Python 研究代码移到 C++ 时，两个问题常被错误地混在一起：数值结果为何改变，以及调用为什么没有变快。code/ch16 把它们拆成一个无第三方依赖的 C++20 核心和一个可选 pybind11 薄绑定。核心始终可构建；只有环境已经提供 pybind11 时才生成 Python 模块。

从项目根目录运行独立快照：

```
cmake -S code/ch16 -B build/ch16-debug -DCMAKE_BUILD_TYPE=Debug
cmake --build build/ch16-debug -j
ctest --test-dir build/ch16-debug --output-on-failure
./build/ch16-debug/ch16_stability_report
./build/ch16-debug/ch16_boundary_report
```

首个稳定输出为：

```
stability-ok exact-0.1+0.2=false tolerant=true
naive=3.0 stable=4.0 oracle=4.0
tolerance-source input-abs=0.000e+00 operations=2
scale=3.000e-01 abs=1.332e-16 rel=4.441e-16
```

独立手算 oracle 是 $10^{16} + 1 - 10^{16} + 3 = 4$ 。程序若只打印“计算完成”或拿同一个函数生成 expected，不能证明算法正确。

16.2 浮点表示、比较与容差来源

double 通常按 IEEE 754 binary64 保存符号、指数和有限位有效数。0.1 与 0.2 在二进制中都是无限小数，字面量先舍入到相邻可表示值，再参与加法；因此 $0.1 + 0.2 == 0.3$ 为假并不表示 CPU 随机算错。== 仍适合整数、枚举，以及本来就要求位级完全相同的协议值；问题是把近似实数误当成精确编码。

numerics.cpp 的比较规则同时给绝对与相对下界：

```
const double difference = std::abs(lhs - rhs);
const double scale = std::max(std::abs(lhs), std::abs(rhs));
return difference <=
```

```
std::max(absolute_tolerance, relative_tolerance * scale);
```

绝对容差保护接近零的值；相对容差随量级缩放。可运行的 `tolerance_from_budget` 接收输入绝对误差、预计舍入操作数和结果尺度。本例输入误差为 0、加法含两次舍入、尺度为 0.3，按 `double epsilon` 导出上面的绝对与相对界限。报价精度或参考实现已有误差时，应计入 `input-abs`；容差过大仍会吞掉真实回归。

Python 的 `math.isclose` 也使用相对与绝对容差，但默认值、NaN 规则和调用点不会自动成为 C++ 合约。失败可由直接断言 `simple_return(100, 102.5) == 0.025` 触发；诊断是值只在末位不同，根因是两次舍入，修复是给出业务来源的容差并保留独立 `expected`。立即练习：只把 `input-abs` 从 0 改为 10^{-4} ；输出 `absolute` 应约为 10^{-4} ，并解释这代表输入精度变化而非“为了过测试”。

16.3 求和顺序、补偿项与在线方差

浮点加法不满足实数意义下的结合律。顺序求和先执行 $10^{16} + 1$ 时，较小的 1 会被舍掉；之后再减 10^{16} 已无法恢复。`std::accumulate` 能表达归约，却仍按给定顺序反复做同一种加法，不能自动补偿已丢低位。

本章用 Neumaier 补偿求和：`total` 保存主和，`correction` 收集每次加法中未进入主和的低位。当新数绝对值更大时，误差表达式的方向也随之交换。完整可运行入口如下；核心实现来自同一静态库：

Listing 16.1: 朴素求和、补偿求和与独立 oracle

```
1  #include "quant/ch16/numerics.hpp"
2
3  #include <array>
4  #include <iomanip>
5  #include <iostream>
6
7  int main() {
8      const bool exact = 0.1 + 0.2 == 0.3;
9      const quant::ch16::TolerancePolicy tolerance_policy =
10         quant::ch16::tolerance_from_budget(0.0, 2, 0.3);
11      const bool tolerant =
12         quant::ch16::almost_equal(0.1 + 0.2, 0.3,
13                                   tolerance_policy.absolute,
14                                   tolerance_policy.relative);
15      constexpr std::array values{1.0e16, 1.0, -1.0e16, 3.0};
16      const double naive = quant::ch16::naive_sum(values);
17      const double stable = quant::ch16::neumaier_sum(values);
18      constexpr double oracle = 4.0;
19      const bool valid = !exact && tolerant && naive == 3.0 && stable == oracle;
20
21      std::cout << "stability-ok exact-0.1+0.2=" << std::boolalpha << exact
22                << " tolerant=" << tolerant << std::fixed << std::setprecision(1)
23                << " naive=" << naive << " stable=" << stable
24                << " oracle=" << oracle << '\n'
25                << std::scientific << std::setprecision(3)
26                << "tolerance-source input-abs="
27                << tolerance_policy.input_absolute_error
28                << " operations=" << tolerance_policy.rounded_operations
29                << " scale=" << tolerance_policy.scale
30                << " abs=" << tolerance_policy.absolute
```

```

31         << " rel=" << tolerance_policy.relative << '\n';
32     return valid ? 0 : 2;
33 }

```

方差也不能机械套用 $E[X^2] - E[X]^2$ ：两项很大而差很小时会发生灾难性消减。`sample_moments` 使用 Welford 更新，每读一个值就更新 `count`、`mean` 与平方离差和，最后明确除以 $n - 1$ ，所以输出叫 `sample variance`；若业务要总体方差，则分母与名称必须同时改变。

Python 的 `math.fsum` 提供高精度求和工具，NumPy 的归约还可能按 `dtype`、轴和实现选择不同顺序；C++ 补偿算法不会自动等同于某个 NumPy 版本。失败触发是把输入顺序换成大数、小数、反向大数并观察朴素和偏离 `oracle`；诊断类别是 `cancellation/order sensitivity`，根因是有效位丢失，修复是改变算法、提高精度或重构问题，而非扩大测试容差。立即练习：只把数组顺序改为 `{1e16, -1e16, 1, 3}`；朴素和应变为 4，稳定和仍为 4，并解释“这次朴素正确”为何不能证明算法稳健。

16.4 金额、收益率与方差不是同一种数

金额通常有法定或业务最小单位。本章示例把已完成舍入的金额保存为 `int64_t cents`。转换函数只在明确入口把 `major units` 乘 100，并用 `std::round` 定义半数远离零。输入 10.125 可由二进制精确表示，得到 1013 分；真实 CSV 若含十进制文本，应优先按字符串规则解析或使用经批准的 `decimal` 库，不能先丢精度再声称精确。

收益率是比值，通常保留 `double`，并记录 `simple return` 还是 `log return`、价格复权和缺失值规则。方差的单位是收益率平方，还必须声明 `sample` 或 `population`。运行 `ch16_finance_report` 得到：

```

numeric-policy money=int64-cents rounded-cents=1013
return=0.025000 mean=0.006667 sample-variance=0.000633
overflow-rejected=true

```

Python 的整数也能无损保存 `cents`，`decimal.Decimal` 能表达十进制规则且整数不会溢出；C++ 固定宽度整数必须检查范围，标准库也没有金融 `decimal` 的直接替代品。失败可由向金额入口传 `NaN`、无穷或超出 `int64_t` 的值触发；稳定类别分别是 `invalid input` 与 `overflow`，根因是表示域不匹配，修复是边界拒绝而不是静默截断。立即练习：只把金额改为 -10.125，按本章口径应得到 -1013 分；再写一句说明退款是否允许为负属于领域规则而非浮点规则。

16.5 逐元素调用、批量复制与零拷贝借用

跨 Python/C++ 边界有固定成本：参数转换、动态分派、错误翻译和可能的 GIL 操作。把四个元素逐个调用 C++ 会付四次边界成本；一次批量复制只跨一次边界，但分配并复制；零拷贝借用同样只跨一次，却把布局与寿命前置条件交给接口。

`boundary_report.cpp` 让三条路径处理 `{0.25, -0.125, 0.5, 0.375}`，共同 `checksum` 为 1：

```

boundary-ok rows=4 scalar-calls=4 bulk-copies=1
zero-copy-borrows=1 checksum=1.000000

```

调用次数和复制次数只描述成本模型，不是耗时结论。小数组可能由转换成本主导，大数组复制后反而可能因连续布局更快；应像第 14 章那样固定结果、规模、预热和样本再测量。

Python/NumPy 的向量化把循环移到扩展内部，但“写成数组表达式”不保证零拷贝，也不保证 `dtype` 未转换。失败触发是对每个行情点调用一次绑定后只测 C++ 循环；诊断是 `profiler` 显示大量 `binding/call overhead`，根因是边界粒度太细，修复是设计批量领域操作。立即练习：只把输入行数从 4 改为 8；三条 `checksum` 仍须相同，`scalar-calls` 应为 8，而 `bulk-copies` 与 `zero-copy-borrows` 仍各为 1。

16.6 dtype、连续性与缓冲所有权

NumPy 数组不仅是一个地址。dtype 决定一个元素如何解释；shape 给每维长度；stride 给同一维相邻元素的字节距离。切片 `values[::2]` 可以共享原数组却不连续。若 C++ 把 float32 地址强转成 `double*`，每八字节会拼接两个四字节元素；若忽略 stride，读取的也不是逻辑相邻值。

本章的 BatchView 要求一维 float64 且 stride 等于 `sizeof(double)`，不允许隐式 forcecast。错误输入得到稳定边界：

```
dtype-error expected=float64 actual=float32
layout-error expected=c-contiguous stride-bytes=16
ownership-error temporary-view-rejected owner-missing=true
```

严格拒绝适合教学和延迟敏感路径；另一种合法 API 可以明确命名 `allow_copy`，由它转换 dtype 与布局并报告发生了复制。两种语义不能藏在同一个“看起来零拷贝”的函数里。

Python 视图常通过 base 保持源对象存活；C++ span 只是指针加长度，不拥有元素。故意失败的 `temporary_buffer_view.cpp` 返回局部 vector 的 span，离开函数即悬空；ASan 稳定报告 `heap-use-after-free`。pybind11 返回借用数组时必须把原 Python 对象设为 base，返回新 C++ 缓冲时则用 capsule 或拥有型数组托管释放。立即练习：只把步长 16 改回 8，layout 错误应消失；dtype 仍为 float32 时必须继续拒绝，说明两个前置条件彼此独立。

16.7 GIL 与可选 pybind11 薄绑定

Python 调入扩展时当前线程持有 GIL。长时间纯 C++ 循环若一直持有它，会阻止其他 Python 线程执行 Python 字节码；但释放 GIL 只移除解释器锁，不会替 C++ 共享状态建立 mutex 或 happens-before。

`bindings/module.cpp` 先在持有 GIL 时取得 `buffer_info`，随后只在不调用 Python API 的核心求和或填充循环周围创建 `py::gil_scoped_release`；作用域结束自动重新取得 GIL，之后才构造 Python 异常或返回对象。若循环中需要回调 Python、调整引用计数或创建数组，就不能在释放区间执行这些操作。

可选构建遵守依赖边界：`find_package(pybind11 CONFIG QUIET)` 找不到依赖时只打印状态并继续构建核心；找到时生成 `quant_ch16`，其 `sum_returns` 仍调用同一 `sum_batch`。Python 验收脚本检查 float64 连续输入、错误 dtype、切片 stride，以及删除原名称后借用视图仍由 base 保活。

Python 的线程同样受 GIL 约束，但 NumPy 或其他扩展是否释放、释放多久属于各函数实现契约。失败触发是两个 Python 线程调用长 C++ 循环却完全串行；profiler 显示第二线程没有 Python 进展，根因通常是持锁计算。修复是在纯 C++ 且对象寿命已固定的最小区间释放，并用 ThreadSanitizer/行为测试另证 C++ 线程安全。立即练习：只把 `gil_scoped_release` 移出数据指针取得之前，指出为什么此时访问 `py::array` 元数据不再安全；正确答案不是“多加一个锁”，而是缩回释放作用域。

无需 pybind11 的 `gil_probe.cpp` 还用 CPython 扩展 API 建立可运行诊断：新 C++ 线程必须取得 GIL 后才设置原子进度。调用方持有 GIL 等待时进度稳定为 false；仅在等待区间释放后才为 true，测试输出 `gil-diagnostic held-progress=false released-progress=true`。探针以 deadline 防止故障无限挂起，但不靠 sleep 猜测调度。

16.8 输出任务与自测

输出任务：提交 `code/ch16` 的 `clean-build` 命令、容差预算与三组精确输出，以及一页边界说明。说明须给出金额舍入口径、收益与方差定义；比较逐元素、批量复制和零拷贝；列出 dtype、stride、所有者与 GIL 的前置条件。附 ASan 的 `heap-use-after-free` 与 GIL probe 两类稳定证据。环境有 pybind11 时再提交可选模块输出；没有时提交“dependency-free core only”配置证据。

自测：

1. 为什么 `0.1 + 0.2 == 0.3` 失败，而整数 `cents` 可以精确比较？

2. 绝对容差与相对容差分别保护什么量级，容差应从哪里来？
3. Neumaier 补偿项保存了什么，为什么不能只把容差调大？
4. Welford 方差的 count、mean 与平方离差和如何逐步变化？
5. 金额、收益率和样本方差为何不应共用一种表示口径？
6. 批量复制与零拷贝各把什么成本和责任交给调用者？
7. dtype、shape、stride 与 owner 各阻止哪类错误？
8. 释放 GIL 能解决什么，又绝不能替代什么？

问题 16.1 面试检查点 被问“如何把 Python 因子计算迁到 C++”时，先给独立数值 oracle 与误差口径；再说明批量 API、dtype/连续性拒绝策略、复制或借用的所有权；最后才讨论 GIL 释放和 benchmark。主动说明零拷贝增加寿命约束，释放 GIL 也不自动带来 C++ 线程安全。

16.9 本章小结

数值可靠性来自表示、算法和验收口径；互操作性能来自足够粗的边界。先以独立 oracle 验证无依赖核心，再把 dtype、连续性、所有者和 GIL 写进薄绑定协议。

附录 A 工具链速查

A.1 本书验证环境

本项目在 Windows 上使用 MiKTeX 26.1、XeLaTeX、latexmk、CMake 3.29、GCC 13.2（MinGW-w64）验证。书稿主线代码以 Linux GCC/Clang 为目标，Windows 命令用于本机复现。

A.2 构建 C++

```
cmake -S . -B build-mingw -G "MinGW Makefiles" \  
      -DCMAKE_BUILD_TYPE=Release  
cmake --build build-mingw  
ctest --test-dir build-mingw --output-on-failure
```

Linux/Ninja 常用命令：

```
cmake -S . -B build -G Ninja -DCMAKE_BUILD_TYPE=Debug  
cmake --build build  
ctest --test-dir build --output-on-failure
```

诊断配置可加入 `-fsanitize=address,undefined -fno-omit-frame-pointer`。ThreadSanitizer 通常单独构建，不与 AddressSanitizer 混用。

A.3 构建 PDF

```
latexmk -xelatex -interaction=nonstopmode -halt-on-error \  
      -outdir=build main.tex
```

若 MiKTeX 为最小安装，可用新版 CLI 显式保证依赖：

```
miktex packages require elegantbook csquotes comment esint mwe \  
      enumitem caption footmisc multirow fancyvrb makecell lipsum \  
      hologo titlesec appendix biblatex pgf apptools bbding tcolorbox \  
      manfnt fancyhdr tocloft siunitx
```

本项目复制的 ElegantBook 4.7 在 MiKTeX 26.1 上遇到 `adorn` 的缺失间接依赖；本地类文件仅把问题集标题的两个装饰符替换为模板已加载的 `pifont` 字形。原模板目录未修改。

A.4 日志检查

```
rg -n "Undefined|undefined references|Missing character|Fatal error" \  
      build/main.log  
rg -n "Overfull|Underfull" build/main.log
```

未定义引用、缺字和致命错误必须修复。Overfull 需要逐条检查；Underfull 常是可接受的排版松弛，但不应忽略大段异常。

附录 B 术语表

术语	本书中的含义
ABI	二进制接口约定，包括调用、布局与符号兼容。
AoS / SoA	结构数组与数组结构，两种字段布局方式。
backpressure	下游跟不上时，系统对生产者施加的限速、阻塞或丢弃策略。
benchmark	在定义好的工作负载和环境测量性能的实验。
concept	C++20 对模板参数能力的编译期约束。
data race	未同步的并发冲突访问，至少一个写入；在 C++ 中属于 UB。
event	已验证、带时间和代码的市场事实。
fill	撮合后形成的成交事实，而不是订单意图。
happens-before	C++ 内存模型中保证效果可见与排序的关系。
iterator invalidation	容器操作使既有迭代器、引用或指针不再有效。
latency tail	延迟分布高百分位，如 p_{99} 与 $p_{99.9}$ 。
market event	规范化行情事件，包含时间、代码、价格与数量。
move	从可被转移资源的对象构造/赋值； <code>std::move</code> 仅转换值类别。
order intent	策略产生的买卖意图，尚不是成交。
ownership	谁负责保持对象存活并最终释放其资源。
PortfolioSnapshot	组合通过公开接口暴露的只读持仓、现金与权益值。
profiling	定位程序时间、分配或硬件事件分布的测量。
RAII	将资源取得与对象初始化绑定、释放与析构绑定。
seam	通过公开接口观察行为的测试边界。
slippage	决策或参考价格与实际模拟/成交价格之间的差异。
tracer bullet	从输入到输出贯通、可独立验证的最小纵向切片。
UB	未定义行为；语言标准不再约束程序结果。
value semantics	对象复制后成为具有相同含义的独立值。
view	通常不拥有元素、惰性描述遍历方式的范围适配器。
zero-cost abstraction	不使用不付费，使用抽象通常不劣于对应手写底层机制的设计原则。

附录 C 练习答案与思路

本附录给出检查点而非唯一实现。编码题应以可编译代码、行为测试和复杂度说明为最终答案。

C.1 第 1 章

自测答案

1. `#include` 在预处理阶段处理，不是以分号结束的 C++ 语句；它让当前翻译单元看见 `iostream` 中的声明。
2. `main` 是程序入口名，空圆括号表示本版本不接收参数，花括号包围函数体；状态 0 表示成功，非零表示失败。
3. `std::` 指定标准库命名空间，`<<` 把右侧文本送入输出流，双引号界定字符串字面量，`\n` 换行，分号结束语句。
4. 流水线是预处理、编译、链接和运行；`-c` 在目标文件处停止，因此能证明编译通过，却不能证明链接成功。
5. 缺分号是编译失败；只有声明没有定义是链接失败；打印启动错误并返回 2 是运行期失败。修复分别恢复语法、加入定义、补齐启动条件。
6. CMake 用 `-S` 指源目录、`-B` 指构建目录。删除旧构建树后从不存在的 `build/` 成功配置、构建和运行，才是干净构建证据。
7. Windows 可执行文件带 `.exe`，路径分隔和编译器警告选项不同，多配置生成器还包含配置目录；预处理、编译、链接、运行的阶段模型不变。

编码任务反馈

1. 完整文件见 `code/ch01/first_program.cpp` 与 `code/ch01/CMakeLists.txt`。直接 GCC 和干净 CMake 构建都应输出 `quant-cpp-ready` 并返回 0。
2. 三个失败源依次是 `missing_semicolon.cpp`、`missing_definition.cpp` 与 `startup_failure.cpp`，均位于 `code/ch01/failures/`；验收类别分别是编译期缺分号、链接期缺定义、运行期消息加状态 2。
3. `//` 到行末结束，`/* ... */` 可以跨行但不能嵌套；两种注释都不应改变输出。删除分号应使编译失败。恢复后重跑 `ctest -R ch01_`，五个第 1 章行为测试应全部通过。

C.2 第 2 章

1. 初始化在对象创建时给出初始状态；赋值修改已存在对象。局部 `int x`；没有可靠初值，`int x{}`；值初始化为零。字符字面量用单引号，字符串字面量用双引号；可运行演示见 `code/ch02/objects_and_literals.cpp`。完整筛选答案见 `code/ch02/market_filter.cpp`，给定五行输入应输出 `selected=2 rejected=3 buy_notional=1035.75 average=517.875`。
2. `auto` 仍在编译期推导出固定类型；当位宽或领域含义重要时显式写出类型。`5 / 2` 先做整数除法得到 2，`5.0 / 2` 得到 2.5；平均值分母可写成 `static_cast<double>(selected)`。
3. `a && b` 在 `a` 为假时不求值 `b`。`switch` 漏写 `break` 会贯穿后续标签；`default` 接住未识别方向。已知迭代次数用 `for`，由状态决定终止用 `while`。
4. 风险预算题的完整答案见 `code/ch02/exercises/risk_budget_solution.cpp`；3200 按每笔 750 扣减四次后剩 200。计数循环的独立答案见 `code/ch02/for_scope.cpp`。若循环体不扣减预算，条件永不改变，会形成无限循环。

- code/ch02/failures/narrowing.cpp 应在编译阶段得到“列表初始化拒绝窄化转换”类别。可把接收对象改为 double；若业务确实需要整数，先定义截断、四舍五入或拒绝策略，再显式转换。

C.3 第3章

自测答案

- 声明给出返回类型、名称和参数表并以分号结束；定义给出匹配接口与函数体；调用用实参创建参数对象并取得结果。定义的右花括号后不加函数声明式分号。
- 按值或引用参数在调用期间有效，局部对象在离开函数时销毁；返回值交回调用者。早返回在满足条件时立即结束本次调用，跳过后续语句。
- 按值允许修改独立参数而不影响调用者；const T& 只读借用；T& 可修改调用者；const T* 是可空的只读观察，检查不为 nullptr 后才能解引用。
- 重载靠参数数量或类型区分，并要求调用时存在唯一最佳候选。只改变返回类型无法总由实参确定选择，因此不能形成可调用的重载集合。
- 每个 .cpp 连同包含的声明形成独立翻译单元并编译为目标文件；链接器把目标文件与库组合为可执行文件。头文件自身不会自动提供实现。
- include guard 让同一头文件在一个翻译单元重复包含时只展开一次。项目头文件用双引号配合目标 include 路径；命名空间降低项目名称与其他库冲突的风险。
- 缺失定义与重复定义都在各翻译单元编译后由链接器报告；返回局部对象引用可由高警告级别在编译时指出，其后读取是未定义行为，不运行来验收结果。

编码任务反馈

- 完整多文件答案位于 code/ch03/include/quant/ch03/market_math.hpp、code/ch03/src/market_math.cpp 与 code/ch03/src/main.cpp；独立 code/ch03/CMakeLists.txt 应从空构建树得到 notional=4010.00 quantity=40 vwap=100.25。
- 完整可运行答案如下；输出应为 rate=0.0015 basis_points=15，并证明调用者值不变：

```

1  #include <iostream>
2
3  double basis_points(const double& rate) {
4      return rate * 10000.0;
5  }
6
7  int main() {
8      const double rate{0.0015};
9      const double converted{basis_points(rate)};
10
11     std::cout << "rate=" << rate << " basis_points=" << converted << '\n';
12     return 0;
13 }
```

- 三个故障源码位于 code/ch03/failures/。缺失与重复定义分别匹配链接器的 undefined 与 multiple-definition 类别；悬空引用匹配返回局部对象引用的编译诊断。恢复方式分别是补上唯一定义、删除额外定义、改为按值返回或证明被引用对象寿命足够长。

C.4 第4章

自测答案

1. `string` 保存拥有的字符序列; `vector<double>` 保存动态数量的同类型连续元素; `array<string, 3>` 的长度 3 属于类型。三者都不能像 Python list 一样随意混放无关类型。
2. `size` 是已存在元素数, `capacity` 是当前分配可容纳数。`reserve` 只提高容量下界, `resize` 改变大小并构造或销毁元素。
3. 半开区间包含 `first` 而不包含 `last`; 空区间可由 `first == last` 表示, 因此尾后迭代器只用于比较, 不能解引用。
4. `vector` 重新分配会使全部旧位置失效; 插入或删除还可能使操作点及其后位置失效。可预留足够容量, 或修改后重新取得迭代器、引用和指针。
5. `lambda` 的 `[]` 是捕获列表, `()` 是参数表, `{}` 是函数体; 空捕获不读取周围局部对象。`[minimum]` 按值捕获阈值。
6. 操作正好是查找、计数、复制、排序、转换或归约时, 算法直接表达意图; 多状态、早停或逐行诊断常由普通循环更清楚。两者都必须满足范围、输出空间和迭代器有效性前置条件。
7. 打开失败属于文件边界; 列数与数值错误属于带行号的数据边界。三者均返回 2, 但诊断分别为 `cannot open`、`expected columns` 与 `invalid number`。
8. 三个平行 `vector` 的同一索引共同表示一行, 因此长度必须相等。第 5 章用一个行情记录类型把 `symbol`、`price`、`quantity` 组合成单个元素, 消除同步修改三列的脆弱约束。

编码任务反馈

CSV 读取与统计函数的完整实现位于 `code/ch04/src/csv_stats.cpp`; 它只接受本章定义的三列方言, 任何失败都会清空平行列并保留行号。该文件由 `ch04_csv_stats` 与行为测试共同编译, 读者应以源码而不是附录副本为准。

完整答案如下; 使用 `code/ch04/data/market_rows.csv` 与参数 `AAPL` 时, 应输出 `symbol=AAPL rows=2 quantity=15 notional=1505.00`:

```

1  #include <fstream>
2  #include <iomanip>
3  #include <iostream>
4  #include <string>
5  #include <vector>
6
7  #include "quant/ch04/csv_stats.hpp"
8
9  int main(const int argc, char* argv[]) {
10     if (argc != 3) {
11         std::cerr << "error=usage: ch04_symbol_summary_solution <market.csv> <symbol>\n";
12         return 2;
13     }
14
15     std::ifstream input{argv[1]};
16     if (!input) {
17         std::cerr << "error=cannot open input file\n";
18         return 2;
19     }

```

```

20
21     std::vector<std::string> symbols;
22     std::vector<double> prices;
23     std::vector<int> quantities;
24     std::string error;
25     if (!quant::ch04::read_market_csv(input, symbols, prices, quantities,
26                                     error)) {
27         std::cerr << "error=" << error << '\n';
28         return 2;
29     }
30
31     const std::string wanted{argv[2]};
32     std::size_t rows{0};
33     int quantity{0};
34     double notional{0.0};
35     for (std::size_t index{0}; index < symbols.size(); ++index) {
36         if (symbols[index] == wanted) {
37             ++rows;
38             quantity += quantities[index];
39             notional += prices[index] * quantities[index];
40         }
41     }
42
43     std::cout << std::fixed << std::setprecision(2) << "symbol=" << wanted
44               << " rows=" << rows << " quantity=" << quantity
45               << " notional=" << notional << '\n';
46 }

```

空代码、非法价格和错误列数的固定输入与预期错误分别位于 `code/ch04/data/` 和 `code/ch04/expected/`。故意失效迭代器只通过 ASan 验收 `heap-use-after-free` 类别，不读取未定义的数值结果。

C.5 第 5 章

自测答案

1. `struct` 默认公开，`class` 默认私有；两者都能声明访问标签、构造函数和成员函数，不是两套对象模型。选择应表达“透明记录”还是“维护不变量的抽象”。
2. 作用域枚举把枚举项保留在类型作用域并阻止静默整数转换，因此写 `Trend::up`。分支直接 `return` 或 `throw` 时不需 `break`；仍要继续执行函数时，`break` 防止贯穿下一标签。
3. 构造函数建立对象，没有单独返回值。成员初始化列表在进入函数体前构造成员；函数体随后检查字段组合，不满足时抛出且对象不交付。
4. `price()` `const` 的尾随 `const` 约束当前对象不被成员函数修改；`const std::string&` 约束调用者通过返回引用只读观察字符串，并要求借用不超过对象寿命。
5. `this` 是当前对象的指针；`this->prices_` 通过指针访问当前对象成员，`quote.price()` 用点号从已有对象调用成员。无歧义时成员函数内可省略 `this->`。
6. `MarketQuote` 的空代码、非法价格和非正数量会破坏所有后续统计，适合在构造边界拒绝。`MarketSummary` 是已经计算完成、无需继续维护跨字段状态的透明结果，适合公开字段。

- 异常传播会按逆序销毁已完成构造的自动对象；未完成构造的目标对象不存在。CLI 有能力决定 `stderr` 与退出码，因此在那里记录一次，内层只补充字段和行号上下文。
- 读取 `quantity_` 违反访问控制，在编译期得到 `private/inaccessible` 类别；负价格语法有效但违反领域不变量，在运行期抛出并转成状态 2。恢复分别是调用公开观察函数、修正输入或在构造边界拒绝。

编码任务反馈

行情对象和分析器的完整实现见 `code/ch05/src/market.cpp`；`add` 拒绝代码不一致，`summary` 拒绝空序列。

CSV 读取层见 `code/ch05/src/csv_reader.cpp`，它保留表头、列数、数字字段和构造异常的行号上下文。两份完整源码均参加 `ch05_market_analyzer` 构建与精确 `stderr` 测试，避免在附录维护第二份实现。

价格区问题的完整答案如下，输出应为 `symbol=AAPL range=1.00 trend=up`：

```

1 #include <iomanip>
2 #include <iostream>
3
4 #include "quant/ch05/market.hpp"
5
6 int main() {
7     quant::ch05::MarketAnalyzer analyzer{"AAPL"};
8     analyzer.add(quant::ch05::MarketQuote{"AAPL", 100.0, 10});
9     analyzer.add(quant::ch05::MarketQuote{"AAPL", 101.0, 5});
10
11     const auto result{analyzer.summary()};
12     std::cout << std::fixed << std::setprecision(2)
13               << "symbol=" << result.symbol
14               << " range=" << result.high - result.low
15               << " trend=" << quant::ch05::trend_name(result.trend) << '\n';
16 }
```

独立快照入口为 `code/ch05/CMakeLists.txt`。私有访问实验只验收编译器的 `private/inaccessible` 类别；非法 CSV 固定输入和精确 `stderr` 位于 `code/ch05/data/` 与 `code/ch05/expected/`。

C.6 第 6 章

自测答案

- 构造成功后对象生命周期开始，析构完成后结束；存储中的旧比特不会让已析构对象重新存在。自动对象按逆构造顺序销毁。
- `std::move` 只把表达式转换为可选择移动操作的值类别。移动后的源对象仍可析构、可赋值，但除非类型契约另有保证，不能假定保留原值。
- 引用和裸指针通常只借用；`unique_ptr` 是可转移的唯一所有者；`shared_ptr` 让多个参与者共同决定寿命。直接值仍是最简单的默认所有权。
- C 数组退化为指针后不再携带长度。`new[]` 建立的数组必须由 `delete[]` 释放；与标量释放形式混用是未定义行为。
- RAII 让栈展开和所有提前离开路径都触发析构；Rule of Zero 把释放交给可靠成员，避免外层类型手写一组容易不一致的复制、移动和析构操作。
- `weak_ptr` 不增加强计数，因此可打破共享环；`shared_ptr` 只协调控制块寿命，不会自动同步被管理对象的字段。

7. ASan 报告同时给出分配、释放和非法访问位置。按这三处重建生命周期图，再恢复单一明确的所有者和不越界的借用期。

编码练习反馈

1. 复制/移动轨迹的完整答案及精确输出见 `code/ch06/lifetime_trace.cpp` 与 `code/ch06/expected/lifetime_trace.txt`。
2. 引用、可空观察、C 数组和手动释放的可运行答案见 `code/ch06/borrowing_and_legacy.cpp`。
3. RAII 文件答案见 `code/ch06/raii_file.cpp`：输出流析构后输入流读到两行，输入流析构后文件才删除。
4. 所有权输出任务见 `code/ch06/ownership_choices.cpp`；答案必须与对应 `expected` 文件逐字一致。
5. 故意失败源码位于目录 `code/ch06/failures/` 的 `use_after_free.cpp`。ASan 应非零退出并报告 `heap-use-after-free`，不验收未定义的输出值。

C.7 第 7 章

1. 成交时间线按到达顺序追加和扫描，`vector` 的连续拥有存储适合批处理；仓位按代码查询，`map` 把键查找写进接口。若改用其他容器，必须给出访问与失效依据。
2. `positions[key]` 会在缺失时插入值初始化的整数，适合累计；`positions.at(key)` 只查询，缺失时抛出异常。范围循环按键顺序遍历 `map`，本例得到 AAPL、MSFT。
3. 排序会改变顺序，因此复制后排序价格；`find_if` 必须观察原时间线，才能回答“首笔卖出”而不是“排序后第一笔卖出”。
4. `view` 创建后才把 99 改成 110，最终仍输出 220，证明筛选与转换在迭代时求值。`view` 按值拥有 `adaptor` 状态，却借用 `prices` 的元素与存储。
5. 返回 `view` 时局部 `vector` 已析构，`view` 内的引用悬空。可让源容器由调用者拥有、在原作用域消费，或在返回前复制到拥有元素的容器。
6. `back_inserter` 把赋值转换为 `push_back`，使 `ranges::copy` 能向初始为空的 `vector` 追加结果，而不要求预先创建可写元素。
7. 初值决定累加器类型。0.0 让名义金额按 `double` 累加；写 0 会让累加器成为 `int`，每一步都可能截断小数。
8. 浮点加法不满足数学上的结合律，舍入发生在每一步。本章只承诺固定程序与输入下的测试结果，不承诺任意重排逐位相同；稳定求和与容差来源见第 16 章。

核心编码任务的完整答案如下。它先把惰性筛选结果物化，再排序拥有元素的副本，最后用 `take(2)` 限制输出：

Listing C.1: 过滤、降序排序并输出前两笔 AAPL 成交

```

1  #include <algorithm>
2  #include <iomanip>
3  #include <iostream>
4  #include <iterator>
5  #include <ranges>
6  #include <vector>
7
8  #include "quant/ch07/trade.hpp"
9
10 int main() {
11     const std::vector<quant::ch07::Trade> trades{
12         {"AAPL", quant::ch07::Side::buy, 10, 100.0},
13         {"MSFT", quant::ch07::Side::buy, 4, 200.0},

```

```

14     {"AAPL", quant::ch07::Side::sell, 5, 102.0},
15     {"AAPL", quant::ch07::Side::buy, 10, 101.0},
16 };
17
18 auto apple = trades |
19     std::views::filter([](const quant::ch07::Trade& trade) {
20         return trade.symbol == "AAPL";
21     });
22 std::vector<quant::ch07::Trade> ranked;
23 std::ranges::copy(apple, std::back_inserter(ranked));
24 std::ranges::sort(
25     ranked, [](const quant::ch07::Trade& left,
26               const quant::ch07::Trade& right) {
27         return left.price > right.price;
28     });
29
30 std::cout << std::fixed << std::setprecision(2) << "top=";
31 bool first{true};
32 for (const auto& trade : ranked | std::views::take(2)) {
33     if (!first) {
34         std::cout << ',';
35     }
36     std::cout << trade.symbol << '@' << trade.price;
37     first = false;
38 }
39 std::cout << '\n';
40 }

```

项目答案位于 `code/ch07/src/pipeline.cpp`，独立构建入口为 `code/ch07/CMakeLists.txt`。非法生命周期实验只通过 ASan 的非零退出与栈访问类别，不验收未定义输出。

C.8 第 8 章

1. 接口应从“低价空仓下单、代码一致且流动性足够才成交、组合应用成交”等用例产生；没有用例支持任意改价，因此不提供 `setter`。
2. `Order` 拥有自己的 `string` 和数量，复制得到独立对象；`const Order&` 只借用调用者对象，不复制也不延长其生命。
3. 数量必须为正；行情和成交价格还必须为正且有限，因为 NaN 与零比较可能绕过仅有的大小检查。
4. 不下单是正常缺席，使用空 `optional`；数量为零是非法订单，不能让一个哨兵对象同时表达两种语义。
5. 代码不匹配或流动性不足是本次无法撮合，返回空 `optional`；构造非法 `Fill` 则违反领域前置条件。
6. `apply` 在写现金和仓位前完成超卖与现金检查；测试比较拒绝前后公开 `snapshot`，而不读取私有 `map`。
7. 默认用值成员和组合；只有启动后必须通过同一接口选择实现时使用虚函数；编译期泛型留到第 9 章。
8. `virtual` 开启动态分派，`=0` 声明纯虚接口，`override` 让编译器核对签名，`final` 禁止继续派生或覆盖。

99 元买 25 股后现金为 7525；101 元卖 10 股后现金为 8535、持仓为 15，以 101 元估值的权益为 10050。完整答案如下，独立构建入口为 `code/ch08/CMakeLists.txt`。

Listing C.2: 第 8 章卖出部分持仓练习答案

```

1 #include <iomanip>

```

```

2  #include <iostream>
3
4  #include "quant/ch08/execution.hpp"
5
6  int main() {
7      quant::ch08::Portfolio portfolio{10'000.0};
8      portfolio.apply(
9          quant::ch08::Fill{"AAPL", quant::ch08::Side::buy, 25, 99.0});
10     portfolio.apply(
11         quant::ch08::Fill{"AAPL", quant::ch08::Side::sell, 10, 101.0});
12
13     const auto result = portfolio.snapshot("AAPL", 101.0);
14     std::cout << std::fixed << std::setprecision(2)
15         << "quantity=" << result.quantity << " cash=" << result.cash
16         << " equity=" << result.equity << '\n';
17 }

```

C.9 第 9 章

1. 只有一种稳定类型、需要清楚诊断或实现隐藏时，普通函数更直接；需要对多种编译期类型复用同一算法时再用模板。
2. 类型参数从 `template` 声明开始作用于随后的声明或定义；本例由函数实参的静态类型推导 `Strategy`。
3. 实例化把模板与一组实际类型结合成函数或类特化；定义通常必须可见，编译器才能生成并检查代码。
4. 尖括号中的 `ThresholdStrategy` 决定运行器成员的具体类型；不同模板实参形成不同类类型。
5. `requires` 的局部参数只用于形成表达式，不创建运行时对象。
6. `same_as` 检查调用返回类型恰为 `optional<Order>`；它不能证明价格阈值、数量或风控语义正确。
7. 无约束版本在实例化函数体后报告成员不存在；受约束版本先报告 `constraints not satisfied`，并定位失败要求。
8. 编译期固定类型适合静态方案；运行期配置或插件适合虚接口。模板可能增加编译时间和代码体积，虚接口承担间接调用与 ABI 约束。

编码任务完整答案如下：

Listing C.3: 第 9 章最大持仓策略练习答案

```

1  #include <cstdint>
2  #include <iostream>
3  #include <optional>
4  #include <stdexcept>
5  #include <string>
6  #include <vector>
7
8  #include "quant/ch09/strategy_runner.hpp"
9
10 class MaximumPositionStrategy final {
11 public:
12     MaximumPositionStrategy(std::int64_t maximum, std::int64_t order_quantity)
13         : maximum_(maximum), order_quantity_(order_quantity) {
14         if (maximum_ < 0 || order_quantity_ <= 0 || order_quantity_ > maximum_) {

```

```

15     throw std::invalid_argument{"invalid maximum-position configuration"};
16 }
17 }
18
19 std::optional<quant::ch09::Order> on_event(
20     const quant::ch09::MarketEvent& event,
21     const quant::ch09::PortfolioSnapshot& portfolio) const {
22     if (event.symbol() != portfolio.symbol()) {
23         throw std::logic_error{"event and portfolio symbols must match"};
24     }
25     if (portfolio.quantity < 0) {
26         throw std::logic_error{"portfolio quantity must not be negative"};
27     }
28     if (portfolio.quantity > maximum_ - order_quantity_) {
29         return std::nullopt;
30     }
31     return quant::ch09::Order{event.symbol(), quant::ch09::Side::buy,
32                               order_quantity_};
33 }
34
35 private:
36     std::int64_t maximum_;
37     std::int64_t order_quantity_;
38 };
39
40 std::string order_label(std::size_t order_count) {
41     if (order_count == 1) {
42         return "buy";
43     }
44     return "none";
45 }
46
47 int main() {
48     const quant::ch09::StrategyRunner<MaximumPositionStrategy> runner{
49         MaximumPositionStrategy{50, 25}};
50     const std::vector<quant::ch09::MarketEvent> events{
51         {"AAPL", 99.0, 1'000}};
52     const quant::ch09::PortfolioSnapshot flat{"AAPL", 0, 10'000.0, 10'000.0};
53     const quant::ch09::PortfolioSnapshot full{"AAPL", 50, 5'000.0, 10'000.0};
54     const quant::ch09::PortfolioSnapshot wrong_symbol{
55         "MSFT", 0, 10'000.0, 10'000.0};
56     bool mismatch_rejected = false;
57     try {
58         runner.count_orders(events, wrong_symbol);
59     } catch (const std::logic_error&) {
60         mismatch_rejected = true;
61     }
62     if (!mismatch_rejected) {
63         return 1;

```

```

64     }
65     std::cout << "flat=" << order_label(runner.count_orders(events, flat))
66               << " full=" << order_label(runner.count_orders(events, full))
67               << '\n';
68 }

```

C.10 第 10 章

1. optional 无值是调用契约允许的正常结果，没有失败原因；ParseError 表示输入不满足契约，必须携带可处置上下文。
 2. variant 任一时刻只持有一个备选，因此成功值与错误不能同时存在；先用 holds_alternative 检查，再用 get 读取。
 3. assert 可在定义 NDEBUG 时移除，外部输入在 Release 中仍可能错误，所以必须使用显式结果或异常。
 4. 抛出后沿栈寻找匹配 catch，途中局部对象按逆序析构；按 const 引用捕获避免切片和复制，并保留 what()。
 5. 基本保证维持可析构与不变量；强保证承诺失败时公开状态不变；不抛保证承诺函数不会抛出。
 6. candidate 隔离所有可能失败的更新，成功后通过 noexcept swap 一次提交；代价是复制时间与额外内存。
 7. 日志解释具体事件，指标聚合计数或分布，追踪用关联 ID 串起跨组件阶段。
 8. 底层只返回或传播错误；掌握跳过、重试或终止策略的边界记录一次，避免 catch-and-rethrow 重复日志。
- 编码任务完整答案如下：

Listing C.4: 第 10 章错误注入与强保证练习答案

```

1  #include <iostream>
2  #include <stdexcept>
3
4  #include "quant/ch10/portfolio.hpp"
5
6  int main() {
7      quant::ch10::Portfolio portfolio{10'000.0};
8      portfolio.apply_batch(
9          {quant::ch10::Fill{"AAPL", quant::ch10::Side::buy, 10, 100.0}});
10     const auto before = portfolio.snapshot("AAPL", 100.0);
11
12     bool injected = false;
13     try {
14         portfolio.apply_batch(
15             {quant::ch10::Fill{"AAPL", quant::ch10::Side::buy, 5, 100.0},
16              quant::ch10::Fill{"AAPL", quant::ch10::Side::sell, 100, 100.0}});
17     } catch (const std::logic_error&) {
18         injected = true;
19     }
20
21     const auto after = portfolio.snapshot("AAPL", 100.0);
22     const bool unchanged = after.quantity == before.quantity &&
23                           after.cash == before.cash &&
24                           after.equity == before.equity;
25     if (!injected || !unchanged) {
26         return 1;

```

```

27     }
28     std::cout << "injected=1 rolled_back=1 unchanged=1\n";
29 }

```

C.11 第 11 章

1. 配置读取项目并检测工具链，生成写出底层构建文件，构建才调用编译器和链接器，CTest 运行已登记且已构建的测试。
2. 旧二进制和缓存可能来自旧源码或旧选项；只有全新构建树能证明当前提交的配置路径完整。
3. 公共头文件目录同时供库与消费者使用，故为 PUBLIC；实现宏只供库自身使用，故为 PRIVATE。
4. INTERFACE 目标没有编译产物，但其 usage requirements 沿链接边进入消费者的编译与链接命令。
5. add_test 只登记运行命令；测试程序仍须由 add_executable 或其他目标创建。
6. 头文件不可见通常在编译期暴露；声明可见但定义未链接通常在链接期暴露。
7. Debug 支持逐步诊断，Sanitized 提供内存与 UB 证据，Release 验证优化配置；独立目录避免缓存串味。
8. 单配置生成器在配置时设置 CMAKE_BUILD_TYPE；多配置生成器在 build/ctest 时使用 --config 或 -C，目标图不变。

编码任务的完整源码如下；在 code/ch11/CMakeLists.txt 中以 add_executable 创建目标，再 PRIVATE 链接 ch11_market，即可继承公共头文件和 C++20 要求：

Listing C.5: 第 11 章新增消费者目标练习答案

```

1  #include <iomanip>
2  #include <iostream>
3  #include <vector>
4
5  #include "quant/ch11/market_summary.hpp"
6
7  int main() {
8      const std::vector<quant::ch11::MarketRow> rows{{99.0, 1}, {101.0, 1}};
9      const auto summary = quant::ch11::summarize(rows);
10     std::cout << std::fixed << std::setprecision(2)
11               << "exercise-notional=" << summary.notional << '\n';
12 }

```

C.12 第 12 章

1. 公开行为允许内部结构重构；私有成员和调用顺序会让无行为变化的重构也破坏测试。
2. 红灯须因需求尚未满足而失败；期望值来自独立手算，避免测试复制同一个错误公式。
3. 定义 NDEBUG 后标准 assert 可被删除，连条件中的副作用也不执行；测试应使用框架或显式检查。
4. 参数为 1000、100、2、1，notional 为 200，返回值却为 801，故差异首次出现在返回表达式的费用符号。
5. 警告分析编译实体，静态分析探索可能路径，sanitizer 必须插桩并实际运行到缺陷路径。
6. 错误访问栈指出症状位置，释放栈指出生命周期结束位置，分配栈指出资源来源。
7. 溢出在窄类型运算完成时已经发生；事后转换只会扩展无效结果，必须在运算前检查或先转宽类型。
8. 至少保存失败输入、工具链与配置、复现命令、期望/实际与退出码、稳定诊断类别、根因、最小修复和回归结果。

编码任务答案保留显式非零失败路径，所以 Debug 与 Release 都执行相同检查：

Listing C.6: 第 12 章逻辑错误诊断与修复答案

```

1 #include <iomanip>
2 #include <iostream>
3
4 double cash_after_buy(double cash, double price, int quantity, double fee) {
5     const double notional = price * static_cast<double>(quantity);
6     return cash - notional - fee;
7 }
8
9 int main() {
10     const double observed = cash_after_buy(1'000.0, 100.0, 2, 1.0);
11     if (observed != 799.0) {
12         std::cerr << "cash update is still wrong\n";
13         return 1;
14     }
15     std::cout << std::fixed << std::setprecision(0)
16               << "fixed-cash=" << observed << '\n';
17 }

```

C.13 第 13 章

1. `sizeof` 是对象数组步长, `alignof` 是起始地址对齐要求, `offsetof` 是 `standard-layout` 成员偏移; 具体数字通常属于 ABI/实现观察。
2. 成员之间和末尾可有填充; 尾部填充让数组中下一个对象仍满足对齐要求。
3. AoS 适合一起读取完整记录, SoA 适合连续扫描少数列; 访问模式变化时选择也可能变化。
4. `checksum` 不同表示业务语义已不同, 此时耗时差异不能归因于布局。
5. `reserve` 改 `capacity` 而不构造元素, `resize` 改 `size` 并构造/销毁元素; 超过容量仍会使地址失效。
6. 预分配可减少搬迁, 却可能因不可信或过大上界增加峰值内存、失败和工作集压力。
7. `memory resource` 必须晚于 PMR 容器及其使用者销毁; 需要逃逸的数据应复制到更长寿命所有者。
8. `alignas(64)` 只保证源码请求的对齐, 64 是实验假设, 不是 C++ 对硬件缓存行的承诺。

核心输出任务的列式行情扫描如下。阈值等于 100 的行情也应入选, 因此比较使用 `>=`; 独立手算为 $100 \times 3 + 101.5 \times 4 + 102 \times 1 = 808$ 。

Listing C.7: 第 13 章证据门控的行情扫描答案

```

1 #include <cstdint>
2 #include <iomanip>
3 #include <iostream>
4 #include <vector>
5
6 int main() {
7     const std::vector<double> prices{99.0, 100.0, 101.5, 98.0, 102.0};
8     const std::vector<std::int64_t> quantities{2, 3, 4, 5, 1};
9     constexpr double threshold = 100.0;
10
11     std::size_t selected = 0;
12     double notional = 0.0;
13     for (std::size_t index = 0; index < prices.size(); ++index) {

```



```

14     if (prices[index] >= threshold) {
15         ++selected;
16         notional += prices[index] * static_cast<double>(quantities[index]);
17     }
18 }
19
20 std::cout << std::fixed << std::setprecision(2)
21           << "quote-scan-ok rows=" << prices.size()
22           << " selected=" << selected << " notional=" << notional << '\n';
23 }

```

C.14 第 14 章

1. 若 checksum 不同，两条路径计算的业务函数已不同；耗时差异可能只是少做工作，不能归因于实现优化。
2. 预热减少首次执行效应，重复样本暴露分布，交替顺序减少固定先后偏差；三者都不能消除其他进程、频率变化或不代表生产的输入。
3. checksum 参与分支和最终输出，使计算结果具有可观察影响；优化器仍可内联、向量化、重排满足语言语义的操作，因此还要记录配置并在需要时检查汇编。
4. 中位数都为 10 只描述中心；bursty 组的 IQR 为 37.5、最大/nearest-rank p_{99} 为 100，说明波动与尾部风险远高于 stable 组。
5. IQR 依赖四分位定义，吞吐依赖操作单位、并发和窗口， p_{99} 依赖观测单位与足够样本；没有这些口径，数字不可比较。
6. 8 个样本的 nearest-rank p_{99} 只能取第 8 个、即最大样本；一个新增值就可能大幅改变结果，无法估计 1% 尾部频率。
7. 微基准隔离一个实现假设，profiler 在端到端工作负载中定位时间分布；前者不能替代真实热点证据，后者也不能自动证明改动正确或值得。
8. Amdahl 计算为 $1/(0.4 + 0.6/4) = 1.818\dots$ 。未受影响的 40% 使无限局部加速的整体上限为 $1/0.4 = 2.5$ 。

输出任务没有唯一的性能胜负答案。合格报告应附 `ch14_layout_benchmark` 的全部 21 对原始样本，先确认 `checksum-match=true`，再陈述本机环境下的中心与离散程度；若只运行一个进程，应把结论限制为一次观察。Profiler 部分应保存命令和热点表，并把实测阶段份额代入下面的可运行 Amdahl 模型：

Listing C.8: 第 14 章端到端收益估算答案

```

1  #include <iomanip>
2  #include <iostream>
3
4  int main() {
5      constexpr double baseline_total_ms = 100.0;
6      constexpr double hotspot_ms = 60.0;
7      constexpr double local_speedup = 4.0;
8
9      constexpr double unaffected_ms = baseline_total_ms - hotspot_ms;
10     constexpr double predicted_total_ms =
11         unaffected_ms + hotspot_ms / local_speedup;
12     constexpr double overall_speedup =
13         baseline_total_ms / predicted_total_ms;
14     constexpr double ceiling_speedup = baseline_total_ms / unaffected_ms;

```

```

15
16     std::cout << std::fixed << std::setprecision(2)
17         << "amdahl-ok total-ms=" << baseline_total_ms
18         << " hotspot-ms=" << hotspot_ms
19         << " fraction=" << hotspot_ms / baseline_total_ms
20         << " local-speedup=" << local_speedup
21         << " predicted-total-ms=" << predicted_total_ms
22         << " overall-speedup=" << overall_speedup
23         << " ceiling-speedup=" << ceiling_speedup << '\n';
24 }

```

C.15 第 15 章

1. data race 是未由 happens-before 排序的冲突访问，且至少一个是写；它属于 UB。一般竞态只表示结果依赖合法调度顺序，不必然含有未同步内存冲突。
2. 工作线程完成同步于成功返回的 join，因此线程内写入 happens-before 之后读取；引用捕获还要求被借用对象至少活到 join 完成。
3. atomic 只使各自对象的操作原子化；读取 cash 后、读取 position 前，另一个线程仍可完成一半更新，所以二者不组成同一快照。
4. 条件变量可伪唤醒；wait 必须在同一 mutex 下重查“已关闭或队列非空”等谓词，睡眠时释放锁，返回前重新取得锁。
5. 阻塞、拒绝/重试、丢新、丢旧、合并或溢写都可能合理；依据是数据可丢性、顺序、延迟、内存预算和恢复成本。
6. request_stop 发出协作取消请求，close 宣布不再生产并允许排空，join 等待线程结束并建立完成同步；三者不能互相替代。
7. 未捕获异常逃出线程入口会 terminate；async 把异常存进共享状态，future::get 在调用方重新抛出。
8. 8 行样本只验证结果。性能报告还需 Release 环境、真实工作量、线程数、启动口径、原始样本以及队列阻塞、满和丢弃指标。

核心输出任务的 drop-newest 完整答案如下。五个输入必须满足 offered=accepted+dropped，容量 2 时只保留 10 与 20：

Listing C.9: 第 15 章有界队列 drop-newest 答案

```

1  #include "quant/ch15/bounded_queue.hpp"
2
3  #include <array>
4  #include <iostream>
5  #include <stop_token>
6
7  int main() {
8      using quant::ch15::BoundedQueue;
9      using quant::ch15::PushStatus;
10
11     BoundedQueue<int> queue{2};
12     const std::array offered{10, 20, 30, 40, 50};
13     int accepted = 0;
14     int dropped = 0;

```

```

15  for (const int value : offered) {
16      const PushStatus status = queue.try_push(value);
17      accepted += status == PushStatus::accepted ? 1 : 0;
18      dropped += status == PushStatus::full ? 1 : 0;
19  }
20
21  std::stop_source running;
22  const auto first = queue.wait_pop(running.get_token());
23  const auto second = queue.wait_pop(running.get_token());
24  queue.close();
25  const int drained_sum = first.value.value() + second.value.value();
26  const bool valid = accepted == 2 && dropped == 3 && drained_sum == 30;
27
28  std::cout << "drop-newest-ok offered=" << offered.size()
29              << " accepted=" << accepted << " dropped=" << dropped
30              << " drained-sum=" << drained_sum << '\n';
31  return valid ? 0 : 2;
32 }

```

C.16 第 16 章

1. binary64 不能精确表示 0.1 与 0.2，逐步舍入后不必与单独舍入的 0.3 位级相等；整数 cents 在 int64_t 范围内没有这种表示误差，故可精确比较。
2. 绝对容差保护零附近，相对容差随数值量级缩放。容差应来自报价粒度、输入误差、参考实现精度与允许运算误差，不能只选一个让当前测试通过的常量。
3. Neumaier 的 correction 收集主和没有容纳的低位；它改变算法误差。放大容差只会让错误更难被测试发现，并不能恢复已经丢失的信息。
4. 每个新值先令 count 加一，以与旧 mean 的差更新 mean，再用新旧均值之间的两个差更新平方离差和；样本方差最后除以 $n - 1$ ，少于两个值时拒绝。
5. 金额有最小货币单位和舍入规则，收益率是价格比值，方差单位是收益率平方且有 sample/population 分母。名称、类型与口径必须一起保存。
6. 批量复制支付分配和复制成本，换得独立寿命与目标布局；零拷贝省去复制，却要求 dtype、stride 和源对象寿命一直满足协议。
7. dtype 决定字节解释，shape 决定元素范围，stride 决定逻辑相邻地址，owner 阻止底层缓冲先于视图销毁。任一缺失都可能造成错误值或悬空访问。
8. 释放 GIL 允许其他 Python 线程推进；它不保护 C++ 共享对象，也不允许在释放区间调用 Python API。C++ 同步仍需第 15 章的所有权与 happens-before 协议。

三价格 $100 \rightarrow 125 \rightarrow 100$ 的两期 simple return 为 0.25 与 -0.2，手算均值为 0.025，样本方差为 0.10125。完整答案如下：

Listing C.10: 第 16 章收益率与样本方差答案

```

1  #include "quant/ch16/numerics.hpp"
2
3  #include <array>
4  #include <iomanip>
5  #include <iostream>

```

```

6
7 int main() {
8     constexpr std::array prices{100.0, 125.0, 100.0};
9     std::array<double, prices.size() - 1> returns{};
10    for (std::size_t index = 1; index < prices.size(); ++index) {
11        returns[index - 1] =
12            quant::ch16::simple_return(prices[index - 1], prices[index]);
13    }
14    const quant::ch16::Moments moments =
15        quant::ch16::sample_moments(returns);
16    const quant::ch16::TolerancePolicy tolerance_policy =
17        quant::ch16::tolerance_from_budget(0.0, 24, 1.0);
18    const bool valid =
19        quant::ch16::almost_equal(moments.mean, 0.025,
20                                   tolerance_policy.absolute,
21                                   tolerance_policy.relative) &&
22        quant::ch16::almost_equal(
23            moments.sample_variance, 0.10125, tolerance_policy.absolute,
24            tolerance_policy.relative);
25
26    std::cout << "return-stats-ok periods=" << moments.count << std::fixed
27        << std::setprecision(6) << " mean=" << moments.mean
28        << " sample-variance=" << moments.sample_variance << '\n';
29    return valid ? 0 : 2;
30 }

```

附录 D 推荐阅读与 30 天路线

D.1 推荐阅读

- Bjarne Stroustrup, *A Tour of C++*: 建立现代语言全貌。
- Scott Meyers, *Effective Modern C++*: C++11/14 机制仍是所有权与移动语义基础。
- Anthony Williams, *C++ Concurrency in Action*: 线程、内存模型与并发结构。
- Agner Fog 的优化手册: 指令、微架构与测量, 阅读时结合目标硬件更新资料。
- Brendan Gregg, *Systems Performance*: 系统观测、方法与工具。
- John Lakos 等, *Large-Scale C++*: 依赖、组件和大型代码库边界。
- cppreference: 核对语言与标准库语义; 不要只依赖博客摘要。
- C++ Core Guidelines: 作为审查清单, 结合团队规则而非机械执行。

D.2 第 1 周: 语言与所有权

日	任务
1	配置 CMake/GCC/Clang, 构建第 1 章示例, 解释编译与链接。
2	完成类型、初始化与窄化练习。
3	练习引用、 <code>const</code> 、值类别与移动。
4	用 RAII 封装文件和计时器。
5	比较值成员、 <code>unique pointer</code> 与 <code>shared pointer</code> 。
6	完成 <code>vector</code> 、算法和 <code>ranges</code> 数据管道。
7	复盘四章面试检查点, 不看书口述答案。

D.3 第 2 周: 接口与可靠性

第 8–9 天设计事件、订单、成交和组合不变量; 第 10 天练习模板与 `concepts`; 第 11 天完成 CSV 错误测试; 第 12 天运行四个 `seam`; 第 13 天添加 `sanitizer` 构建; 第 14 天从干净目录重新构建并写一页架构说明。

D.4 第 3 周: 性能与并发

第 15 天学习缓存与布局; 第 16 天重复布局基准并做统计; 第 17 天使用 `profiler`; 第 18 天复习浮点误差; 第 19 天实现稳定归约; 第 20 天画出并发所有权图; 第 21 天实现一个有界队列设计文档并说明背压, 不急于写无锁代码。

D.5 第 4 周: 项目与求职

第 22–24 天为回测项目增加一个经 TDD 驱动的真实特性; 第 25 天完善 README 和复现命令; 第 26 天写性能报告; 第 27 天准备语言问答; 第 28 天做系统设计模拟; 第 29 天录制项目讲解; 第 30 天完成一次全流程模拟面试并制定下一月计划。

D.6 完成判据

30 天后，不以“读完章节”为成功，而以以下证据验收：能从干净目录构建；能解释四个 seam；能手算一个回测结果；能展示一次红绿循环；能提供带环境和限制的性能报告；能画出系统边界、背压和停止协议；能诚实陈述项目非目标。

参考文献

- [1] *cppreference.com*. URL: <https://en.cppreference.com/> (visited on 07/11/2026).
- [2] Brendan Gregg. *Systems Performance: Enterprise and the Cloud*. 2nd ed. Addison-Wesley, 2020.
- [3] Scott Meyers. *Effective Modern C++*. O'Reilly Media, 2014.
- [4] Bjarne Stroustrup. *A Tour of C++*. 3rd ed. Addison-Wesley, 2022.
- [5] Bjarne Stroustrup and Herb Sutter. *C++ Core Guidelines*. URL: <https://isocpp.github.io/CppCoreGuidelines/> (visited on 07/11/2026).
- [6] Anthony Williams. *C++ Concurrency in Action*. 2nd ed. Manning, 2019.

模板与许可说明

本书使用 ElegantBook 模板排版，项目主页如下：

<https://github.com/ElegantLaTeX/ElegantBook>

类文件依照 LaTeX Project Public License 1.3c 或更高版本发布。模板许可文本随项目保存在独立文件中：

ELEGANTBOOK-LICENSE